

Competitive Programmer's Handbook

Antti Laaksonen

Vietnamese edition prepared by

Ho Thien Phuc 🇻🇳

Mục lục

I	Kỹ thuật cơ bản	1
1	Giới thiệu	3
1.1	Ngôn ngữ lập trình	3
1.2	Đầu vào và đầu ra	4
1.3	Làm việc với các con số	6
1.4	Rút ngắn code	8
1.5	Toán học	10
1.6	Các kỳ thi và tài nguyên	15
2	Time complexity	17
2.1	Calculation rules	17
2.2	Complexity classes	20
2.3	Estimating efficiency	21
2.4	Maximum subarray sum	22
3	Sắp xếp	25
3.1	Lý thuyết sắp xếp	25
3.2	Sắp xếp trong C++	29
3.3	Tìm kiếm nhị phân	31
4	Cấu trúc dữ liệu	35
4.1	Mảng động	35
4.2	Cấu trúc tập hợp	37
4.3	Cấu trúc ánh xạ	38
4.4	Bộ lặp và đoạn	39
4.5	Các cấu trúc khác	41
4.6	So sánh với sắp xếp	44
5	Duyệt vét cạn	47
5.1	Sinh tập con	47
5.2	Sinh hoán vị	49
5.3	Quay lui	50
5.4	Cắt tỉa cây tìm kiếm	51
5.5	Gặp nhau ở giữa	54

6	Thuật toán tham lam	57
6.1	Bài toán đồng xu	57
6.2	Lập lịch	58
6.3	Công việc và hạn chót	60
6.4	Tối thiểu hóa tổng	61
6.5	Nén dữ liệu	62
7	Quy hoạch động	65
7.1	Bài toán đồng xu	65
7.2	Dãy con tăng dài nhất	70
7.3	Đường đi trong lưới	71
7.4	Bài toán cái túi	72
7.5	Khoảng cách chỉnh sửa	74
7.6	Đếm lát phủ	75
8	Phân tích khấu hao	77
8.1	Phương pháp hai con trỏ	77
8.2	Phần tử nhỏ hơn gần nhất	79
8.3	Cực tiểu của số trượt	81
9	Truy vấn đoạn	83
9.1	Truy vấn trên mảng tĩnh	84
9.2	Cây chỉ số nhị phân	86
9.3	Cây phân đoạn	89
9.4	Các kỹ thuật bổ sung	92
10	Thao tác bit	95
10.1	Biểu diễn bit	95
10.2	Thao tác bit	96
10.3	Biểu diễn tập hợp	98
10.4	Tối ưu hóa bằng bit	100
10.5	Quy hoạch động	101
II	Thuật toán đồ thị	107
11	Cơ bản về đồ thị	109
11.1	Thuật ngữ đồ thị	109
11.2	Biểu diễn đồ thị	113
12	Duyệt đồ thị	117
12.1	Tìm kiếm theo chiều sâu	117
12.2	Tìm kiếm theo chiều rộng	119
12.3	Ứng dụng	121

13 Đường đi ngắn nhất	125
13.1 Thuật toán Bellman–Ford	125
13.2 Thuật toán Dijkstra	128
13.3 Thuật toán Floyd–Warshall	131
14 Thuật toán trên cây	135
14.1 Duyệt cây	136
14.2 Đường kính	137
14.3 Tất cả các đường đi dài nhất	139
14.4 Cây nhị phân	141
Tài liệu tham khảo	143

Phần I

Kỹ thuật cơ bản

Chương 1

Giới thiệu

Lập trình thi đấu (competitive programming) bao gồm hai chủ đề: (1) thiết kế thuật toán và (2) cài đặt thuật toán.

Thiết kế thuật toán đòi hỏi khả năng giải quyết vấn đề và tư duy toán học. Cần có kỹ năng phân tích bài toán và giải quyết chúng một cách sáng tạo. Một thuật toán giải một bài toán phải vừa đúng đắn vừa hiệu quả, và phần cốt lõi của bài toán thường nằm ở việc nghĩ ra được một thuật toán hiệu quả.

Kiến thức lý thuyết về thuật toán rất quan trọng đối với người lập trình thi đấu. Thông thường, một lời giải cho một bài toán là sự kết hợp giữa các kỹ thuật đã biết và những ý tưởng mới. Các kỹ thuật xuất hiện trong lập trình thi đấu cũng tạo nên nền tảng cho nghiên cứu khoa học về thuật toán.

Cài đặt thuật toán đòi hỏi kỹ năng lập trình tốt. Trong lập trình thi đấu, các lời giải được chấm bằng cách kiểm thử thuật toán đã cài đặt trên một tập các bộ test. Vì vậy, chỉ ý tưởng đúng thôi là chưa đủ; phần cài đặt cũng phải chính xác.

Một phong cách code tốt trong các kỳ thi là rõ ràng và súc tích. Chương trình nên được viết nhanh, vì thời gian có hạn. Khác với kỹ nghệ phần mềm truyền thống, chương trình ở đây ngắn (thường nhiều nhất là vài trăm dòng code) và không cần được bảo trì sau kỳ thi.

1.1 Ngôn ngữ lập trình

Tại thời điểm hiện nay, các ngôn ngữ lập trình phổ biến nhất được dùng trong các kỳ thi là C++, Python và Java. Ví dụ, trong Google Code Jam 2017, trong số 3.000 thí sinh xuất sắc nhất, 79% dùng C++, 16% dùng Python và 8% dùng Java [29]. Một số thí sinh còn dùng nhiều ngôn ngữ.

Nhiều người cho rằng C++ là lựa chọn tốt nhất cho người lập trình thi đấu, và C++ gần như luôn có sẵn trên các hệ thống thi. Lợi ích của việc dùng C++ là nó là một ngôn ngữ rất hiệu quả và thư viện chuẩn chứa một bộ sưu tập lớn các cấu trúc dữ liệu (data structures) và thuật toán (algorithms).

Mặt khác, việc thông thạo nhiều ngôn ngữ và hiểu rõ điểm mạnh của từng ngôn ngữ cũng có ích. Ví dụ, nếu bài toán cần đến số nguyên lớn, Python có thể là một lựa chọn tốt vì nó có sẵn các phép toán với số nguyên lớn. Tuy nhiên, hầu hết các bài toán trong các kỳ thi lập trình đều được ra sao cho việc dùng một ngôn ngữ cụ thể không

tạo ra lợi thế bất công.

Tất cả các chương trình ví dụ trong cuốn sách này được viết bằng C++, và các cấu trúc dữ liệu cùng thuật toán của thư viện chuẩn thường được sử dụng. Các chương trình tuân theo chuẩn C++11, chuẩn này hiện nay có thể dùng được ở hầu hết các kỳ thi. Nếu bạn vẫn chưa thể lập trình bằng C++, bây giờ là thời điểm tốt để bắt đầu học.

Mẫu code C++

Một mẫu code C++ điển hình cho lập trình thi đấu trông như sau:

```
#include <bits/stdc++.h>

using namespace std;

int main() {
    // solution comes here
}
```

Dòng `#include` ở đầu đoạn code là một tính năng của trình biên dịch `g++` cho phép ta nạp toàn bộ thư viện chuẩn. Như vậy, không cần phải nạp riêng từng thư viện như `iostream`, `vector` và `algorithm`; chúng đã có sẵn một cách tự động.

Dòng `using` khai báo rằng các lớp và hàm của thư viện chuẩn có thể được dùng trực tiếp trong code. Không có dòng `using`, ta sẽ phải viết, chẳng hạn, `std::cout`; nhưng nhờ nó, chỉ cần viết `cout` là đủ.

Code có thể được biên dịch bằng lệnh sau:

```
g++ -std=c++11 -O2 -Wall test.cpp -o test
```

Lệnh này tạo ra một file nhị phân `test` từ mã nguồn `test.cpp`. Trình biên dịch tuân theo chuẩn C++11 (`-std=c++11`), tối ưu hóa code (`-O2`) và hiển thị cảnh báo về các lỗi tiềm ẩn (`-Wall`).

1.2 Đầu vào và đầu ra

Trong hầu hết các kỳ thi, các luồng chuẩn (standard streams) được dùng để đọc đầu vào và ghi đầu ra. Trong C++, các luồng chuẩn là `cin` cho đầu vào và `cout` cho đầu ra. Ngoài ra, các hàm trong C `scanf` và `printf` cũng có thể được sử dụng.

Đầu vào của chương trình thường gồm các số và các chuỗi được tách bởi khoảng trắng và dấu xuống dòng. Chúng có thể được đọc từ luồng `cin` như sau:

```
int a, b;
string x;
cin >> a >> b >> x;
```

Đoạn code kiểu này luôn hoạt động, giả sử có ít nhất một khoảng trắng hoặc dấu xuống dòng giữa các phân tử trong đầu vào. Ví dụ, đoạn code trên có thể đọc được cả hai đầu vào sau:

```
123 456 monkey
```

```
123    456  
monkey
```

Luồng cout được dùng cho đầu ra như sau:

```
int a = 123, b = 456;  
string x = "monkey";  
cout << a << " " << b << " " << x << "\n";
```

Đôi khi đầu vào và đầu ra là điểm nghẽn (bottleneck) của chương trình. Các dòng sau ở đầu đoạn code giúp đầu vào và đầu ra hiệu quả hơn:

```
ios::sync_with_stdio(0);  
cin.tie(0);
```

Lưu ý rằng dấu xuống dòng "\n" chạy nhanh hơn endl, vì endl luôn gây ra một thao tác xả bộ đệm (flush).

Các hàm C scanf và printf là một lựa chọn thay thế cho các luồng chuẩn của C++. Chúng thường nhanh hơn một chút, nhưng cũng khó dùng hơn. Đoạn code sau đọc hai số nguyên từ đầu vào:

```
int a, b;  
scanf("%d %d", &a, &b);
```

Đoạn code sau in ra hai số nguyên:

```
int a = 123, b = 456;  
printf("%d %d\n", a, b);
```

Đôi khi chương trình cần đọc cả một dòng từ đầu vào, có thể chứa các khoảng trắng. Việc này có thể thực hiện bằng cách dùng hàm getline:

```
string s;  
getline(cin, s);
```

Nếu không biết trước lượng dữ liệu, vòng lặp sau đây sẽ hữu ích:

```
while (cin >> x) {  
    // code  
}
```

Vòng lặp này đọc các phần tử từ đầu vào lần lượt cho đến khi không còn dữ liệu nào trong đầu vào.

Ở một số hệ thống thi, file được dùng để đọc đầu vào và ghi đầu ra. Một cách giải quyết đơn giản cho việc này là viết code như bình thường dùng các luồng chuẩn, nhưng thêm các dòng sau vào đầu code:

```
freopen("input.txt", "r", stdin);
freopen("output.txt", "w", stdout);
```

Sau đó, chương trình sẽ đọc đầu vào từ file "input.txt" và ghi đầu ra ra file "output.txt".

1.3 Làm việc với các con số

Số nguyên

Kiểu số nguyên (integer) được dùng nhiều nhất trong lập trình thi đấu là `int`, một kiểu 32-bit với miền giá trị $-2^{31} \dots 2^{31} - 1$ hay khoảng $-2 \cdot 10^9 \dots 2 \cdot 10^9$. Nếu kiểu `int` không đủ, ta có thể dùng kiểu 64-bit `long long`. Kiểu này có miền giá trị $-2^{63} \dots 2^{63} - 1$ hay khoảng $-9 \cdot 10^{18} \dots 9 \cdot 10^{18}$.

Đoạn code sau định nghĩa một biến `long long`:

```
long long x = 123456789123456789LL;
```

Hậu tố `LL` nghĩa là kiểu của số này là `long long`.

Một lỗi thường gặp khi dùng kiểu `long long` là kiểu `int` vẫn còn được dùng ở đâu đó trong code. Ví dụ, đoạn code sau chứa một lỗi tinh vi:

```
int a = 123456789;
long long b = a*a;
cout << b << "\n"; // -1757895751
```

Mặc dù biến `b` có kiểu `long long`, cả hai số trong biểu thức `a*a` đều có kiểu `int` và kết quả cũng có kiểu `int`. Vì vậy, biến `b` sẽ chứa một kết quả sai. Lỗi này có thể khắc phục bằng cách đổi kiểu của `a` sang `long long` hoặc đổi biểu thức thành `(long long)a*a`.

Thông thường, các bài thi được ra sao cho kiểu `long long` là đủ. Tuy vậy, cũng nên biết rằng trình biên dịch `g++` còn cung cấp một kiểu 128-bit `__int128_t` với miền giá trị $-2^{127} \dots 2^{127} - 1$ hay khoảng $-10^{38} \dots 10^{38}$. Tuy nhiên, kiểu này không có sẵn ở mọi hệ thống thi.

Số học modulo

Ta ký hiệu $x \bmod m$ là phần dư khi x chia cho m . Ví dụ, $17 \bmod 5 = 2$, vì $17 = 3 \cdot 5 + 2$.

Đôi khi, đáp số của một bài toán là một số rất lớn nhưng chỉ cần in ra "modulo m ", tức là phần dư khi đáp số chia cho m (ví dụ, "modulo $10^9 + 7$ "). Ý tưởng là dù đáp số thật rất lớn, vẫn chỉ cần dùng các kiểu `int` và `long long`.

Một tính chất quan trọng của phép lấy dư là với phép cộng, trừ và nhân, phép lấy dư có thể được thực hiện trước phép toán:

$$\begin{aligned}(a + b) \bmod m &= (a \bmod m + b \bmod m) \bmod m \\(a - b) \bmod m &= (a \bmod m - b \bmod m) \bmod m \\(a \cdot b) \bmod m &= (a \bmod m \cdot b \bmod m) \bmod m\end{aligned}$$

Do đó, ta có thể lấy dư sau mỗi phép toán và các con số sẽ không bao giờ trở nên quá lớn.

Ví dụ, đoạn code sau tính $n!$, giai thừa (factorial) của n , theo modulo m :

```
long long x = 1;
for (int i = 2; i <= n; i++) {
    x = (x*i)%m;
}
cout << x%m << "\n";
```

Thường ta muốn phần dư luôn nằm trong khoảng $0 \dots m - 1$. Tuy nhiên, trong C++ và một số ngôn ngữ khác, phần dư của một số âm hoặc bằng 0, hoặc âm. Một cách đơn giản để đảm bảo không có phần dư âm là tính phần dư như bình thường rồi cộng thêm m nếu kết quả là âm:

```
x = x%m;
if (x < 0) x += m;
```

Tuy nhiên, việc này chỉ cần thiết khi trong code có phép trừ và phần dư có thể trở thành âm.

Số dấu chấm động

Các kiểu số dấu chấm động thường dùng trong lập trình thi đấu là kiểu 64-bit double và, như một phần mở rộng trong trình biên dịch g++, kiểu 80-bit long double. Trong hầu hết trường hợp, double là đủ, nhưng long double chính xác hơn.

Độ chính xác cần thiết của đáp số thường được ghi trong đề bài. Một cách đơn giản để in đáp số là dùng hàm printf và chỉ định số chữ số thập phân trong chuỗi định dạng. Ví dụ, đoạn code sau in ra giá trị của x với 9 chữ số thập phân:

```
printf("%.9f\n", x);
```

Một khó khăn khi dùng số dấu chấm động là một số con số không thể biểu diễn chính xác dưới dạng số dấu chấm động, và sẽ có sai số làm tròn. Ví dụ, kết quả của đoạn code sau khá bất ngờ:

```
double x = 0.3*3+0.1;
printf("%.20f\n", x); // 0.99999999999999988898
```

Do sai số làm tròn, giá trị của x nhỏ hơn 1 một chút, trong khi giá trị đúng phải là 1.

So sánh các số dấu chấm động bằng toán tử $==$ là rủi ro, vì có thể các giá trị lẽ ra phải bằng nhau nhưng lại không bằng do sai số làm tròn. Một cách tốt hơn để so sánh các số dấu chấm động là xem hai số là bằng nhau nếu hiệu của chúng nhỏ hơn ε , với ε là một số rất nhỏ.

Trên thực tế, các số có thể được so sánh như sau ($\varepsilon = 10^{-9}$):

```
if (abs(a-b) < 1e-9) {
```

```
} // a and b are equal
```

Lưu ý rằng dù số dấu chấm động không chính xác, các số nguyên đến một giới hạn nhất định vẫn có thể được biểu diễn chính xác. Ví dụ, dùng `double`, ta có thể biểu diễn chính xác mọi số nguyên có trị tuyệt đối nhiều nhất bằng 2^{53} .

1.4 Rút ngắn code

Code ngắn là lý tưởng trong lập trình thi đấu, vì các chương trình cần được viết càng nhanh càng tốt. Vì vậy, người lập trình thi đấu thường định nghĩa các tên ngắn hơn cho kiểu dữ liệu và các thành phần khác của code.

Tên kiểu

Dùng lệnh `typedef` ta có thể đặt một tên ngắn hơn cho một kiểu dữ liệu. Ví dụ, tên `long long` dài, nên ta có thể định nghĩa tên ngắn hơn là `ll`:

```
typedef long long ll ;
```

Sau đó, đoạn code

```
long long a = 123456789;
long long b = 987654321;
cout << a*b << "\n";
```

có thể rút gọn thành:

```
ll a = 123456789;
ll b = 987654321;
cout << a*b << "\n";
```

Lệnh `typedef` cũng có thể được dùng với các kiểu phức tạp hơn. Ví dụ, đoạn code sau đặt tên `vi` cho một vector các số nguyên và tên `pi` cho một cặp chứa hai số nguyên.

```
typedef vector<int> vi;
typedef pair<int,int> pi;
```

Macro

Một cách khác để rút ngắn code là định nghĩa **macro**. Một macro nghĩa là một số chuỗi nhất định trong code sẽ được thay thế trước khi biên dịch. Trong C++, macro được định nghĩa bằng từ khóa `#define`.

Ví dụ, ta có thể định nghĩa các macro sau:

```
#define F first
```

```
#define S second
#define PB push_back
#define MP make_pair
```

Sau đó, đoạn code

```
v.push_back(make_pair(y1,x1));
v.push_back(make_pair(y2,x2));
int d = v[i].first + v[i].second;
```

có thể rút gọn thành:

```
v.PB(MP(y1,x1));
v.PB(MP(y2,x2));
int d = v[i].F + v[i].S;
```

Một macro cũng có thể có tham số, cho phép rút ngắn vòng lặp và các cấu trúc khác. Ví dụ, ta có thể định nghĩa macro sau:

```
#define REP(i,a,b) for (int i = a; i <= b; i++)
```

Sau đó, đoạn code

```
for (int i = 1; i <= n; i++) {
    search(i);
}
```

có thể rút gọn thành:

```
REP(i,1,n) {
    search(i);
}
```

Đôi khi macro gây ra các lỗi khó phát hiện. Ví dụ, xét macro sau dùng để tính bình phương của một số:

```
#define SQ(a) a*a
```

Macro này *không* luôn hoạt động như kỳ vọng. Ví dụ, đoạn code

```
cout << SQ(3+3) << "\n";
```

tương ứng với đoạn code

```
cout << 3+3*3+3 << "\n"; // 15
```

Một phiên bản tốt hơn của macro là:

```
#define SQ(a) (a)*(a)
```

Bây giờ, đoạn code

```
cout << SQ(3+3) << "\n";
```

tương ứng với đoạn code

```
cout << (3+3)*(3+3) << "\n"; // 36
```

1.5 Toán học

Toán học đóng một vai trò quan trọng trong lập trình thi đấu, và không thể trở thành một người lập trình thi đấu thành công nếu không có kỹ năng toán học tốt. Phần này thảo luận một số khái niệm và công thức toán học quan trọng sẽ cần đến ở các chương sau.

Công thức tổng

Mọi tổng dạng

$$\sum_{x=1}^n x^k = 1^k + 2^k + 3^k + \dots + n^k,$$

trong đó k là một số nguyên dương, đều có một công thức dạng đóng là một đa thức (polynomial) bậc $k + 1$. Ví dụ¹,

$$\sum_{x=1}^n x = 1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2}$$

và

$$\sum_{x=1}^n x^2 = 1^2 + 2^2 + 3^2 + \dots + n^2 = \frac{n(n+1)(2n+1)}{6}.$$

Cấp số cộng (arithmetic progression) là một dãy số trong đó hiệu giữa hai số liên tiếp bất kỳ là một hằng số. Ví dụ,

$$3, 7, 11, 15$$

là một cấp số cộng với công sai 4. Tổng của một cấp số cộng có thể được tính bằng công thức

$$\underbrace{a + \dots + b}_{n \text{ numbers}} = \frac{n(a+b)}{2}$$

trong đó a là số đầu tiên, b là số cuối cùng và n là số lượng số hạng. Ví dụ,

$$3 + 7 + 11 + 15 = \frac{4 \cdot (3 + 15)}{2} = 36.$$

¹ Thực ra có một công thức tổng quát cho những tổng như vậy, gọi là **công thức Faulhaber** (Faulhaber's formula), nhưng nó quá phức tạp để trình bày ở đây.

Công thức này dựa trên thực tế rằng tổng gồm n số và giá trị trung bình của mỗi số là $(a + b)/2$.

Cấp số nhân (geometric progression) là một dãy số trong đó tỉ số giữa hai số liên tiếp bất kỳ là một hằng số. Ví dụ,

$$3, 6, 12, 24$$

là một cấp số nhân với công bội 2. Tổng của một cấp số nhân có thể được tính bằng công thức

$$a + ak + ak^2 + \dots + b = \frac{bk - a}{k - 1}$$

trong đó a là số đầu tiên, b là số cuối cùng và tỉ số giữa hai số liên tiếp là k . Ví dụ,

$$3 + 6 + 12 + 24 = \frac{24 \cdot 2 - 3}{2 - 1} = 45.$$

Công thức này có thể được suy ra như sau. Đặt

$$S = a + ak + ak^2 + \dots + b.$$

Nhân cả hai vế với k , ta được

$$kS = ak + ak^2 + ak^3 + \dots + bk,$$

và giải phương trình

$$kS - S = bk - a$$

cho ta công thức đó.

Một trường hợp đặc biệt của tổng cấp số nhân là công thức

$$1 + 2 + 4 + 8 + \dots + 2^{n-1} = 2^n - 1.$$

Tổng điều hòa (harmonic sum) là tổng dạng

$$\sum_{x=1}^n \frac{1}{x} = 1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n}.$$

Một cận trên cho tổng điều hòa là $\log_2(n) + 1$. Cụ thể, ta có thể sửa mỗi số hạng $1/k$ sao cho k trở thành lũy thừa của 2 gần nhất không vượt quá k . Ví dụ, khi $n = 6$, ta có thể ước lượng tổng như sau:

$$1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \frac{1}{5} + \frac{1}{6} \leq 1 + \frac{1}{2} + \frac{1}{2} + \frac{1}{4} + \frac{1}{4} + \frac{1}{4}.$$

Cận trên này gồm $\log_2(n) + 1$ phần ($1, 2 \cdot 1/2, 4 \cdot 1/4$, v.v.), và giá trị của mỗi phần nhiều nhất bằng 1.

Lý thuyết tập hợp

Tập hợp (set) là một bộ sưu tập các phần tử. Ví dụ, tập hợp

$$X = \{2, 4, 7\}$$

chứa các phần tử 2, 4 và 7. Ký hiệu $\emptyset$ chỉ tập rỗng, và $|S|$ chỉ kích thước của tập S , tức là số phần tử của tập đó. Ví dụ, trong tập hợp ở trên, $|X| = 3$.

Nếu tập S chứa phần tử x , ta viết $x \in S$; ngược lại ta viết $x \notin S$. Ví dụ, trong tập hợp ở trên,

$$4 \in X \quad \text{và} \quad 5 \notin X.$$

Có thể tạo tập hợp mới bằng các phép toán trên tập hợp:

- **Giao** (intersection) $A \cap B$ gồm các phần tử thuộc cả A và B . Ví dụ, nếu $A = \{1, 2, 5\}$ và $B = \{2, 4\}$, thì $A \cap B = \{2\}$.
- **Hợp** (union) $A \cup B$ gồm các phần tử thuộc A hoặc B hoặc cả hai. Ví dụ, nếu $A = \{3, 7\}$ và $B = \{2, 3, 8\}$, thì $A \cup B = \{2, 3, 7, 8\}$.
- **Phần bù** (complement) $\bar{A}$ gồm các phần tử không thuộc A . Cách diễn giải của phần bù phụ thuộc vào **tập phổ dụng** (universal set), chứa tất cả các phần tử có thể có. Ví dụ, nếu $A = \{1, 2, 5, 7\}$ và tập phổ dụng là $\{1, 2, \dots, 10\}$, thì $\bar{A} = \{3, 4, 6, 8, 9, 10\}$.
- **Hiệu** (difference) $A \setminus B = A \cap \bar{B}$ gồm các phần tử thuộc A nhưng không thuộc B . Lưu ý rằng B có thể chứa các phần tử không thuộc A . Ví dụ, nếu $A = \{2, 3, 7, 8\}$ và $B = \{3, 5, 8\}$, thì $A \setminus B = \{2, 7\}$.

Nếu mọi phần tử của A cũng thuộc S , ta nói A là **tập con** (subset) của S , ký hiệu $A \subset S$. Một tập S luôn có $2^{|S|}$ tập con, kể cả tập rỗng. Ví dụ, các tập con của tập $\{2, 4, 7\}$ là

$$\emptyset, \{2\}, \{4\}, \{7\}, \{2, 4\}, \{2, 7\}, \{4, 7\} \text{ và } \{2, 4, 7\}.$$

Một số tập hợp thường gặp là $\mathbb{N}$ (số tự nhiên), $\mathbb{Z}$ (số nguyên), $\mathbb{Q}$ (số hữu tỉ) và $\mathbb{R}$ (số thực). Tập $\mathbb{N}$ có thể được định nghĩa theo hai cách, tùy ngữ cảnh: hoặc $\mathbb{N} = \{0, 1, 2, \dots\}$ hoặc $\mathbb{N} = \{1, 2, 3, \dots\}$.

Ta cũng có thể xây dựng tập hợp bằng một luật dạng

$$\{f(n) : n \in S\},$$

trong đó $f(n)$ là một hàm nào đó. Tập này chứa mọi phần tử dạng $f(n)$, với n là một phần tử trong S . Ví dụ, tập

$$X = \{2n : n \in \mathbb{Z}\}$$

chứa mọi số nguyên chẵn.

Logic

Giá trị của một biểu thức logic hoặc là **đúng** (true, 1) hoặc **sai** (false, 0). Các toán tử logic quan trọng nhất là $\neg$ (**phủ định**, negation), $\wedge$ (**hội**, conjunction), $\vee$ (**tuyển**, disjunction), $\Rightarrow$ (**kéo theo**, implication) và $\Leftrightarrow$ (**tương đương**, equivalence). Bảng dưới đây cho thấy ý nghĩa của các toán tử này:

A	B	$\neg A$	$\neg B$	$A \wedge B$	$A \vee B$	$A \Rightarrow B$	$A \Leftrightarrow B$
0	0	1	1	0	0	1	1
0	1	1	0	0	1	1	0
1	0	0	1	0	1	0	0
1	1	0	0	1	1	1	1

Biểu thức $\neg A$ có giá trị ngược với A . Biểu thức $A \wedge B$ đúng khi và chỉ khi cả A và B cùng đúng, còn biểu thức $A \vee B$ đúng khi A hoặc B hoặc cả hai đúng. Biểu thức $A \Rightarrow B$ đúng nếu mỗi khi A đúng thì B cũng đúng. Biểu thức $A \Leftrightarrow B$ đúng nếu A và B cùng đúng hoặc cùng sai.

Vị từ (predicate) là một biểu thức đúng hoặc sai tùy theo các tham số của nó. Vị từ thường được ký hiệu bằng chữ hoa. Ví dụ, ta có thể định nghĩa một vị từ $P(x)$ đúng khi và chỉ khi x là một số nguyên tố (prime number). Theo định nghĩa này, $P(7)$ đúng còn $P(8)$ sai.

Lượng từ (quantifier) gắn một biểu thức logic với các phần tử của một tập hợp. Các lượng từ quan trọng nhất là $\forall$ (**với mọi**) và $\exists$ (**tồn tại**). Ví dụ,

$$\forall x(\exists y(y < x))$$

nghĩa là với mỗi phần tử x trong tập hợp, tồn tại một phần tử y trong tập hợp sao cho y nhỏ hơn x . Mệnh đề này đúng trong tập các số nguyên, nhưng sai trong tập các số tự nhiên.

Dùng ký hiệu mô tả ở trên, ta có thể diễn đạt nhiều loại mệnh đề logic. Ví dụ,

$$\forall x((x > 1 \wedge \neg P(x)) \Rightarrow (\exists a(\exists b(a > 1 \wedge b > 1 \wedge x = ab))))$$

nghĩa là nếu một số x lớn hơn 1 và không phải số nguyên tố, thì tồn tại các số a và b lớn hơn 1 mà tích của chúng bằng x . Mệnh đề này đúng trong tập các số nguyên.

Hàm

Hàm $\lfloor x \rfloor$ làm tròn số x xuống thành một số nguyên, và hàm $\lceil x \rceil$ làm tròn số x lên thành một số nguyên. Ví dụ,

$$\lfloor 3/2 \rfloor = 1 \quad \text{và} \quad \lceil 3/2 \rceil = 2.$$

Các hàm $\min(x_1, x_2, \dots, x_n)$ và $\max(x_1, x_2, \dots, x_n)$ cho giá trị nhỏ nhất và lớn nhất trong $x_1, x_2, \dots, x_n$. Ví dụ,

$$\min(1, 2, 3) = 1 \quad \text{và} \quad \max(1, 2, 3) = 3.$$

Giai thừa $n!$ có thể được định nghĩa

$$\prod_{x=1}^n x = 1 \cdot 2 \cdot 3 \cdot \dots \cdot n$$

hoặc đệ quy (recursion)

$$\begin{aligned} 0! &= 1 \\ n! &= n \cdot (n-1)! \end{aligned}$$

Số Fibonacci xuất hiện trong nhiều tình huống. Chúng có thể được định nghĩa đệ quy như sau:

$$\begin{aligned} f(0) &= 0 \\ f(1) &= 1 \\ f(n) &= f(n-1) + f(n-2) \end{aligned}$$

Các số Fibonacci đầu tiên là

$$0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, \dots$$

Cũng có một công thức dạng đóng để tính số Fibonacci, đôi khi gọi là **công thức Binet** (Binet's formula):

$$f(n) = \frac{(1 + \sqrt{5})^n - (1 - \sqrt{5})^n}{2^n \sqrt{5}}.$$

Logarit

Logarit (logarithm) của một số x được ký hiệu $\log_k(x)$, trong đó k là cơ số của logarit. Theo định nghĩa, $\log_k(x) = a$ khi và chỉ khi $k^a = x$.

Một tính chất hữu ích của logarit là $\log_k(x)$ bằng số lần ta phải chia x cho k trước khi đạt đến số 1. Ví dụ, $\log_2(32) = 5$ vì cần 5 lần chia cho 2:

$$32 \rightarrow 16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$$

Logarit thường được dùng trong phân tích thuật toán, vì nhiều thuật toán hiệu quả chia đôi một thứ gì đó ở mỗi bước. Do đó, ta có thể ước lượng hiệu quả của các thuật toán này bằng logarit.

Logarit của một tích là

$$\log_k(ab) = \log_k(a) + \log_k(b),$$

và do đó,

$$\log_k(x^n) = n \cdot \log_k(x).$$

Ngoài ra, logarit của một thương là

$$\log_k\left(\frac{a}{b}\right) = \log_k(a) - \log_k(b).$$

Một công thức hữu ích khác là

$$\log_u(x) = \frac{\log_k(x)}{\log_k(u)},$$

nhờ đó, có thể tính logarit ở cơ số bất kỳ nếu có cách tính logarit ở một cơ số cố định nào đó.

Logarit tự nhiên (natural logarithm) $\ln(x)$ của một số x là logarit có cơ số $e \approx 2,71828$. Một tính chất nữa của logarit là số chữ số của một số nguyên x trong cơ số b bằng $\lfloor \log_b(x) + 1 \rfloor$. Ví dụ, biểu diễn của 123 trong cơ số 2 là 1111011 và $\lfloor \log_2(123) + 1 \rfloor = 7$.

1.6 Các kỳ thi và tài nguyên

IOI

Olympic Tin học Quốc tế (International Olympiad in Informatics, IOI) là một cuộc thi lập trình thường niên dành cho học sinh trung học. Mỗi quốc gia được phép cử một đội gồm bốn học sinh đến tham dự. Thông thường có khoảng 300 thí sinh đến từ 80 quốc gia.

IOI gồm hai phần thi mỗi phần kéo dài năm giờ. Trong cả hai phần thi, thí sinh phải giải ba bài toán về thuật toán với độ khó khác nhau. Mỗi bài được chia thành các bài con (subtask), mỗi bài con có một số điểm được quy định sẵn. Mặc dù thí sinh được chia thành các đội, họ thi đấu với tư cách cá nhân.

Đề cương IOI [41] quy định các chủ đề có thể xuất hiện trong các bài thi IOI. Hầu hết các chủ đề trong đề cương IOI đều được trình bày trong cuốn sách này.

Thí sinh dự IOI được chọn qua các kỳ thi quốc gia. Trước IOI, nhiều kỳ thi khu vực được tổ chức, chẳng hạn như Olympic Tin học vùng Baltic (BOI), Olympic Tin học Trung Âu (CEOI) và Olympic Tin học châu Á - Thái Bình Dương (APIO).

Một số quốc gia tổ chức các kỳ thi luyện tập trực tuyến cho các thí sinh IOI tương lai, chẳng hạn như Croatian Open Competition in Informatics [11] và USA Computing Olympiad [68]. Ngoài ra, một bộ sưu tập lớn các bài toán từ các kỳ thi Ba Lan có sẵn trực tuyến [60].

ICPC

International Collegiate Programming Contest (ICPC) là một cuộc thi lập trình thường niên dành cho sinh viên đại học. Mỗi đội trong cuộc thi gồm ba sinh viên, và khác với IOI, các sinh viên hợp tác với nhau; mỗi đội chỉ có một máy tính.

ICPC gồm nhiều vòng, và cuối cùng các đội xuất sắc nhất được mời đến Vòng chung kết Thế giới (World Finals). Mặc dù có hàng chục ngàn thí sinh tham dự cuộc thi, chỉ có một số ít² suất vào chung kết, nên ngay cả việc lọt vào vòng chung kết đã là một thành tích lớn ở một số khu vực.

Trong mỗi kỳ thi ICPC, các đội có năm giờ để giải khoảng mười bài toán thuật toán. Một lời giải cho một bài toán chỉ được chấp nhận nếu nó giải mọi bộ test một cách hiệu quả. Trong suốt cuộc thi, thí sinh có thể xem kết quả của các đội khác, nhưng vào giờ cuối, bảng điểm bị đóng băng và không thể xem kết quả của các lần nộp cuối cùng.

²Số suất chung kết cụ thể thay đổi qua các năm; vào năm 2017, có 133 suất chung kết.

Các chủ đề có thể xuất hiện ở ICPC không được quy định rõ ràng như ở IOI. Dù vậy, rõ ràng là cần nhiều kiến thức hơn ở ICPC, đặc biệt là nhiều kỹ năng toán học hơn.

Các kỳ thi trực tuyến

Cũng có nhiều kỳ thi trực tuyến mở cho mọi người. Hiện tại, trang thi đấu sôi động nhất là Codeforces, nơi tổ chức các kỳ thi gần như hàng tuần. Ở Codeforces, thí sinh được chia thành hai hạng: người mới thi ở Div2 và lập trình viên có kinh nghiệm hơn thi ở Div1. Các trang thi đấu khác bao gồm AtCoder, CS Academy, HackerRank và Topcoder.

Một số công ty tổ chức các kỳ thi trực tuyến với vòng chung kết trực tiếp. Ví dụ về các kỳ thi như vậy là Facebook Hacker Cup, Google Code Jam và Yandex.Algorithm. Tất nhiên, các công ty cũng dùng những kỳ thi này để tuyển dụng: thành tích tốt trong một kỳ thi là cách tốt để chứng minh năng lực của bản thân.

Sách

Cũng đã có một số cuốn sách (ngoài cuốn này) tập trung vào lập trình thi đấu và giải quyết bài toán bằng thuật toán:

- S. S. Skiena and M. A. Revilla: *Programming Challenges: The Programming Contest Training Manual* [59]
- S. Halim and F. Halim: *Competitive Programming 3: The New Lower Bound of Programming Contests* [33]
- K. Diks et al.: *Looking for a Challenge? The Ultimate Problem Set from the University of Warsaw Programming Competitions* [15]

Hai cuốn đầu dành cho người mới bắt đầu, trong khi cuốn cuối chứa các tài liệu nâng cao.

Tất nhiên, các sách thuật toán tổng quát cũng phù hợp với người lập trình thi đấu. Một số cuốn sách phổ biến là:

- T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein: *Introduction to Algorithms* [13]
- J. Kleinberg and É. Tardos: *Algorithm Design* [45]
- S. S. Skiena: *The Algorithm Design Manual* [58]

Chương 2

Time complexity

Hiệu quả của các thuật toán rất quan trọng trong lập trình thi đấu. Thông thường, việc thiết kế một thuật toán giải bài toán một cách chậm chạp là khá dễ, nhưng thách thức thật sự là nghĩ ra một thuật toán nhanh. Nếu thuật toán quá chậm, nó sẽ chỉ nhận được một phần điểm hoặc không có điểm nào.

Độ phức tạp thời gian **time complexity** của một thuật toán ước lượng lượng thời gian mà thuật toán sẽ sử dụng cho một đầu vào nào đó. Ý tưởng là biểu diễn hiệu quả như một hàm có tham số là kích thước của đầu vào. Bằng cách tính độ phức tạp thời gian, ta có thể biết thuật toán có đủ nhanh hay không mà không cần cài đặt nó.

2.1 Calculation rules

Độ phức tạp thời gian của một thuật toán được ký hiệu là $O(\dots)$, trong đó ba dấu chấm biểu diễn một hàm nào đó. Thông thường, biến n biểu diễn kích thước đầu vào. Ví dụ, nếu đầu vào là một mảng các số, n sẽ là kích thước của mảng, và nếu đầu vào là một chuỗi, n sẽ là độ dài của chuỗi.

Loops

Một lý do phổ biến khiến một thuật toán chậm là nó chứa nhiều vòng lặp đi qua đầu vào. Thuật toán càng có nhiều vòng lặp lồng nhau, nó càng chậm. Nếu có k vòng lặp lồng nhau, độ phức tạp thời gian là $O(n^k)$.

Ví dụ, độ phức tạp thời gian của đoạn mã sau là $O(n)$:

```
for (int i = 1; i <= n; i++) {  
    // code  
}
```

Và độ phức tạp thời gian của đoạn mã sau là $O(n^2)$:

```
for (int i = 1; i <= n; i++) {  
    for (int j = 1; j <= n; j++) {  
        // code  
    }  
}
```

```
}
```

Order of magnitude

Một độ phức tạp thời gian không cho ta biết số lần chính xác mà mã bên trong một vòng lặp được thực thi, mà chỉ cho biết bậc độ lớn. Trong các ví dụ sau, mã bên trong vòng lặp được thực thi $3n$, $n + 5$ và $\lceil n/2 \rceil$ lần, nhưng độ phức tạp thời gian của mỗi đoạn mã đều là $O(n)$.

```
for (int i = 1; i <= 3*n; i++) {  
    // code  
}
```

```
for (int i = 1; i <= n+5; i++) {  
    // code  
}
```

```
for (int i = 1; i <= n; i += 2) {  
    // code  
}
```

Như một ví dụ khác, độ phức tạp thời gian của đoạn mã sau là $O(n^2)$:

```
for (int i = 1; i <= n; i++) {  
    for (int j = i+1; j <= n; j++) {  
        // code  
    }  
}
```

Phases

Nếu thuật toán gồm các pha liên tiếp, tổng độ phức tạp thời gian là độ phức tạp thời gian lớn nhất của một pha đơn lẻ. Lý do là pha chậm nhất thường là nút thắt cổ chai của mã.

Ví dụ, đoạn mã sau gồm ba pha với độ phức tạp thời gian lần lượt là $O(n)$, $O(n^2)$ và $O(n)$. Do đó, tổng độ phức tạp thời gian là $O(n^2)$.

```
for (int i = 1; i <= n; i++) {  
    // code  
}  
for (int i = 1; i <= n; i++) {  
    for (int j = 1; j <= n; j++) {  
        // code  
    }  
}
```



```
for (int i = 1; i <= n; i++) {  
    // code  
}
```

Several variables

Đôi khi độ phức tạp thời gian phụ thuộc vào nhiều yếu tố. Trong trường hợp này, công thức độ phức tạp thời gian chứa nhiều biến.

Ví dụ, độ phức tạp thời gian của đoạn mã sau là $O(nm)$:

```
for (int i = 1; i <= n; i++) {  
    for (int j = 1; j <= m; j++) {  
        // code  
    }  
}
```

Recursion

Độ phức tạp thời gian của một hàm đệ quy phụ thuộc vào số lần hàm được gọi và độ phức tạp thời gian của một lời gọi đơn lẻ. Tổng độ phức tạp thời gian là tích của hai giá trị này.

Ví dụ, xét hàm sau:

```
void f(int n) {  
    if (n == 1) return;  
    f(n-1);  
}
```

Lời gọi $f(n)$ gây ra n lời gọi hàm, và độ phức tạp thời gian của mỗi lời gọi là $O(1)$. Do đó, tổng độ phức tạp thời gian là $O(n)$.

Như một ví dụ khác, xét hàm sau:

```
void g(int n) {  
    if (n == 1) return;  
    g(n-1);  
    g(n-1);  
}
```

Trong trường hợp này, mỗi lời gọi hàm sinh ra hai lời gọi khác, ngoại trừ khi $n = 1$. Hãy xem điều gì xảy ra khi g được gọi với tham số n . Bảng sau cho thấy các lời gọi hàm được tạo ra bởi lời gọi đơn lẻ này:

lời gọi hàm	số lời gọi
$g(n)$	1
$g(n-1)$	2
$g(n-2)$	4
$\dots$	$\dots$
$g(1)$	2^{n-1}

Dựa trên điều này, độ phức tạp thời gian là

$$1 + 2 + 4 + \dots + 2^{n-1} = 2^n - 1 = O(2^n).$$

2.2 Complexity classes

Danh sách sau chứa các độ phức tạp thời gian phổ biến của thuật toán:

$O(1)$ Thời gian chạy của một thuật toán thời gian hằng số **constant-time** không phụ thuộc vào kích thước đầu vào. Một thuật toán thời gian hằng số điển hình là một công thức trực tiếp để tính đáp án.

$O(\log n)$ Một thuật toán logarit **logarithmic** thường chia đôi kích thước đầu vào ở mỗi bước. Thời gian chạy của một thuật toán như vậy có dạng logarit, vì $\log_2 n$ bằng số lần cần chia n cho 2 để nhận được 1.

$O(\sqrt{n})$ Một thuật toán căn bậc hai **square root algorithm** chậm hơn $O(\log n)$ nhưng nhanh hơn $O(n)$. Một tính chất đặc biệt của căn bậc hai là $\sqrt{n} = n/\sqrt{n}$, nên căn bậc hai $\sqrt{n}$ nằm, theo một nghĩa nào đó, ở giữa đầu vào.

$O(n)$ Một thuật toán tuyến tính **linear** đi qua đầu vào một số lần hằng số. Đây thường là độ phức tạp thời gian tốt nhất có thể, vì thông thường cần truy cập mỗi phần tử đầu vào ít nhất một lần trước khi đưa ra đáp án.

$O(n \log n)$ Độ phức tạp thời gian này thường cho thấy rằng thuật toán sắp xếp đầu vào, vì độ phức tạp thời gian của các thuật toán sắp xếp hiệu quả là $O(n \log n)$. Một khả năng khác là thuật toán sử dụng một cấu trúc dữ liệu mà mỗi thao tác mất thời gian $O(\log n)$.

$O(n^2)$ Một thuật toán bậc hai **quadratic** thường chứa hai vòng lặp lồng nhau. Có thể duyệt qua mọi cặp phần tử đầu vào trong thời gian $O(n^2)$.

$O(n^3)$ Một thuật toán bậc ba **cubic** thường chứa ba vòng lặp lồng nhau. Có thể duyệt qua mọi bộ ba phần tử đầu vào trong thời gian $O(n^3)$.

$O(2^n)$ Độ phức tạp thời gian này thường cho thấy rằng thuật toán lặp qua mọi tập con của các phần tử đầu vào. Ví dụ, các tập con của $\{1, 2, 3\}$ là $\emptyset$, $\{1\}$, $\{2\}$, $\{3\}$, $\{1, 2\}$, $\{1, 3\}$, $\{2, 3\}$ và $\{1, 2, 3\}$.

$O(n!)$ Độ phức tạp thời gian này thường cho thấy rằng thuật toán lặp qua mọi hoán vị của các phần tử đầu vào. Ví dụ, các hoán vị của $\{1, 2, 3\}$ là $(1, 2, 3)$, $(1, 3, 2)$, $(2, 1, 3)$, $(2, 3, 1)$, $(3, 1, 2)$ và $(3, 2, 1)$.

Một thuật toán là đa thức **polynomial** nếu độ phức tạp thời gian của nó nhiều nhất là $O(n^k)$, trong đó k là một hằng số. Tất cả các độ phức tạp thời gian ở trên, ngoại trừ $O(2^n)$ và $O(n!)$, đều là đa thức. Trong thực tế, hằng số k thường nhỏ, và do đó độ phức tạp thời gian đa thức gần như có nghĩa là thuật toán là hiệu quả *efficient*.

Hầu hết các thuật toán trong sách này là đa thức. Dù vậy, có nhiều bài toán quan trọng mà hiện nay chưa biết thuật toán đa thức nào cho chúng, tức là không ai biết cách giải chúng một cách hiệu quả. Các bài toán NP-hard **NP-hard** là một tập hợp quan trọng các bài toán mà chưa biết thuật toán đa thức nào cho chúng¹.

2.3 Estimating efficiency

Bằng cách tính độ phức tạp thời gian của một thuật toán, có thể kiểm tra, trước khi cài đặt thuật toán, rằng nó có đủ hiệu quả cho bài toán hay không. Điểm xuất phát cho các ước lượng là thực tế rằng một máy tính hiện đại có thể thực hiện vài trăm triệu phép toán trong một giây.

Ví dụ, giả sử giới hạn thời gian cho một bài toán là một giây và kích thước đầu vào là $n = 10^5$. Nếu độ phức tạp thời gian là $O(n^2)$, thuật toán sẽ thực hiện khoảng $(10^5)^2 = 10^{10}$ phép toán. Việc này sẽ mất ít nhất vài chục giây, nên thuật toán có vẻ quá chậm để giải bài toán.

Mặt khác, với kích thước đầu vào đã cho, ta có thể thử phỏng đoán *guess* độ phức tạp thời gian cần thiết của thuật toán giải bài toán. Bảng sau chứa một số ước lượng hữu ích với giả định giới hạn thời gian là một giây.

kích thước đầu vào	độ phức tạp thời gian cần thiết
$n \leq 10$	$O(n!)$
$n \leq 20$	$O(2^n)$
$n \leq 500$	$O(n^3)$
$n \leq 5000$	$O(n^2)$
$n \leq 10^6$	$O(n \log n)$ hoặc $O(n)$
n lớn	$O(1)$ hoặc $O(\log n)$

Ví dụ, nếu kích thước đầu vào là $n = 10^5$, có khả năng đề bài mong đợi rằng độ phức tạp thời gian của thuật toán là $O(n)$ hoặc $O(n \log n)$. Thông tin này giúp việc thiết kế thuật toán dễ hơn, vì nó loại trừ những cách tiếp cận sẽ tạo ra một thuật toán có độ phức tạp thời gian tệ hơn.

Dù vậy, điều quan trọng cần nhớ là một độ phức tạp thời gian chỉ là một ước lượng về hiệu quả, vì nó che giấu các hệ số hằng *constant factors*. Ví dụ, một thuật toán chạy trong thời gian $O(n)$ có thể thực hiện $n/2$ hoặc $5n$ phép toán. Điều này có ảnh hưởng quan trọng đến thời gian chạy thực tế của thuật toán.

¹A classic book on the topic is M. R. Garey's and D. S. Johnson's *Computers and Intractability: A Guide to the Theory of NP-Completeness* [28].

2.4 Maximum subarray sum

Thường có nhiều thuật toán khả dĩ để giải một bài toán, và chúng có độ phức tạp thời gian khác nhau. Mục này thảo luận một bài toán kinh điển có lời giải trực tiếp $O(n^3)$. Tuy nhiên, bằng cách thiết kế một thuật toán tốt hơn, có thể giải bài toán trong thời gian $O(n^2)$ và thậm chí trong thời gian $O(n)$.

Cho một mảng gồm n số, nhiệm vụ của ta là tính tổng mảng con lớn nhất **maximum subarray sum**, tức là tổng lớn nhất có thể của một dãy các giá trị liên tiếp trong mảng². Bài toán trở nên thú vị khi trong mảng có thể có các giá trị âm. Ví dụ, trong mảng

-1	2	4	-3	5	2	-5	2
----	---	---	----	---	---	----	---

mảng con sau tạo ra tổng lớn nhất 10:

-1	2	4	-3	5	2	-5	2
----	---	---	----	---	---	----	---

Ta giả sử rằng mảng con rỗng được phép, nên tổng mảng con lớn nhất luôn ít nhất là 0.

Algorithm 1

Một cách trực tiếp để giải bài toán là duyệt qua mọi mảng con có thể, tính tổng các giá trị trong từng mảng con và duy trì tổng lớn nhất. Đoạn mã sau cài đặt thuật toán này:

```
int best = 0;
for (int a = 0; a < n; a++) {
    for (int b = a; b < n; b++) {
        int sum = 0;
        for (int k = a; k <= b; k++) {
            sum += array[k];
        }
        best = max(best, sum);
    }
}
cout << best << "\n";
```

Các biến a và b cố định chỉ số đầu tiên và chỉ số cuối cùng của mảng con, và tổng các giá trị được tính vào biến sum . Biến $best$ chứa tổng lớn nhất tìm được trong quá trình tìm kiếm.

Độ phức tạp thời gian của thuật toán là $O(n^3)$, vì nó gồm ba vòng lặp lồng nhau đi qua đầu vào.

Algorithm 2

Dễ dàng làm cho Thuật toán 1 hiệu quả hơn bằng cách loại bỏ một vòng lặp khỏi nó. Điều này có thể thực hiện bằng cách tính tổng đồng thời khi đầu phải của mảng con di chuyển. Kết quả là đoạn mã sau:

²J. Bentley's book *Programming Pearls* [8] made the problem popular.

```

int best = 0;
for (int a = 0; a < n; a++) {
    int sum = 0;
    for (int b = a; b < n; b++) {
        sum += array[b];
        best = max(best, sum);
    }
}
cout << best << "\n";

```

Sau thay đổi này, độ phức tạp thời gian là $O(n^2)$.

Algorithm 3

Điều đáng ngạc nhiên là có thể giải bài toán trong thời gian $O(n)^3$, nghĩa là chỉ một vòng lặp là đủ. Ý tưởng là tính, với mỗi vị trí trong mảng, tổng lớn nhất của một mảng con kết thúc tại vị trí đó. Sau đó, đáp án của bài toán là giá trị lớn nhất trong các tổng đó.

Xét bài toán con tìm mảng con có tổng lớn nhất kết thúc tại vị trí k . Có hai khả năng:

1. Mảng con chỉ chứa phần tử ở vị trí k .
2. Mảng con gồm một mảng con kết thúc tại vị trí $k - 1$, theo sau là phần tử ở vị trí k .

Trong trường hợp sau, vì ta muốn tìm một mảng con có tổng lớn nhất, mảng con kết thúc tại vị trí $k - 1$ cũng nên có tổng lớn nhất. Do đó, ta có thể giải bài toán một cách hiệu quả bằng cách tính tổng mảng con lớn nhất cho từng vị trí kết thúc từ trái sang phải.

Đoạn mã sau cài đặt thuật toán:

```

int best = 0, sum = 0;
for (int k = 0; k < n; k++) {
    sum = max(array[k], sum + array[k]);
    best = max(best, sum);
}
cout << best << "\n";

```

Thuật toán chỉ chứa một vòng lặp đi qua đầu vào, nên độ phức tạp thời gian là $O(n)$. Đây cũng là độ phức tạp thời gian tốt nhất có thể, vì bất kỳ thuật toán nào cho bài toán cũng phải xét tất cả phần tử của mảng ít nhất một lần.

³In [8], this linear-time algorithm is attributed to J. B. Kadane, and the algorithm is sometimes called **Kadane's algorithm**.

Efficiency comparison

Việc nghiên cứu các thuật toán hiệu quả đến mức nào trong thực tế là điều thú vị. Bảng sau cho thấy thời gian chạy của các thuật toán ở trên với các giá trị n khác nhau trên một máy tính hiện đại.

Trong mỗi phép thử, đầu vào được sinh ngẫu nhiên. Thời gian cần để đọc đầu vào không được đo.

kích thước mảng n	Thuật toán 1	Thuật toán 2	Thuật toán 3
10^2	0.0 s	0.0 s	0.0 s
10^3	0.1 s	0.0 s	0.0 s
10^4	> 10.0 s	0.1 s	0.0 s
10^5	> 10.0 s	5.3 s	0.0 s
10^6	> 10.0 s	> 10.0 s	0.0 s
10^7	> 10.0 s	> 10.0 s	0.0 s

So sánh cho thấy rằng tất cả các thuật toán đều hiệu quả khi kích thước đầu vào nhỏ, nhưng các đầu vào lớn hơn làm bộc lộ những khác biệt đáng kể trong thời gian chạy của các thuật toán. Thuật toán 1 trở nên chậm khi $n = 10^4$, và Thuật toán 2 trở nên chậm khi $n = 10^5$. Chỉ Thuật toán 3 có thể xử lý ngay cả các đầu vào lớn nhất một cách tức thì.

Chương 3

Sắp xếp

Sắp xếp (sorting) là một bài toán cơ bản trong thiết kế thuật toán. Nhiều thuật toán hiệu quả sử dụng sắp xếp như một thủ tục con, vì xử lý dữ liệu thường dễ hơn nếu các phần tử đã ở thứ tự đã sắp.

Ví dụ, bài toán "một mảng có chứa hai phần tử bằng nhau không?" dễ giải bằng cách sắp xếp. Nếu mảng chứa hai phần tử bằng nhau, sau khi sắp xếp chúng sẽ nằm cạnh nhau, nên rất dễ tìm. Tương tự, bài toán "phần tử nào xuất hiện nhiều nhất trong một mảng?" cũng có thể được giải theo cách đó.

Có rất nhiều thuật toán cho việc sắp xếp, và chúng cũng là những ví dụ tốt cho việc áp dụng các kỹ thuật thiết kế thuật toán khác nhau. Các thuật toán sắp xếp tổng quát hiệu quả chạy trong thời gian $O(n \log n)$, và nhiều thuật toán dùng sắp xếp như một thủ tục con cũng có độ phức tạp thời gian này.

3.1 Lý thuyết sắp xếp

Bài toán cơ bản trong sắp xếp như sau:

Cho một mảng (array) gồm n phần tử, nhiệm vụ của bạn là sắp xếp các phần tử theo thứ tự tăng dần.

Ví dụ, mảng

1	3	8	2	9	2	5	6
---	---	---	---	---	---	---	---

sau khi sắp xếp sẽ trở thành:

1	2	2	3	5	6	8	9
---	---	---	---	---	---	---	---

Các thuật toán $O(n^2)$

Những thuật toán đơn giản để sắp xếp một mảng chạy trong thời gian $O(n^2)$. Các thuật toán như vậy ngắn và thường gồm hai vòng lặp lồng nhau. Một thuật toán sắp

xếp $O(n^2)$ nổi tiếng là **sắp xếp nổi bọt** (bubble sort), trong đó các phần tử "nổi bọt" trong mảng theo giá trị của chúng.

Sắp xếp nổi bọt gồm n vòng. Trong mỗi vòng, thuật toán duyệt qua các phần tử của mảng. Mỗi khi thấy hai phần tử liên tiếp không đúng thứ tự, thuật toán đổi chỗ chúng. Thuật toán có thể được cài đặt như sau:

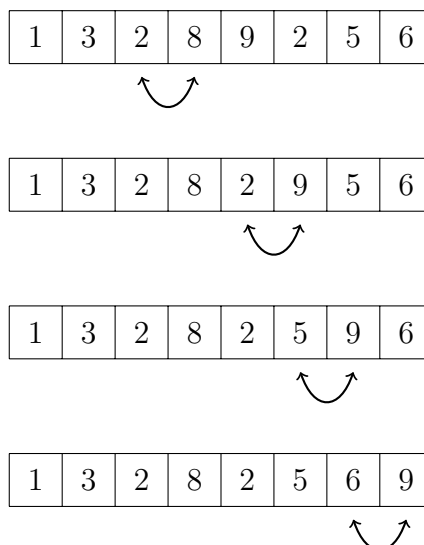
```
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < n-1; j++) {  
        if (array[j] > array[j+1]) {  
            swap(array[j], array[j+1]);  
        }  
    }  
}
```

Sau vòng đầu tiên của thuật toán, phần tử lớn nhất sẽ ở đúng vị trí, và tổng quát, sau k vòng, k phần tử lớn nhất sẽ ở đúng vị trí. Do đó, sau n vòng, toàn bộ mảng sẽ được sắp xếp.

Ví dụ, trong mảng

1	3	8	2	9	2	5	6
---	---	---	---	---	---	---	---

vòng đầu tiên của sắp xếp nổi bọt đổi chỗ các phần tử như sau:



Nghịch thế

Sắp xếp nổi bọt là một ví dụ về thuật toán sắp xếp luôn đổi chỗ các phần tử *liên tiếp* trong mảng. Hóa ra, độ phức tạp thời gian của một thuật toán như vậy *luôn* là ít nhất $O(n^2)$, vì trong trường hợp xấu nhất, cần $O(n^2)$ phép đổi chỗ để sắp xếp mảng.

Một khái niệm hữu ích khi phân tích các thuật toán sắp xếp là **nghịch thế** (inversion): một cặp phần tử ($array[a]$, $array[b]$) sao cho $a < b$ và $array[a] > array[b]$, tức là các phần tử ở sai thứ tự. Ví dụ, mảng

1	2	2	6	3	5	9	8
---	---	---	---	---	---	---	---

có ba nghịch thế: $(6, 3)$, $(6, 5)$ và $(9, 8)$. Số nghịch thế cho biết cần làm bao nhiêu công việc để sắp xếp mảng. Một mảng được sắp xếp hoàn toàn khi không còn nghịch thế nào. Ngược lại, nếu các phần tử của mảng ở thứ tự ngược lại, số nghịch thế là lớn nhất có thể:

$$1 + 2 + \dots + (n - 1) = \frac{n(n - 1)}{2} = O(n^2)$$

Đổi chỗ một cặp phần tử liên tiếp đang ở sai thứ tự sẽ loại bỏ đúng một nghịch thế khỏi mảng. Do đó, nếu một thuật toán sắp xếp chỉ có thể đổi chỗ các phần tử liên tiếp, mỗi phép đổi chỗ loại bỏ nhiều nhất một nghịch thế, và độ phức tạp thời gian của thuật toán ít nhất là $O(n^2)$.

Các thuật toán $O(n \log n)$

Có thể sắp xếp một mảng một cách hiệu quả trong thời gian $O(n \log n)$ bằng các thuật toán không bị giới hạn ở việc đổi chỗ các phần tử liên tiếp. Một thuật toán như vậy là **sắp xếp trộn** (merge sort)¹, dựa trên đệ quy.

Sắp xếp trộn sắp xếp một mảng con $\text{array}[a \dots b]$ như sau:

1. Nếu $a = b$, không làm gì cả, vì mảng con đã được sắp xếp.
2. Tính vị trí của phần tử ở giữa: $k = \lfloor (a + b)/2 \rfloor$.
3. Sắp xếp đệ quy mảng con $\text{array}[a \dots k]$.
4. Sắp xếp đệ quy mảng con $\text{array}[k + 1 \dots b]$.
5. Trộn các mảng con đã sắp xếp $\text{array}[a \dots k]$ và $\text{array}[k + 1 \dots b]$ thành một mảng con đã sắp xếp $\text{array}[a \dots b]$.

Sắp xếp trộn là một thuật toán hiệu quả, vì nó giảm kích thước mảng con đi một nửa ở mỗi bước. Đệ quy gồm $O(\log n)$ tầng, và xử lý mỗi tầng mất $O(n)$ thời gian. Việc trộn các mảng con $\text{array}[a \dots k]$ và $\text{array}[k + 1 \dots b]$ có thể được thực hiện trong thời gian tuyến tính, vì chúng đã được sắp xếp.

Ví dụ, xét việc sắp xếp mảng sau:

1	3	6	2	8	2	5	9
---	---	---	---	---	---	---	---

Mảng sẽ được chia thành hai mảng con như sau:

1	3	6	2
---	---	---	---

8	2	5	9
---	---	---	---

Sau đó, các mảng con được sắp xếp đệ quy như sau:

1	2	3	6
---	---	---	---

2	5	8	9
---	---	---	---

¹Theo [47], sắp xếp trộn được J. von Neumann phát minh năm 1945.

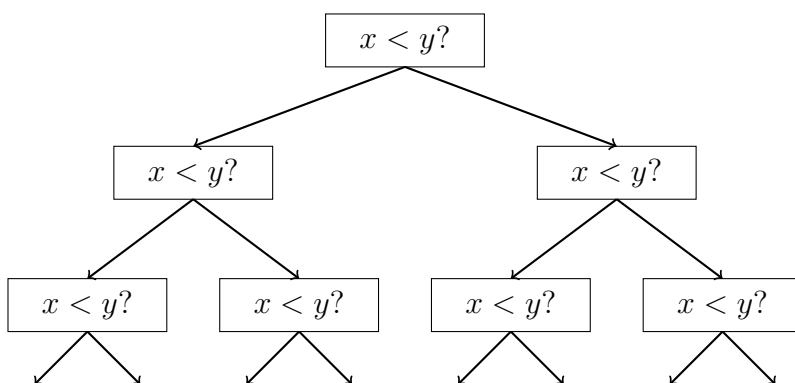
Cuối cùng, thuật toán trộn các mảng con đã sắp xếp và tạo ra mảng đã sắp xếp cuối cùng:

1	2	2	3	5	6	8	9
---	---	---	---	---	---	---	---

Cận dưới của sắp xếp

Có thể sắp xếp một mảng nhanh hơn $O(n \log n)$ không? Hóa ra, điều này *không* thể nếu ta giới hạn ở các thuật toán sắp xếp dựa trên việc so sánh các phần tử của mảng.

Cận dưới cho độ phức tạp thời gian có thể được chứng minh bằng cách xem sắp xếp như một quá trình mà mỗi lần so sánh hai phần tử cho ta thêm thông tin về nội dung của mảng. Quá trình này tạo ra cây sau:



Ở đây, " $x < y$?" nghĩa là so sánh hai phần tử x và y . Nếu $x < y$, quá trình tiếp tục sang trái, ngược lại sang phải. Các kết quả của quá trình là các cách có thể để sắp xếp mảng, tổng cộng $n!$ cách. Vì lý do này, chiều cao của cây phải ít nhất là

$$\log_2(n!) = \log_2(1) + \log_2(2) + \dots + \log_2(n).$$

Ta nhận được một cận dưới cho tổng này bằng cách chọn $n/2$ số hạng cuối và đổi giá trị mỗi số hạng thành $\log_2(n/2)$. Việc này cho ta ước lượng

$$\log_2(n!) \geq (n/2) \cdot \log_2(n/2),$$

nên chiều cao của cây và số bước tối thiểu có thể của một thuật toán sắp xếp trong trường hợp xấu nhất ít nhất là $n \log n$.

Sắp xếp đếm

Cận dưới $n \log n$ không áp dụng cho các thuật toán không so sánh các phần tử của mảng mà sử dụng thông tin khác. Một ví dụ của thuật toán như vậy là **sắp xếp đếm** (counting sort), thuật toán này sắp xếp một mảng trong thời gian $O(n)$ với giả định mọi phần tử của mảng là một số nguyên trong khoảng $0 \dots c$ và $c = O(n)$.

Thuật toán tạo ra một mảng *kế toán* (bookkeeping), mà chỉ số là các phần tử của mảng gốc. Thuật toán duyệt qua mảng gốc và tính xem mỗi phần tử xuất hiện bao nhiêu lần trong mảng.

Ví dụ, mảng

1	3	6	9	9	3	5	9
---	---	---	---	---	---	---	---

tương ứng với mảng kế toán sau:

1	2	3	4	5	6	7	8	9
1	0	2	0	1	1	0	0	3

Ví dụ, giá trị ở vị trí 3 trong mảng kế toán là 2, vì phần tử 3 xuất hiện 2 lần trong mảng gốc.

Việc xây dựng mảng kế toán mất $O(n)$ thời gian. Sau đó, mảng đã sắp xếp có thể được tạo ra trong $O(n)$ thời gian vì số lần xuất hiện của mỗi phần tử có thể lấy từ mảng kế toán. Như vậy, tổng độ phức tạp thời gian của sắp xếp đếm là $O(n)$.

Sắp xếp đếm là một thuật toán rất hiệu quả nhưng chỉ có thể dùng khi hằng số c đủ nhỏ, để các phần tử của mảng có thể được dùng làm chỉ số trong mảng kế toán.

3.2 Sắp xếp trong C++

Hầu như không bao giờ là ý hay nếu dùng một thuật toán sắp xếp tự viết trong một kỳ thi, vì đã có các cài đặt tốt sẵn trong các ngôn ngữ lập trình. Ví dụ, thư viện chuẩn C++ chứa hàm `sort` có thể được dùng để sắp xếp mảng và các cấu trúc dữ liệu khác.

Có nhiều lợi ích khi dùng hàm thư viện. Thứ nhất, tiết kiệm thời gian vì không cần cài đặt hàm. Thứ hai, cài đặt thư viện chắc chắn đúng và hiệu quả: không có khả năng cao là một hàm sắp xếp tự viết sẽ tốt hơn.

Trong mục này, ta sẽ xem cách dùng hàm `sort` của C++. Đoạn code sau sắp xếp một vector theo thứ tự tăng dần:

```
vector<int> v = {4,2,5,3,5,8,3};  
sort(v.begin(), v.end());
```

Sau khi sắp xếp, nội dung của vector sẽ là `[2, 3, 3, 4, 5, 5, 8]`. Thứ tự sắp xếp mặc định là tăng dần, nhưng có thể sắp xếp theo thứ tự ngược lại như sau:

```
sort(v.rbegin(), v.rend());
```

Một mảng thông thường có thể được sắp xếp như sau:

```
int n = 7; // array size  
int a[] = {4,2,5,3,5,8,3};  
sort(a, a+n);
```

Đoạn code sau sắp xếp chuỗi s:

```
string s = "monkey";
sort(s.begin(), s.end());
```

Sắp xếp một chuỗi có nghĩa là các ký tự của chuỗi được sắp xếp. Ví dụ, chuỗi "monkey" trở thành "ekmnoy".

Toán tử so sánh

Hàm sort yêu cầu một **toán tử so sánh** (comparison operator) được định nghĩa cho kiểu dữ liệu của các phần tử cần sắp xếp. Khi sắp xếp, toán tử này sẽ được dùng mỗi khi cần xác định thứ tự của hai phần tử.

Hầu hết các kiểu dữ liệu C++ có sẵn toán tử so sánh, và các phần tử thuộc kiểu đó có thể được sắp xếp tự động. Ví dụ, các số được sắp xếp theo giá trị và các chuỗi được sắp xếp theo thứ tự bảng chữ cái.

Các cặp (pair) được sắp xếp chủ yếu theo phần tử đầu tiên (first). Tuy nhiên, nếu phần tử đầu tiên của hai cặp bằng nhau, chúng được sắp xếp theo phần tử thứ hai (second):

```
vector<pair<int,int>> v;
v.push_back({1,5});
v.push_back({2,3});
v.push_back({1,2});
sort(v.begin(), v.end());
```

Sau đó, thứ tự các cặp là (1, 2), (1, 5) và (2, 3).

Tương tự, các bộ (tuple) được sắp xếp chủ yếu theo phần tử đầu tiên, thứ yếu theo phần tử thứ hai, v.v.²:

```
vector<tuple<int,int,int>> v;
v.push_back({2,1,4});
v.push_back({1,5,3});
v.push_back({2,1,3});
sort(v.begin(), v.end());
```

Sau đó, thứ tự các tuple là (1, 5, 3), (2, 1, 3) và (2, 1, 4).

Struct do người dùng định nghĩa

Các struct do người dùng định nghĩa không có sẵn toán tử so sánh. Toán tử nên được định nghĩa bên trong struct dưới dạng một hàm operator<, mà tham số là một phần tử khác cùng kiểu. Toán tử nên trả về true nếu phần tử nhỏ hơn tham số, và false nếu ngược lại.

²Lưu ý rằng ở một số trình biên dịch cũ, hàm make_tuple phải được dùng để tạo một tuple thay cho dấu ngoặc nhọn (ví dụ, make_tuple(2,1,4) thay cho {2,1,4}).

Ví dụ, struct P sau chứa tọa độ x và y của một điểm. Toán tử so sánh được định nghĩa sao cho các điểm được sắp xếp chủ yếu theo tọa độ x và thứ yếu theo tọa độ y.

```
struct P {
    int x, y;
    bool operator<(const P &p) {
        if (x != p.x) return x < p.x;
        else return y < p.y;
    }
};
```

Hàm so sánh

Cũng có thể đưa một **hàm so sánh** (comparison function) bên ngoài vào hàm sort dưới dạng hàm gọi lại (callback). Ví dụ, hàm so sánh comp sau sắp xếp các chuỗi chủ yếu theo độ dài và thứ yếu theo thứ tự bảng chữ cái:

```
bool comp(string a, string b) {
    if (a.size() != b.size()) return a.size() < b.size();
    return a < b;
}
```

Bây giờ một vector các chuỗi có thể được sắp xếp như sau:

```
sort(v.begin(), v.end(), comp);
```

3.3 Tìm kiếm nhị phân

Một cách tổng quát để tìm kiếm một phần tử trong một mảng là dùng một vòng lặp for duyệt qua các phần tử của mảng. Ví dụ, đoạn code sau tìm kiếm một phần tử x trong một mảng:

```
for (int i = 0; i < n; i++) {
    if (array[i] == x) {
        // x found at index i
    }
}
```

Độ phức tạp thời gian của cách này là $O(n)$, vì trong trường hợp xấu nhất, cần phải kiểm tra mọi phần tử của mảng. Nếu thứ tự các phần tử là tùy ý, đây cũng là cách tốt nhất có thể, vì không có thêm thông tin nào về vị trí trong mảng mà ta nên tìm phần tử x .

Tuy nhiên, nếu mảng đã *được sắp xếp*, tình huống khác hẳn. Trong trường hợp này, có thể thực hiện tìm kiếm nhanh hơn nhiều, vì thứ tự các phần tử trong mảng dẫn dắt việc tìm kiếm. Thuật toán **tìm kiếm nhị phân** (binary search) sau tìm kiếm một phần tử trong mảng đã sắp xếp một cách hiệu quả trong thời gian $O(\log n)$.

Cách 1

Cách thông thường để cài đặt tìm kiếm nhị phân gọi nhớ đến việc tra một từ trong từ điển. Quá trình tìm kiếm duy trì một vùng đang hoạt động trong mảng, ban đầu chứa toàn bộ các phần tử của mảng. Sau đó, ta thực hiện một số bước, mỗi bước cắt đôi kích thước của vùng đó.

Ở mỗi bước, tìm kiếm kiểm tra phần tử ở giữa của vùng đang hoạt động. Nếu phần tử ở giữa là phần tử cần tìm, tìm kiếm kết thúc. Ngược lại, tìm kiếm tiếp tục đệ quy vào nửa trái hoặc nửa phải của vùng, tùy theo giá trị của phần tử ở giữa.

Ý tưởng trên có thể được cài đặt như sau:

```
int a = 0, b = n-1;
while (a <= b) {
    int k = (a+b)/2;
    if (array[k] == x) {
        // x found at index k
    }
    if (array[k] > x) b = k-1;
    else a = k+1;
}
```

Trong cài đặt này, vùng đang hoạt động là $a \dots b$, và ban đầu vùng đó là $0 \dots n-1$. Thuật toán cắt đôi kích thước của vùng ở mỗi bước, nên độ phức tạp thời gian là $O(\log n)$.

Cách 2

Một cách khác để cài đặt tìm kiếm nhị phân dựa trên một cách duyệt hiệu quả qua các phần tử của mảng. Ý tưởng là thực hiện các bước nhảy và giảm tốc độ khi ta đến gần phần tử cần tìm.

Quá trình tìm kiếm đi qua mảng từ trái sang phải, và độ dài bước nhảy ban đầu là $n/2$. Ở mỗi bước, độ dài bước nhảy sẽ được cắt đôi: đầu tiên $n/4$, rồi $n/8$, $n/16$, v.v., cho đến khi cuối cùng độ dài bằng 1. Sau các bước nhảy, hoặc đã tìm thấy phần tử cần tìm, hoặc ta biết rằng nó không xuất hiện trong mảng.

Đoạn code sau cài đặt ý tưởng trên:

```
int k = 0;
for (int b = n/2; b >= 1; b /= 2) {
    while (k+b < n && array[k+b] <= x) k += b;
}
if (array[k] == x) {
    // x found at index k
}
```

Trong quá trình tìm kiếm, biến b chứa độ dài bước nhảy hiện tại. Độ phức tạp thời gian của thuật toán là $O(\log n)$, vì đoạn code trong vòng lặp while được thực hiện nhiều nhất hai lần cho mỗi độ dài bước nhảy.

Các hàm C++

Thư viện chuẩn C++ chứa các hàm sau dựa trên tìm kiếm nhị phân và chạy trong thời gian logarit:

- `lower_bound` trả về một con trỏ tới phần tử đầu tiên trong mảng có giá trị ít nhất là x .
- `upper_bound` trả về một con trỏ tới phần tử đầu tiên trong mảng có giá trị lớn hơn x .
- `equal_range` trả về cả hai con trỏ trên.

Các hàm giả định rằng mảng đã được sắp xếp. Nếu không có phần tử thỏa mãn, con trỏ chỉ đến phần tử cuối cùng của mảng. Ví dụ, đoạn code sau kiểm tra xem mảng có chứa phần tử có giá trị x hay không:

```
auto k = lower_bound(array, array+n, x) - array;
if (k < n && array[k] == x) {
    // x found at index k
}
```

Đoạn code sau đếm số phần tử có giá trị bằng x :

```
auto a = lower_bound(array, array+n, x);
auto b = upper_bound(array, array+n, x);
cout << b-a << "\n";
```

Dùng `equal_range`, đoạn code trở nên ngắn hơn:

```
auto r = equal_range(array, array+n, x);
cout << r.second - r.first << "\n";
```

Tìm nghiệm nhỏ nhất

Một ứng dụng quan trọng của tìm kiếm nhị phân là tìm vị trí mà tại đó giá trị của một hàm thay đổi. Giả sử ta muốn tìm giá trị nhỏ nhất k là một nghiệm hợp lệ cho một bài toán. Cho một hàm $ok(x)$ trả về true nếu x là một nghiệm hợp lệ và false nếu ngược lại. Ngoài ra, ta biết rằng $ok(x)$ là false khi $x < k$ và true khi $x \geq k$. Tình huống được mô tả như sau:

x	0	1	$\dots$	$k-1$	k	$k+1$	$\dots$
$ok(x)$	false	false	$\dots$	false	true	true	$\dots$

Bây giờ, giá trị k có thể được tìm bằng tìm kiếm nhị phân:

```
int x = -1;
for (int b = z; b >= 1; b /= 2) {
    while (!ok(x+b)) x += b;
```

```
}  
int k = x+1;
```

Quá trình tìm kiếm tìm giá trị lớn nhất của x mà $ok(x)$ là false. Do đó, giá trị tiếp theo $k = x + 1$ là giá trị nhỏ nhất có thể mà $ok(k)$ là true. Độ dài bước nhảy ban đầu z phải đủ lớn, ví dụ một giá trị mà ta biết trước rằng $ok(z)$ là true.

Thuật toán gọi hàm ok $O(\log z)$ lần, nên tổng độ phức tạp thời gian phụ thuộc vào hàm ok . Ví dụ, nếu hàm chạy trong thời gian $O(n)$, tổng độ phức tạp thời gian là $O(n \log z)$.

Tìm giá trị cực đại

Tìm kiếm nhị phân cũng có thể được dùng để tìm giá trị cực đại của một hàm mà trước tiên tăng rồi sau đó giảm. Nhiệm vụ của ta là tìm một vị trí k sao cho

- $f(x) < f(x + 1)$ khi $x < k$, và
- $f(x) > f(x + 1)$ khi $x \geq k$.

Ý tưởng là dùng tìm kiếm nhị phân để tìm giá trị lớn nhất của x mà $f(x) < f(x+1)$. Điều này suy ra $k = x + 1$ vì $f(x + 1) > f(x + 2)$. Đoạn code sau cài đặt việc tìm kiếm:

```
int x = -1;  
for (int b = z; b >= 1; b /= 2) {  
    while (f(x+b) < f(x+b+1)) x += b;  
}  
int k = x+1;
```

Lưu ý rằng khác với tìm kiếm nhị phân thông thường, ở đây không cho phép các giá trị liên tiếp của hàm bằng nhau. Trong trường hợp đó, sẽ không thể biết làm thế nào để tiếp tục tìm kiếm.

Chương 4

Cấu trúc dữ liệu

Một **cấu trúc dữ liệu (data structure)** là một cách lưu trữ dữ liệu trong bộ nhớ của máy tính. Việc chọn một cấu trúc dữ liệu phù hợp cho một bài toán là rất quan trọng, vì mỗi cấu trúc dữ liệu có những ưu điểm và nhược điểm riêng. Câu hỏi then chốt là: những thao tác nào hiệu quả trong cấu trúc dữ liệu đã chọn?

Chương này giới thiệu các cấu trúc dữ liệu quan trọng nhất trong thư viện chuẩn C++. Ta nên dùng thư viện chuẩn bất cứ khi nào có thể, vì nó sẽ tiết kiệm rất nhiều thời gian. Ở các phần sau của sách, ta sẽ học về những cấu trúc dữ liệu tinh vi hơn không có sẵn trong thư viện chuẩn.

4.1 Mảng động

Một **mảng động (dynamic array)** là một mảng mà kích thước có thể thay đổi trong quá trình thực thi chương trình. Mảng động phổ biến nhất trong C++ là cấu trúc vector, có thể được dùng gần như một mảng thông thường.

Đoạn mã sau tạo một vector rỗng và thêm ba phần tử vào đó:

```
vector<int> v;  
v.push_back(3); // [3]  
v.push_back(2); // [3,2]  
v.push_back(5); // [3,2,5]
```

Sau đó, có thể truy cập các phần tử như trong một mảng thông thường:

```
cout << v[0] << "\n"; // 3  
cout << v[1] << "\n"; // 2  
cout << v[2] << "\n"; // 5
```

Hàm size trả về số phần tử trong vector. Đoạn mã sau duyệt qua vector và in ra tất cả các phần tử trong đó:

```
for (int i = 0; i < v.size(); i++) {  
    cout << v[i] << "\n";  
}
```

Một cách ngắn hơn để duyệt qua một vector như sau:

```
for (auto x : v) {  
    cout << x << "\n";  
}
```

Hàm `back` trả về phần tử cuối cùng trong vector, và hàm `pop_back` xóa phần tử cuối cùng:

```
vector<int> v;  
v.push_back(5);  
v.push_back(2);  
cout << v.back() << "\n"; // 2  
v.pop_back();  
cout << v.back() << "\n"; // 5
```

Đoạn mã sau tạo một vector có năm phần tử:

```
vector<int> v = {2,4,2,5,1};
```

Một cách khác để tạo vector là cung cấp số phần tử và giá trị ban đầu cho mỗi phần tử:

```
// size 10, initial value 0  
vector<int> v(10);
```

```
// size 10, initial value 5  
vector<int> v(10, 5);
```

Cài đặt bên trong của vector sử dụng một mảng thông thường. Nếu kích thước của vector tăng lên và mảng trở nên quá nhỏ, một mảng mới sẽ được cấp phát và tất cả các phần tử được chuyển sang mảng mới. Tuy nhiên, việc này không xảy ra thường xuyên và độ phức tạp thời gian trung bình của `push_back` là $O(1)$.

Cấu trúc string cũng là một mảng động có thể được dùng gần như một vector. Ngoài ra, chuỗi có cú pháp đặc biệt không có trong các cấu trúc dữ liệu khác. Các chuỗi có thể được ghép bằng ký hiệu `+`. Hàm `substr( $k, x$ )` trả về chuỗi con bắt đầu tại vị trí k và có độ dài x , và hàm `find( $t$ )` tìm vị trí xuất hiện đầu tiên của chuỗi con t .

Đoạn mã sau trình bày một số thao tác với chuỗi:

```
string a = "hatti";  
string b = a+a;  
cout << b << "\n"; // hattihatti  
b[5] = 'v';  
cout << b << "\n"; // hattivatti  
string c = b.substr(3,4);  
cout << c << "\n"; // tiva
```

4.2 Cấu trúc tập hợp

Một **set** là một cấu trúc dữ liệu duy trì một tập hợp các phần tử. Các thao tác cơ bản trên tập hợp là chèn, tìm kiếm và xóa phần tử.

Thư viện chuẩn C++ chứa hai cách cài đặt tập hợp: Cấu trúc set dựa trên một cây nhị phân cân bằng và các thao tác của nó chạy trong thời gian $O(\log n)$. Cấu trúc `unordered_set` sử dụng băm, và các thao tác của nó chạy trong thời gian $O(1)$ trung bình.

Việc chọn cách cài đặt tập hợp nào để dùng thường là vấn đề sở thích. Lợi ích của cấu trúc set là nó duy trì thứ tự của các phần tử và cung cấp những hàm không có trong `unordered_set`. Mặt khác, `unordered_set` có thể hiệu quả hơn.

Đoạn mã sau tạo một set chứa các số nguyên, và minh họa một số thao tác. Hàm `insert` thêm một phần tử vào set, hàm `count` trả về số lần xuất hiện của một phần tử trong set, và hàm `erase` xóa một phần tử khỏi set.

```
set<int> s;
s.insert(3);
s.insert(2);
s.insert(5);
cout << s.count(3) << "\n"; // 1
cout << s.count(4) << "\n"; // 0
s.erase(3);
s.insert(4);
cout << s.count(3) << "\n"; // 0
cout << s.count(4) << "\n"; // 1
```

Một set có thể được dùng phần lớn giống như vector, nhưng không thể truy cập các phần tử bằng ký hiệu []. Đoạn mã sau tạo một set, in số phần tử trong đó, rồi duyệt qua tất cả các phần tử:

```
set<int> s = {2,5,6,8};
cout << s.size() << "\n"; // 4
for (auto x : s) {
    cout << x << "\n";
}
```

Một tính chất quan trọng của set là tất cả các phần tử của nó đều *phân biệt*. Do đó, hàm `count` luôn trả về hoặc 0 (phần tử không nằm trong set) hoặc 1 (phần tử nằm trong set), và hàm `insert` không bao giờ thêm một phần tử vào set nếu phần tử đó đã có sẵn. Đoạn mã sau minh họa điều này:

```
set<int> s;
s.insert(5);
s.insert(5);
s.insert(5);
cout << s.count(5) << "\n"; // 1
```

C++ cũng chứa các cấu trúc `multiset` và `unordered_multiset` hoạt động tương tự

như set và unordered_set, nhưng chúng có thể chứa nhiều bản sao của một phần tử. Ví dụ, trong đoạn mã sau, cả ba bản sao của số 5 đều được thêm vào một multiset:

```
multiset<int> s;  
s.insert(5);  
s.insert(5);  
s.insert(5);  
cout << s.count(5) << "\n"; // 3
```

Hàm erase xóa tất cả các bản sao của một phần tử khỏi multiset:

```
s.erase(5);  
cout << s.count(5) << "\n"; // 0
```

Thông thường, chỉ nên xóa một bản sao, việc này có thể làm như sau:

```
s.erase(s.find(5));  
cout << s.count(5) << "\n"; // 2
```

4.3 Cấu trúc ánh xạ

Một **map** là một mảng tổng quát hóa gồm các cặp khóa-giá trị. Trong khi khóa trong một mảng thông thường luôn là các số nguyên liên tiếp $0, 1, \dots, n-1$, trong đó n là kích thước của mảng, khóa trong một map có thể thuộc bất kỳ kiểu dữ liệu nào và không cần là các giá trị liên tiếp.

Thư viện chuẩn C++ chứa hai cách cài đặt map tương ứng với các cách cài đặt set: cấu trúc map dựa trên một cây nhị phân cân bằng và việc truy cập phần tử mất thời gian $O(\log n)$, trong khi cấu trúc unordered_map sử dụng băm và việc truy cập phần tử mất thời gian $O(1)$ trung bình.

Đoạn mã sau tạo một map trong đó các khóa là chuỗi và các giá trị là số nguyên:

```
map<string,int> m;  
m["monkey"] = 4;  
m["banana"] = 3;  
m["harpsichord"] = 9;  
cout << m["banana"] << "\n"; // 3
```

Nếu giá trị của một khóa được yêu cầu nhưng map không chứa khóa đó, khóa sẽ tự động được thêm vào map với một giá trị mặc định. Ví dụ, trong đoạn mã sau, khóa "aybabbu" với giá trị 0 được thêm vào map.

```
map<string,int> m;  
cout << m["aybabbu"] << "\n"; // 0
```

Hàm count kiểm tra một khóa có tồn tại trong map hay không:

```
if (m.count("aybabbu")) {
```

```
} // key exists
```

Đoạn mã sau in tất cả các khóa và giá trị trong một map:

```
for (auto x : m) {  
    cout << x.first << " " << x.second << "\n";  
}
```

4.4 Bộ lặp và đoạn

Nhiều hàm trong thư viện chuẩn C++ làm việc với các bộ lặp. Một **bộ lặp (iterator)** là một biến trỏ tới một phần tử trong một cấu trúc dữ liệu.

Các bộ lặp thường dùng `begin` và `end` xác định một đoạn chứa tất cả phần tử trong một cấu trúc dữ liệu. Bộ lặp `begin` trỏ tới phần tử đầu tiên trong cấu trúc dữ liệu, và bộ lặp `end` trỏ tới vị trí *sau* phần tử cuối cùng. Tình huống như sau:

```
    { 3, 4, 6, 8, 12, 13, 14, 17 }  
      ↑                               ↑  
    s.begin()                       s.end()
```

Lưu ý sự bất đối xứng trong các bộ lặp: `s.begin()` trỏ tới một phần tử trong cấu trúc dữ liệu, trong khi `s.end()` trỏ ra ngoài cấu trúc dữ liệu. Do đó, đoạn được xác định bởi các bộ lặp là *nửa mở*.

Làm việc với đoạn

Bộ lặp được dùng trong các hàm của thư viện chuẩn C++ nhận một đoạn phần tử trong một cấu trúc dữ liệu. Thông thường, ta muốn xử lý tất cả phần tử trong một cấu trúc dữ liệu, nên các bộ lặp `begin` và `end` được truyền cho hàm.

Ví dụ, đoạn mã sau sắp xếp một vector bằng hàm `sort`, sau đó đảo thứ tự các phần tử bằng hàm `reverse`, và cuối cùng xáo trộn thứ tự các phần tử bằng hàm `random_shuffle`.

```
sort(v.begin(), v.end());  
reverse(v.begin(), v.end());  
random_shuffle(v.begin(), v.end());
```

Các hàm này cũng có thể được dùng với một mảng thông thường. Trong trường hợp này, các hàm nhận con trỏ tới mảng thay vì bộ lặp:

```
sort(a, a+n);
reverse(a, a+n);
random_shuffle(a, a+n);
```

Bộ lặp của tập hợp

Bộ lặp thường được dùng để truy cập các phần tử của một set. Đoạn mã sau tạo một bộ lặp `it` trở tới phần tử nhỏ nhất trong một set:

```
set<int>::iterator it = s.begin();
```

Một cách ngắn hơn để viết đoạn mã là như sau:

```
auto it = s.begin();
```

Có thể truy cập phần tử mà một bộ lặp trở tới bằng ký hiệu `*`. Ví dụ, đoạn mã sau in phần tử đầu tiên trong set:

```
auto it = s.begin();
cout << *it << "\n";
```

Bộ lặp có thể được di chuyển bằng các toán tử `++` (tiến) và `--` (lùi), nghĩa là bộ lặp di chuyển tới phần tử kế tiếp hoặc phần tử trước đó trong set.

Đoạn mã sau in tất cả các phần tử theo thứ tự tăng dần:

```
for (auto it = s.begin(); it != s.end(); it++) {
    cout << *it << "\n";
}
```

Đoạn mã sau in phần tử lớn nhất trong set:

```
auto it = s.end(); it--;
cout << *it << "\n";
```

Hàm `find( $x$ )` trả về một bộ lặp trở tới một phần tử có giá trị là x . Tuy nhiên, nếu set không chứa x , bộ lặp sẽ là `end`.

```
auto it = s.find(x);
if (it == s.end()) {
    // x is not found
}
```

Hàm `lower_bound( $x$ )` trả về một bộ lặp tới phần tử nhỏ nhất trong set có giá trị *ít nhất* là x , và hàm `upper_bound( $x$ )` trả về một bộ lặp tới phần tử nhỏ nhất trong set có giá trị *lớn hơn* x . Trong cả hai hàm, nếu phần tử như vậy không tồn tại, giá trị trả về là `end`. Các hàm này không được hỗ trợ bởi cấu trúc `unordered_set`, vốn không duy trì thứ tự của các phần tử.

Ví dụ, đoạn mã sau tìm phần tử gần x nhất:

```
auto it = s.lower_bound(x);
if (it == s.begin()) {
    cout << *it << "\n";
} else if (it == s.end()) {
    it--;
    cout << *it << "\n";
} else {
    int a = *it; it--;
    int b = *it;
    if (x-b < a-x) cout << b << "\n";
    else cout << a << "\n";
}
```

Đoạn mã giả sử rằng set không rỗng, và xét tất cả các trường hợp có thể bằng một bộ lặp `it`. Đầu tiên, bộ lặp trở tới phần tử nhỏ nhất có giá trị ít nhất là x . Nếu `it` bằng `begin`, phần tử tương ứng là gần x nhất. Nếu `it` bằng `end`, phần tử lớn nhất trong set là gần x nhất. Nếu không rơi vào các trường hợp trên, phần tử gần x nhất là phần tử tương ứng với `it` hoặc phần tử trước đó.

4.5 Các cấu trúc khác

Tập bit

Một **tập bit (bitset)** là một mảng trong đó mỗi giá trị là 0 hoặc 1. Ví dụ, đoạn mã sau tạo một tập bit chứa 10 phần tử:

```
bitset<10> s;
s[1] = 1;
s[3] = 1;
s[4] = 1;
s[7] = 1;
cout << s[4] << "\n"; // 1
cout << s[5] << "\n"; // 0
```

Lợi ích của việc dùng tập bit là chúng cần ít bộ nhớ hơn mảng thông thường, vì mỗi phần tử trong một tập bit chỉ dùng một bit bộ nhớ. Ví dụ, nếu n bit được lưu trong một mảng `int`, $32n$ bit bộ nhớ sẽ được dùng, nhưng tập bit tương ứng chỉ cần n bit bộ nhớ. Ngoài ra, các giá trị của một tập bit có thể được thao tác hiệu quả bằng các toán tử bit, nhờ đó có thể tối ưu hóa thuật toán bằng các tập bit.

Đoạn mã sau cho thấy một cách khác để tạo tập bit ở trên:

```
bitset<10> s(string("0010011010")); // from right to left
cout << s[4] << "\n"; // 1
cout << s[5] << "\n"; // 0
```

Hàm `count` trả về số bit một trong tập bit:

```
bitset<10> s(string("0010011010"));
cout << s.count() << "\n"; // 4
```

Đoạn mã sau cho thấy các ví dụ về việc dùng thao tác bit:

```
bitset<10> a(string("0010110110"));
bitset<10> b(string("1011011000"));
cout << (a&b) << "\n"; // 0010010000
cout << (a|b) << "\n"; // 1011111110
cout << (a^b) << "\n"; // 1001101110
```

Hàng đợi hai đầu

Một **hàng đợi hai đầu (deque)** là một mảng động mà kích thước có thể được thay đổi hiệu quả ở cả hai đầu của mảng. Giống như vector, hàng đợi hai đầu cung cấp các hàm `push_back` và `pop_back`, nhưng nó cũng có các hàm `push_front` và `pop_front` không có trong vector.

Hàng đợi hai đầu có thể được dùng như sau:

```
deque<int> d;
d.push_back(5); // [5]
d.push_back(2); // [5,2]
d.push_front(3); // [3,5,2]
d.pop_back(); // [3,5]
d.pop_front(); // [5]
```

Cài đặt bên trong của hàng đợi hai đầu phức tạp hơn của vector, và vì lý do này, hàng đợi hai đầu chậm hơn vector. Dù vậy, cả việc thêm và xóa phần tử đều mất thời gian $O(1)$ trung bình ở cả hai đầu.

Ngăn xếp

Một **ngăn xếp (stack)** là một cấu trúc dữ liệu cung cấp hai thao tác thời gian $O(1)$: thêm một phần tử vào đỉnh, và xóa một phần tử khỏi đỉnh. Chỉ có thể truy cập phần tử trên cùng của một ngăn xếp.

Đoạn mã sau cho thấy cách dùng ngăn xếp:

```
stack<int> s;
s.push(3);
s.push(2);
s.push(5);
cout << s.top(); // 5
s.pop();
cout << s.top(); // 2
```


Hàng đợi

Một **hàng đợi (queue)** cũng cung cấp hai thao tác thời gian $O(1)$: thêm một phần tử vào cuối hàng đợi, và xóa phần tử đầu tiên trong hàng đợi. Chỉ có thể truy cập phần tử đầu tiên và cuối cùng của một hàng đợi.

Đoạn mã sau cho thấy cách dùng hàng đợi:

```
queue<int> q;
q.push(3);
q.push(2);
q.push(5);
cout << q.front(); // 3
q.pop();
cout << q.front(); // 2
```

Hàng đợi ưu tiên

Một **hàng đợi ưu tiên (priority queue)** duy trì một tập hợp các phần tử. Các thao tác được hỗ trợ là chèn và, tùy vào kiểu hàng đợi, lấy và xóa phần tử nhỏ nhất hoặc lớn nhất. Việc chèn và xóa mất thời gian $O(\log n)$, và việc lấy phần tử mất thời gian $O(1)$.

Trong khi một set có thứ tự hỗ trợ hiệu quả tất cả các thao tác của hàng đợi ưu tiên, lợi ích của việc dùng hàng đợi ưu tiên là nó có các hệ số hằng nhỏ hơn. Hàng đợi ưu tiên thường được cài đặt bằng cấu trúc đống (heap), đơn giản hơn nhiều so với cây nhị phân cân bằng dùng trong một set có thứ tự.

Theo mặc định, các phần tử trong một hàng đợi ưu tiên của C++ được sắp xếp theo thứ tự giảm dần, và có thể tìm và xóa phần tử lớn nhất trong hàng đợi. Đoạn mã sau minh họa điều này:

```
priority_queue<int> q;
q.push(3);
q.push(5);
q.push(7);
q.push(2);
cout << q.top() << "\n"; // 7
q.pop();
cout << q.top() << "\n"; // 5
q.pop();
q.push(6);
cout << q.top() << "\n"; // 6
q.pop();
```

Nếu ta muốn tạo một hàng đợi ưu tiên hỗ trợ tìm và xóa phần tử nhỏ nhất, ta có thể làm như sau:

```
priority_queue<int, vector<int>, greater<int>>> q;
```

Cấu trúc dữ liệu dựa trên chính sách

Trình biên dịch g++ cũng hỗ trợ một số cấu trúc dữ liệu không thuộc thư viện chuẩn C++. Những cấu trúc như vậy được gọi là cấu trúc dữ liệu *dựa trên chính sách*. Để dùng các cấu trúc này, cần thêm các dòng sau vào mã:

```
#include <ext/pb_ds/assoc_container.hpp>
using namespace __gnu_pbds;
```

Sau đó, ta có thể định nghĩa một cấu trúc dữ liệu `indexed_set` giống như `set` nhưng có thể được đánh chỉ số như một mảng. Định nghĩa cho các giá trị `int` như sau:

```
typedef tree<int,null_type,less<int>,rb_tree_tag,
            tree_order_statistics_node_update> indexed_set;
```

Bây giờ ta có thể tạo một set như sau:

```
indexed_set s;
s.insert(2);
s.insert(3);
s.insert(7);
s.insert(9);
```

Điểm đặc biệt của set này là ta có thể truy cập các chỉ số mà các phần tử sẽ có trong một mảng đã sắp xếp. Hàm `find_by_order` trả về một bộ lặp tới phần tử ở một vị trí cho trước:

```
auto x = s.find_by_order(2);
cout << *x << "\n"; // 7
```

Và hàm `order_of_key` trả về vị trí của một phần tử cho trước:

```
cout << s.order_of_key(7) << "\n"; // 2
```

Nếu phần tử không xuất hiện trong set, ta nhận được vị trí mà phần tử đó sẽ có trong set:

```
cout << s.order_of_key(6) << "\n"; // 2
cout << s.order_of_key(8) << "\n"; // 3
```

Cả hai hàm đều hoạt động trong thời gian logarit.

4.6 So sánh với sắp xếp

Thông thường, có thể giải một bài toán bằng cấu trúc dữ liệu hoặc bằng sắp xếp. Đôi khi có những khác biệt đáng kể về hiệu quả thực tế của các cách tiếp cận này, những khác biệt có thể bị che khuất trong độ phức tạp thời gian.

Hãy xét một bài toán trong đó ta được cho hai danh sách A và B đều chứa n phần

tử. Nhiệm vụ của ta là tính số phần tử thuộc cả hai danh sách. Ví dụ, với các danh sách

$$A = [5, 2, 8, 9] \quad \text{and} \quad B = [3, 2, 9, 5],$$

đáp án là 3 vì các số 2, 5 và 9 thuộc cả hai danh sách.

Một lời giải trực tiếp cho bài toán là duyệt qua tất cả các cặp phần tử trong thời gian $O(n^2)$, nhưng tiếp theo ta sẽ tập trung vào những thuật toán hiệu quả hơn.

Thuật toán 1

Ta xây dựng một set gồm các phần tử xuất hiện trong A , và sau đó duyệt qua các phần tử của B rồi kiểm tra từng phần tử xem nó có thuộc A hay không. Điều này hiệu quả vì các phần tử của A nằm trong một set. Khi dùng cấu trúc set, độ phức tạp thời gian của thuật toán là $O(n \log n)$.

Thuật toán 2

Không cần duy trì một set có thứ tự, vì vậy thay vì cấu trúc set ta cũng có thể dùng cấu trúc `unordered_set`. Đây là một cách dễ dàng để làm thuật toán hiệu quả hơn, vì ta chỉ cần thay đổi cấu trúc dữ liệu nền. Độ phức tạp thời gian của thuật toán mới là $O(n)$.

Thuật toán 3

Thay vì dùng cấu trúc dữ liệu, ta có thể dùng sắp xếp. Đầu tiên, ta sắp xếp cả hai danh sách A và B . Sau đó, ta duyệt qua cả hai danh sách cùng lúc và tìm các phần tử chung. Độ phức tạp thời gian của sắp xếp là $O(n \log n)$, và phần còn lại của thuật toán chạy trong thời gian $O(n)$, nên tổng độ phức tạp thời gian là $O(n \log n)$.

So sánh hiệu quả

Bảng sau cho thấy các thuật toán trên hiệu quả ra sao khi n thay đổi và các phần tử của danh sách là các số nguyên ngẫu nhiên từ $1 \dots 10^9$:

n	Thuật toán 1	Thuật toán 2	Thuật toán 3
10^6	1.5 s	0.3 s	0.2 s
$2 \cdot 10^6$	3.7 s	0.8 s	0.3 s
$3 \cdot 10^6$	5.7 s	1.3 s	0.5 s
$4 \cdot 10^6$	7.7 s	1.7 s	0.7 s
$5 \cdot 10^6$	10.0 s	2.3 s	0.9 s

Thuật toán 1 và 2 giống nhau ngoại trừ việc chúng dùng các cấu trúc set khác nhau. Trong bài toán này, lựa chọn đó có ảnh hưởng quan trọng đến thời gian chạy, vì Thuật toán 2 nhanh hơn Thuật toán 1 từ 4 đến 5 lần.

Tuy nhiên, thuật toán hiệu quả nhất là Thuật toán 3, sử dụng sắp xếp. Nó chỉ dùng một nửa thời gian so với Thuật toán 2. Điều thú vị là độ phức tạp thời gian của cả

Thuật toán 1 và Thuật toán 3 đều là $O(n \log n)$, nhưng dù vậy, Thuật toán 3 nhanh hơn mười lần. Điều này có thể được giải thích bởi thực tế rằng sắp xếp là một quy trình đơn giản và chỉ được thực hiện một lần ở đầu Thuật toán 3, và phần còn lại của thuật toán chạy trong thời gian tuyến tính. Mặt khác, Thuật toán 1 duy trì một cây nhị phân cân bằng phức tạp trong suốt toàn bộ thuật toán.

Chương 5

Duyệt vét cạn

Duyệt vét cạn (complete search) là một phương pháp tổng quát có thể được dùng để giải gần như mọi bài toán thuật toán. Ý tưởng là sinh ra mọi lời giải có thể cho bài toán bằng cách duyệt vét cạn (brute force), sau đó chọn lời giải tốt nhất hoặc đếm số lời giải, tùy thuộc vào bài toán.

Duyệt vét cạn là một kỹ thuật tốt nếu có đủ thời gian để xét qua mọi lời giải, vì việc tìm kiếm thường dễ cài đặt và luôn cho đáp án đúng. Nếu duyệt vét cạn quá chậm, có thể cần đến các kỹ thuật khác, như thuật toán tham lam hoặc quy hoạch động.

5.1 Sinh tập con

Đầu tiên, ta xét bài toán sinh mọi tập con của một tập gồm n phần tử. Ví dụ, các tập con của $\{0, 1, 2\}$ là $\emptyset$, $\{0\}$, $\{1\}$, $\{2\}$, $\{0, 1\}$, $\{0, 2\}$, $\{1, 2\}$ và $\{0, 1, 2\}$. Có hai cách thông dụng để sinh các tập con: ta có thể dùng tìm kiếm đệ quy hoặc khai thác biểu diễn bit của số nguyên.

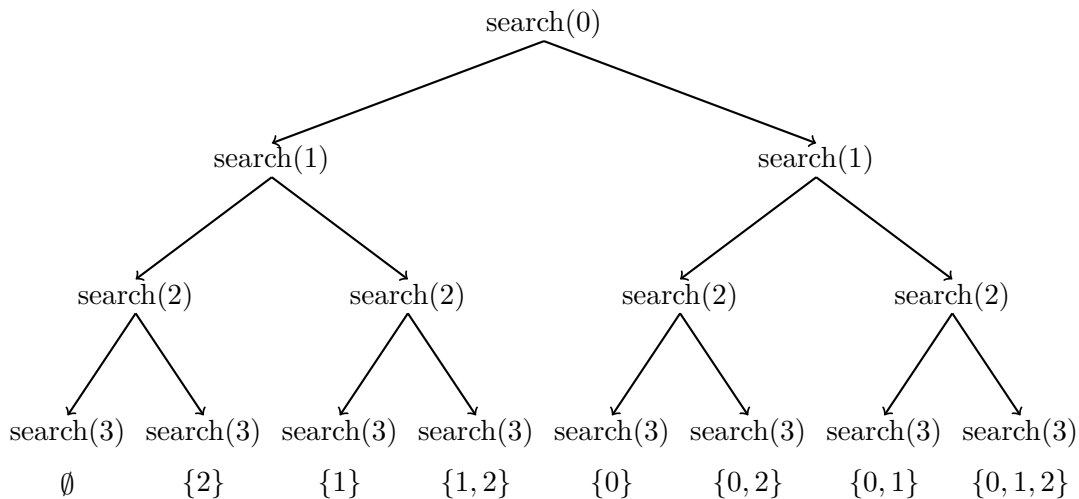
Cách 1

Một cách thanh lịch để duyệt qua mọi tập con của một tập là dùng đệ quy. Hàm `search` sau sinh các tập con của tập $\{0, 1, \dots, n-1\}$. Hàm duy trì một vector `subset` chứa các phần tử của mỗi tập con. Quá trình tìm kiếm bắt đầu khi hàm được gọi với tham số 0.

```
void search(int k) {
    if (k == n) {
        // process subset
    } else {
        search(k+1);
        subset.push_back(k);
        search(k+1);
        subset.pop_back();
    }
}
```

Khi hàm `search` được gọi với tham số k , nó quyết định có đưa phần tử k vào tập con hay không, và trong cả hai trường hợp, sau đó gọi chính nó với tham số $k + 1$. Tuy nhiên, nếu $k = n$, hàm nhận ra rằng mọi phần tử đã được xử lý và một tập con đã được sinh ra.

Cây sau minh họa các lần gọi hàm khi $n = 3$. Ở mỗi bước, ta có thể chọn nhánh trái (k không có trong tập con) hoặc nhánh phải (k có trong tập con).



Cách 2

Một cách khác để sinh tập con dựa trên biểu diễn bit của số nguyên. Mỗi tập con của một tập gồm n phần tử có thể được biểu diễn bằng một dãy n bit, tương ứng với một số nguyên trong khoảng $0 \dots 2^n - 1$. Các bit 1 trong dãy bit cho biết phần tử nào có trong tập con.

Quy ước thông thường là bit cuối tương ứng với phần tử 0, bit kế cuối tương ứng với phần tử 1, và cứ như vậy. Ví dụ, biểu diễn bit của 25 là 11001, tương ứng với tập con $\{0, 3, 4\}$.

Đoạn code sau duyệt qua các tập con của một tập gồm n phần tử

```
for (int b = 0; b < (1<<n); b++) {
    // process subset
}
```

Đoạn code sau cho thấy cách tìm các phần tử của tập con tương ứng với một dãy bit. Khi xử lý mỗi tập con, code xây dựng một vector chứa các phần tử trong tập con đó.

```
for (int b = 0; b < (1<<n); b++) {
    vector<int> subset;
    for (int i = 0; i < n; i++) {
        if (b & (1<<i)) subset.push_back(i);
    }
}
```

5.2 Sinh hoán vị

Tiếp theo, ta xét bài toán sinh mọi hoán vị (permutation) của một tập gồm n phần tử. Ví dụ, các hoán vị của $\{0, 1, 2\}$ là $(0, 1, 2)$, $(0, 2, 1)$, $(1, 0, 2)$, $(1, 2, 0)$, $(2, 0, 1)$ và $(2, 1, 0)$. Cũng có hai cách tiếp cận: ta có thể dùng đệ quy hoặc duyệt qua các hoán vị theo cách lặp.

Cách 1

Giống như tập con, hoán vị có thể được sinh bằng đệ quy. Hàm search sau duyệt qua các hoán vị của tập $\{0, 1, \dots, n-1\}$. Hàm xây dựng một vector permutation chứa hoán vị, và quá trình tìm kiếm bắt đầu khi hàm được gọi không có tham số.

```
void search() {
    if (permutation.size() == n) {
        // process permutation
    } else {
        for (int i = 0; i < n; i++) {
            if (chosen[i]) continue;
            chosen[i] = true;
            permutation.push_back(i);
            search();
            chosen[i] = false;
            permutation.pop_back();
        }
    }
}
```

Mỗi lần gọi hàm thêm một phần tử mới vào permutation. Mảng chosen cho biết phần tử nào đã có trong hoán vị. Nếu kích thước của permutation bằng kích thước của tập, một hoán vị đã được sinh ra.

Cách 2

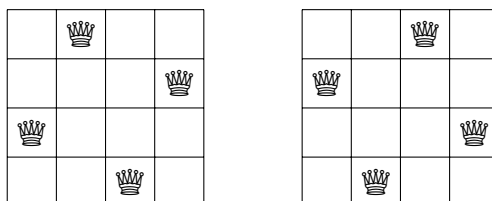
Một cách khác để sinh hoán vị là bắt đầu từ hoán vị $\{0, 1, \dots, n-1\}$ và liên tục dùng một hàm xây dựng hoán vị tiếp theo theo thứ tự tăng dần. Thư viện chuẩn C++ chứa hàm next_permutation có thể dùng cho việc này:

```
vector<int> permutation;
for (int i = 0; i < n; i++) {
    permutation.push_back(i);
}
do {
    // process permutation
} while (next_permutation(permutation.begin(), permutation.end()));
```

5.3 Quay lui

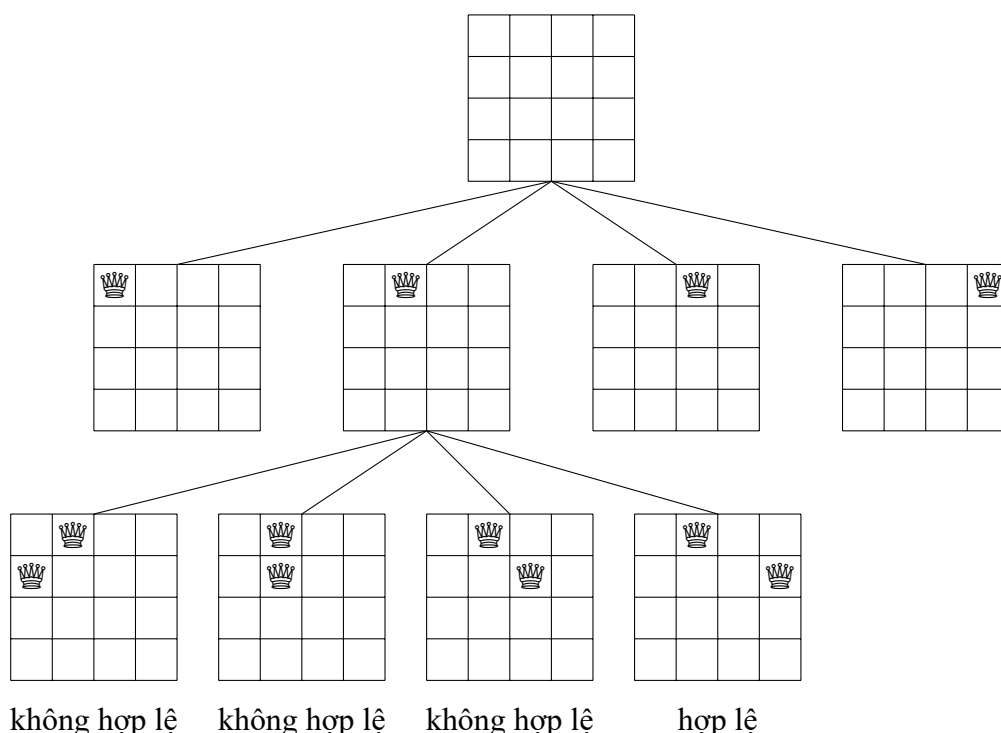
Một thuật toán **quay lui** (backtracking) bắt đầu với một lời giải rỗng và mở rộng lời giải từng bước. Quá trình tìm kiếm đệ quy duyệt qua mọi cách khác nhau mà một lời giải có thể được xây dựng.

Làm ví dụ, xét bài toán tính số cách đặt n quân hậu trên bàn cờ $n \times n$ sao cho không có hai quân hậu nào tấn công nhau. Ví dụ, khi $n = 4$, có hai lời giải có thể:



Bài toán có thể được giải bằng quay lui bằng cách đặt quân hậu lên bàn cờ theo từng hàng. Cụ thể hơn, đúng một quân hậu sẽ được đặt trên mỗi hàng sao cho không có quân hậu nào tấn công các quân hậu đã đặt trước đó. Tìm thấy một lời giải khi cả n quân hậu đều đã được đặt lên bàn cờ.

Ví dụ, khi $n = 4$, một số lời giải bộ phận do thuật toán quay lui sinh ra như sau:



Ở tầng dưới cùng, ba cấu hình đầu là không hợp lệ vì các quân hậu tấn công nhau. Tuy nhiên, cấu hình thứ tư là hợp lệ và có thể mở rộng thành một lời giải đầy đủ bằng cách đặt thêm hai quân hậu nữa lên bàn cờ. Chỉ có một cách đặt hai quân hậu còn lại.

Thuật toán có thể được cài đặt như sau:


```

void search(int y) {
    if (y == n) {
        count++;
        return;
    }
    for (int x = 0; x < n; x++) {
        if (column[x] || diag1[x+y] || diag2[x-y+n-1]) continue;
        column[x] = diag1[x+y] = diag2[x-y+n-1] = 1;
        search(y+1);
        column[x] = diag1[x+y] = diag2[x-y+n-1] = 0;
    }
}

```

Quá trình tìm kiếm bắt đầu bằng cách gọi `search(0)`. Kích thước bàn cờ là $n \times n$, và đoạn code tính số lời giải vào biến `count`.

Đoạn code giả định rằng các hàng và cột của bàn cờ được đánh số từ 0 đến $n - 1$. Khi hàm `search` được gọi với tham số y , nó đặt một quân hậu trên hàng y và sau đó gọi chính nó với tham số $y + 1$. Sau đó, nếu $y = n$, đã tìm được một lời giải và biến `count` được tăng lên 1.

Mảng `column` theo dõi các cột đã có quân hậu, và các mảng `diag1` và `diag2` theo dõi các đường chéo. Không được phép thêm một quân hậu nữa vào cột hoặc đường chéo đã chứa quân hậu. Ví dụ, các cột và đường chéo của bàn cờ 4×4 được đánh số như sau:

0	1	2	3
0	1	2	3
0	1	2	3
0	1	2	3

column

0	1	2	3
1	2	3	4
2	3	4	5
3	4	5	6

diag1

3	4	5	6
2	3	4	5
1	2	3	4
0	1	2	3

diag2

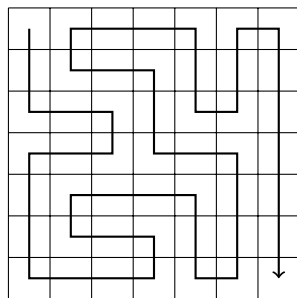
Gọi $q(n)$ là số cách đặt n quân hậu trên bàn cờ $n \times n$. Thuật toán quay lui ở trên cho ta biết, chẳng hạn, $q(8) = 92$. Khi n tăng, việc tìm kiếm nhanh chóng trở nên chậm, vì số lời giải tăng theo hàm mũ. Ví dụ, tính $q(16) = 14772512$ bằng thuật toán trên đã mất khoảng một phút trên một máy tính hiện đại¹.

5.4 Cắt tỉa cây tìm kiếm

Ta thường có thể tối ưu hóa quay lui bằng cách cắt tỉa cây tìm kiếm. Ý tưởng là thêm "sự thông minh" vào thuật toán để nó nhận ra càng sớm càng tốt khi một lời giải bộ phận không thể mở rộng thành một lời giải đầy đủ. Những tối ưu như vậy có thể có ảnh hưởng to lớn đến hiệu quả của việc tìm kiếm.

¹Hiện không có cách nào đã biết để tính hiệu quả các giá trị $q(n)$ lớn hơn. Kỷ lục hiện tại là $q(27) = 234907967154122528$, được tính năm 2016 [55].

Ta xét bài toán tính số đường đi trong một lưới $n \times n$ từ góc trên-trái đến góc dưới-phải sao cho đường đi thăm mỗi ô đúng một lần. Ví dụ, trong lưới 7×7 , có 111712 đường đi như vậy. Một trong số đó là:



Ta tập trung vào trường hợp 7×7 vì mức độ khó của nó phù hợp với nhu cầu của ta. Ta bắt đầu với một thuật toán quay lui đơn giản, rồi tối ưu hóa từng bước bằng các quan sát về việc có thể cắt tĩa quá trình tìm kiếm như thế nào. Sau mỗi lần tối ưu hóa, ta đo thời gian chạy của thuật toán và số lần gọi đệ quy, để thấy rõ tác động của từng lần tối ưu hóa đối với hiệu quả tìm kiếm.

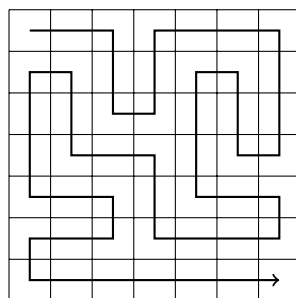
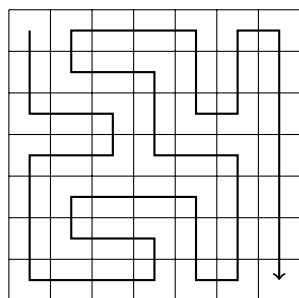
Thuật toán cơ bản

Phiên bản đầu tiên của thuật toán không chứa bất kỳ tối ưu nào. Ta chỉ đơn giản dùng quay lui để sinh mọi đường đi có thể từ góc trên-trái đến góc dưới-phải và đếm số đường đi như vậy.

- thời gian chạy: 483 giây
- số lần gọi đệ quy: 76 tỉ

Tối ưu hóa 1

Trong bất kỳ lời giải nào, đầu tiên ta đi một bước xuống hoặc sang phải. Luôn có hai đường đi đối xứng qua đường chéo của lưới sau bước đầu tiên. Ví dụ, hai đường đi sau đối xứng với nhau:

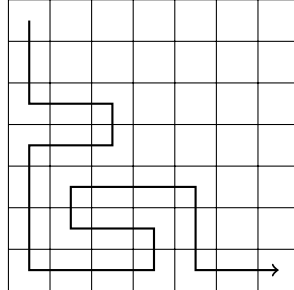


Do đó, ta có thể quy ước luôn đi bước đầu tiên xuống dưới (hoặc sang phải), và cuối cùng nhân số lời giải với hai.

- thời gian chạy: 244 giây
- số lần gọi đệ quy: 38 tỉ

Tối ưu hóa 2

Nếu đường đi đến ô dưới-phải trước khi nó thăm hết các ô khác của lưới, rõ ràng là không thể hoàn thành lời giải. Một ví dụ là đường đi sau:

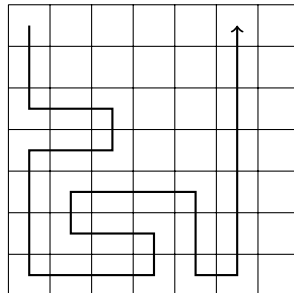


Dựa vào quan sát này, ta có thể kết thúc tìm kiếm ngay lập tức nếu đến ô dưới-phải quá sớm.

- thời gian chạy: 119 giây
- số lần gọi đệ quy: 20 tỉ

Tối ưu hóa 3

Nếu đường đi chạm vào một bức tường và có thể rẽ trái hoặc rẽ phải, lưới chia thành hai phần chứa các ô chưa thăm. Ví dụ, trong tình huống sau, đường đi có thể rẽ trái hoặc rẽ phải:

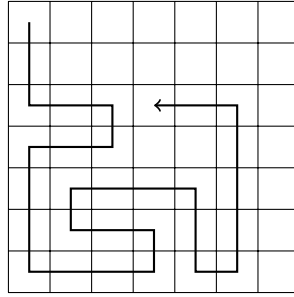


Trong trường hợp này, ta không còn có thể thăm tất cả các ô được nữa, nên ta có thể kết thúc tìm kiếm. Tối ưu hóa này rất hữu ích:

- thời gian chạy: 1.8 giây
- số lần gọi đệ quy: 221 triệu

Tối ưu hóa 4

Ý tưởng của Tối ưu hóa 3 có thể được tổng quát hóa: nếu đường đi không thể đi tiếp về phía trước nhưng có thể rẽ trái hoặc rẽ phải, lưới chia thành hai phần mà cả hai đều chứa các ô chưa thăm. Ví dụ, xét đường đi sau:



Rõ ràng ta không còn thăm hết các ô được nữa, nên ta có thể kết thúc tìm kiếm. Sau lần tối ưu hóa này, việc tìm kiếm trở nên rất hiệu quả:

- thời gian chạy: 0.6 giây
- số lần gọi đệ quy: 69 triệu

Đây là lúc tốt để dừng việc tối ưu thuật toán và xem ta đã đạt được gì. Thời gian chạy của thuật toán ban đầu là 483 giây, và bây giờ sau các lần tối ưu, thời gian chạy chỉ còn 0.6 giây. Như vậy, thuật toán đã trở nên nhanh gấp gần 1000 lần sau các tối ưu hóa.

Đây là một hiện tượng thường gặp trong quay lui, vì cây tìm kiếm thường rất lớn và ngay cả những quan sát đơn giản cũng có thể cắt tía hiệu quả việc tìm kiếm. Đặc biệt hữu ích là các tối ưu xuất hiện trong những bước đầu của thuật toán, tức là ở phía trên của cây tìm kiếm.

5.5 Gặp nhau ở giữa

Gặp nhau ở giữa (meet in the middle) là một kỹ thuật trong đó không gian tìm kiếm được chia thành hai phần có kích thước xấp xỉ bằng nhau. Một quá trình tìm kiếm riêng được thực hiện cho mỗi phần, và cuối cùng kết quả của hai lần tìm kiếm được kết hợp lại.

Kỹ thuật này có thể được dùng nếu có một cách hiệu quả để kết hợp kết quả của các lần tìm kiếm. Trong tình huống đó, hai lần tìm kiếm có thể tốn ít thời gian hơn một lần tìm kiếm lớn. Thông thường, ta có thể biến một thừa số 2^n thành một thừa số $2^{n/2}$ bằng kỹ thuật gặp nhau ở giữa.

Làm ví dụ, xét bài toán cho một danh sách gồm n số và một số x , và ta muốn tìm xem có thể chọn một số phần tử từ danh sách sao cho tổng của chúng bằng x hay không. Ví dụ, với danh sách $[2, 4, 5, 9]$ và $x = 15$, ta có thể chọn các số $[2, 4, 9]$ để có $2 + 4 + 9 = 15$. Tuy nhiên, nếu $x = 10$ với cùng danh sách đó, không thể tạo được tổng.

Một thuật toán đơn giản cho bài toán này là duyệt qua mọi tập con các phần tử và kiểm tra xem tổng của tập con nào đó có bằng x không. Thời gian chạy của thuật toán như vậy là $O(2^n)$, vì có 2^n tập con. Tuy nhiên, dùng kỹ thuật gặp nhau ở giữa, ta có thể đạt được thuật toán hiệu quả hơn với thời gian $O(2^{n/2})^2$. Lưu ý rằng $O(2^n)$ và $O(2^{n/2})$ là các độ phức tạp khác nhau vì $2^{n/2}$ bằng $\sqrt{2^n}$.

²Ý tưởng này được E. Horowitz và S. Sahni giới thiệu năm 1974 [39].

Ý tưởng là chia danh sách thành hai danh sách A và B sao cho cả hai danh sách chứa khoảng một nửa số phần tử. Lần tìm kiếm đầu sinh mọi tập con của A và lưu tổng của chúng vào một danh sách S_A . Tương tự, lần tìm kiếm thứ hai tạo một danh sách S_B từ B . Sau đó, chỉ cần kiểm tra xem có thể chọn một phần tử từ S_A và một phần tử từ S_B sao cho tổng của chúng bằng x hay không. Điều này có thể đúng chính khi có cách tạo tổng x bằng các số trong danh sách ban đầu.

Ví dụ, giả sử danh sách là $[2, 4, 5, 9]$ và $x = 15$. Đầu tiên, ta chia danh sách thành $A = [2, 4]$ và $B = [5, 9]$. Sau đó, ta tạo các danh sách $S_A = [0, 2, 4, 6]$ và $S_B = [0, 5, 9, 14]$. Trong trường hợp này, tổng $x = 15$ có thể tạo được, vì S_A chứa tổng 6, S_B chứa tổng 9, và $6 + 9 = 15$. Điều này tương ứng với lời giải $[2, 4, 9]$.

Ta có thể cài đặt thuật toán sao cho độ phức tạp thời gian của nó là $O(2^{n/2})$. Đầu tiên, ta sinh các danh sách S_A và S_B *đã sắp xếp*, việc này có thể làm trong thời gian $O(2^{n/2})$ bằng một kỹ thuật kiểu trộn. Sau đó, vì các danh sách đã được sắp xếp, ta có thể kiểm tra trong thời gian $O(2^{n/2})$ xem tổng x có thể được tạo từ S_A và S_B hay không.

Chương 6

Thuật toán tham lam

Một **thuật toán tham lam (greedy algorithm)** xây dựng một nghiệm cho bài toán bằng cách luôn đưa ra lựa chọn có vẻ tốt nhất tại thời điểm hiện tại. Thuật toán tham lam không bao giờ quay lại các lựa chọn của nó, mà trực tiếp xây dựng nghiệm cuối cùng. Vì lý do này, các thuật toán tham lam thường rất hiệu quả.

Khó khăn trong việc thiết kế thuật toán tham lam là tìm một chiến lược tham lam luôn tạo ra nghiệm tối ưu cho bài toán. Các lựa chọn tối ưu cục bộ trong một thuật toán tham lam cũng phải là tối ưu toàn cục. Thông thường, việc lập luận rằng một thuật toán tham lam là đúng khá khó.

6.1 Bài toán đồng xu

Như một ví dụ đầu tiên, ta xét một bài toán trong đó ta được cho một tập các đồng xu và nhiệm vụ là tạo ra tổng tiền n bằng các đồng xu đó. Giá trị của các đồng xu là $\text{coins} = \{c_1, c_2, \dots, c_k\}$, và mỗi đồng xu có thể được dùng bao nhiêu lần tùy ý. Cần ít nhất bao nhiêu đồng xu?

Ví dụ, nếu các đồng xu là các đồng euro (tính bằng xu)

$$\{1, 2, 5, 10, 20, 50, 100, 200\}$$

và $n = 520$, ta cần ít nhất bốn đồng xu. Nghiệm tối ưu là chọn các đồng xu $200 + 200 + 100 + 20$ có tổng bằng 520.

Thuật toán tham lam

Một thuật toán tham lam đơn giản cho bài toán luôn chọn đồng xu lớn nhất có thể, cho đến khi tạo được tổng tiền cần thiết. Thuật toán này đúng trong ví dụ trên, vì trước hết ta chọn hai đồng xu 200 xu, sau đó một đồng xu 100 xu và cuối cùng một đồng xu 20 xu. Nhưng thuật toán này có luôn đúng không?

Hóa ra nếu các đồng xu là đồng euro, thuật toán tham lam *luôn* đúng, tức là nó luôn tạo ra nghiệm với số đồng xu ít nhất có thể. Tính đúng đắn của thuật toán có thể được chứng minh như sau:

Trước hết, mỗi đồng xu 1, 5, 10, 50 và 100 xuất hiện nhiều nhất một lần trong một nghiệm tối ưu, vì nếu nghiệm chứa hai đồng xu như vậy, ta có thể thay chúng bằng

một đồng xu và thu được một nghiệm tốt hơn. Ví dụ, nếu nghiệm chứa các đồng xu $5 + 5$, ta có thể thay chúng bằng đồng xu 10.

Tương tự, các đồng xu 2 và 20 xuất hiện nhiều nhất hai lần trong một nghiệm tối ưu, vì ta có thể thay các đồng xu $2 + 2 + 2$ bằng các đồng xu $5 + 1$ và các đồng xu $20 + 20 + 20$ bằng các đồng xu $50 + 10$. Hơn nữa, một nghiệm tối ưu không thể chứa các đồng xu $2 + 2 + 1$ hoặc $20 + 20 + 10$, vì ta có thể thay chúng bằng các đồng xu 5 và 50.

Dùng các nhận xét này, ta có thể chứng minh với mỗi đồng xu x rằng không thể tạo một cách tối ưu tổng x hoặc bất kỳ tổng lớn hơn nào chỉ bằng các đồng xu nhỏ hơn x . Ví dụ, nếu $x = 100$, tổng tối ưu lớn nhất dùng các đồng xu nhỏ hơn là $50 + 20 + 20 + 5 + 2 + 2 = 99$. Do đó, thuật toán tham lam luôn chọn đồng xu lớn nhất sẽ tạo ra nghiệm tối ưu.

Ví dụ này cho thấy rằng việc lập luận một thuật toán tham lam là đúng có thể khó, ngay cả khi bản thân thuật toán rất đơn giản.

Trường hợp tổng quát

Trong trường hợp tổng quát, tập đồng xu có thể chứa bất kỳ đồng xu nào và thuật toán tham lam *không* nhất thiết tạo ra nghiệm tối ưu.

Ta có thể chứng minh rằng một thuật toán tham lam không đúng bằng cách đưa ra một phản ví dụ trong đó thuật toán cho đáp án sai. Trong bài toán này, ta dễ dàng tìm được một phản ví dụ: nếu các đồng xu là $\{1, 3, 4\}$ và tổng cần tạo là 6, thuật toán tham lam tạo ra nghiệm $4 + 1 + 1$ trong khi nghiệm tối ưu là $3 + 3$.

Hiện chưa biết liệu bài toán đồng xu tổng quát có thể được giải bằng một thuật toán tham lam nào hay không¹. Tuy nhiên, như ta sẽ thấy trong Chương 7, trong một số trường hợp, bài toán tổng quát có thể được giải hiệu quả bằng một thuật toán quy hoạch động luôn cho đáp án đúng.

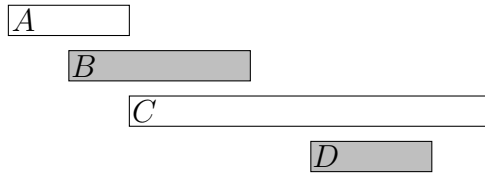
6.2 Lập lịch

Nhiều bài toán lập lịch có thể được giải bằng thuật toán tham lam. Một bài toán kinh điển như sau: Cho n sự kiện với thời điểm bắt đầu và kết thúc, hãy tìm một lịch chứa nhiều sự kiện nhất có thể. Không thể chọn một phần của sự kiện. Ví dụ, xét các sự kiện sau:

sự kiện	thời điểm bắt đầu	thời điểm kết thúc
A	1	3
B	2	5
C	3	9
D	6	8

Trong trường hợp này, số sự kiện lớn nhất là hai. Ví dụ, ta có thể chọn các sự kiện B và D như sau:

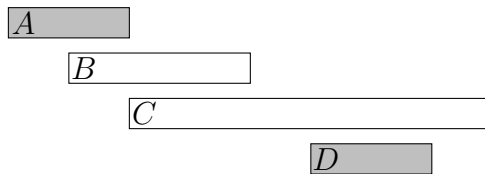
¹However, it is possible to *check* in polynomial time if the greedy algorithm presented in this chapter works for a given set of coins [53].



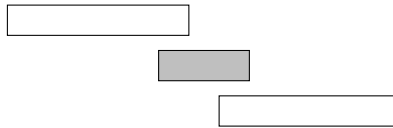
Có thể nghĩ ra nhiều thuật toán tham lam cho bài toán này, nhưng thuật toán nào đúng trong mọi trường hợp?

Thuật toán 1

Ý tưởng đầu tiên là chọn các sự kiện *ngắn* nhất có thể. Trong ví dụ này, thuật toán chọn các sự kiện sau:



Tuy nhiên, chọn các sự kiện ngắn không phải lúc nào cũng là một chiến lược đúng. Ví dụ, thuật toán thất bại trong trường hợp sau:



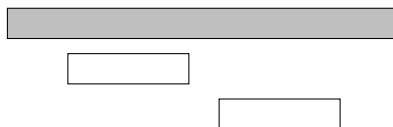
Nếu ta chọn sự kiện ngắn, ta chỉ có thể chọn một sự kiện. Tuy nhiên, có thể chọn cả hai sự kiện dài.

Thuật toán 2

Một ý tưởng khác là luôn chọn sự kiện kế tiếp có thể mà *bắt đầu* càng *sớm* càng tốt. Thuật toán này chọn các sự kiện sau:



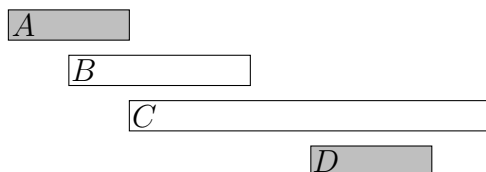
Tuy nhiên, ta cũng có thể tìm một phản ví dụ cho thuật toán này. Ví dụ, trong trường hợp sau, thuật toán chỉ chọn một sự kiện:



Nếu ta chọn sự kiện đầu tiên, không thể chọn thêm sự kiện nào khác. Tuy nhiên, có thể chọn hai sự kiện còn lại.

Thuật toán 3

Ý tưởng thứ ba là luôn chọn sự kiện kế tiếp có thể mà *kết thúc* càng *sớm* càng tốt. Thuật toán này chọn các sự kiện sau:



Hóa ra thuật toán này *luôn* tạo ra một nghiệm tối ưu. Lý do là việc chọn trước một sự kiện kết thúc càng sớm càng tốt luôn là một lựa chọn tối ưu. Sau đó, việc chọn sự kiện kế tiếp bằng cùng chiến lược cũng là tối ưu, v.v., cho đến khi ta không thể chọn thêm sự kiện nào.

Một cách lập luận rằng thuật toán đúng là xét điều gì xảy ra nếu trước hết ta chọn một sự kiện kết thúc muộn hơn sự kiện kết thúc sớm nhất có thể. Khi đó, ta sẽ có nhiều nhất cùng số lựa chọn cho cách chọn sự kiện kế tiếp. Vì vậy, chọn một sự kiện kết thúc muộn hơn không bao giờ có thể cho nghiệm tốt hơn, và thuật toán tham lam là đúng.

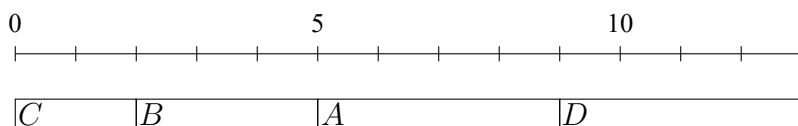
6.3 Công việc và hạn chót

Bây giờ ta xét một bài toán trong đó ta được cho n công việc với thời lượng và hạn chót và nhiệm vụ là chọn một thứ tự để thực hiện các công việc. Với mỗi công việc, ta nhận được $d - x$ điểm trong đó d là hạn chót của công việc và x là thời điểm ta hoàn thành công việc. Tổng điểm lớn nhất có thể đạt được là bao nhiêu?

Ví dụ, giả sử các công việc như sau:

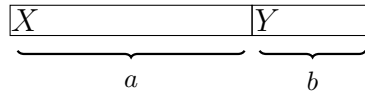
công việc	thời lượng	hạn chót
A	4	2
B	3	5
C	2	7
D	4	5

Trong trường hợp này, một lịch tối ưu cho các công việc như sau:

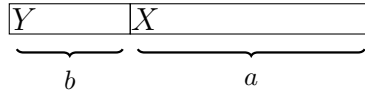


Trong nghiệm này, C cho 5 điểm, B cho 0 điểm, A cho -7 điểm và D cho -8 điểm, nên tổng điểm là -10 .

Điều đáng ngạc nhiên là nghiệm tối ưu của bài toán không phụ thuộc vào các hạn chót, mà một chiến lược tham lam đúng chỉ đơn giản là thực hiện các công việc *được sắp xếp theo thời lượng* theo thứ tự tăng dần. Lý do là nếu ta từng thực hiện hai công việc liên tiếp sao cho công việc đầu tiên mất nhiều thời gian hơn công việc thứ hai, ta có thể thu được nghiệm tốt hơn nếu đổi chỗ hai công việc. Ví dụ, xét lịch sau:



Ở đây $a > b$, nên ta nên đổi chỗ hai công việc:



Bây giờ X cho ít hơn b điểm và Y cho nhiều hơn a điểm, nên tổng điểm tăng thêm $a - b > 0$. Trong một nghiệm tối ưu, với bất kỳ hai công việc liên tiếp nào, công việc ngắn hơn phải đứng trước công việc dài hơn. Do đó, các công việc phải được thực hiện theo thứ tự thời lượng đã sắp xếp.

6.4 Tối thiểu hóa tổng

Tiếp theo ta xét một bài toán trong đó ta được cho n số $a_1, a_2, \dots, a_n$ và nhiệm vụ là tìm một giá trị x làm tối thiểu tổng

$$|a_1 - x|^c + |a_2 - x|^c + \dots + |a_n - x|^c.$$

Ta tập trung vào các trường hợp $c = 1$ và $c = 2$.

Trường hợp $c = 1$

Trong trường hợp này, ta cần tối thiểu hóa tổng

$$|a_1 - x| + |a_2 - x| + \dots + |a_n - x|.$$

Ví dụ, nếu các số là $[1, 2, 9, 2, 6]$, nghiệm tốt nhất là chọn $x = 2$, tạo ra tổng

$$|1 - 2| + |2 - 2| + |9 - 2| + |2 - 2| + |6 - 2| = 12.$$

Trong trường hợp tổng quát, lựa chọn tốt nhất cho x là *trung vị* của các số, tức là số ở giữa sau khi sắp xếp. Ví dụ, danh sách $[1, 2, 9, 2, 6]$ trở thành $[1, 2, 2, 6, 9]$ sau khi sắp xếp, nên trung vị là 2.

Trung vị là một lựa chọn tối ưu, vì nếu x nhỏ hơn trung vị, tổng sẽ nhỏ hơn khi tăng x , và nếu x lớn hơn trung vị, tổng sẽ nhỏ hơn khi giảm x . Vì vậy, nghiệm tối ưu là x bằng trung vị. Nếu n chẵn và có hai trung vị, cả hai trung vị và mọi giá trị nằm giữa chúng đều là lựa chọn tối ưu.

Trường hợp $c = 2$

Trong trường hợp này, ta cần tối thiểu hóa tổng

$$(a_1 - x)^2 + (a_2 - x)^2 + \dots + (a_n - x)^2.$$

Ví dụ, nếu các số là $[1, 2, 9, 2, 6]$, nghiệm tốt nhất là chọn $x = 4$, tạo ra tổng

$$(1 - 4)^2 + (2 - 4)^2 + (9 - 4)^2 + (2 - 4)^2 + (6 - 4)^2 = 46.$$

Trong trường hợp tổng quát, lựa chọn tốt nhất cho x là *trung bình* của các số. Trong ví dụ, trung bình là $(1 + 2 + 9 + 2 + 6)/5 = 4$. Kết quả này có thể được suy ra bằng cách biểu diễn tổng như sau:

$$nx^2 - 2x(a_1 + a_2 + \dots + a_n) + (a_1^2 + a_2^2 + \dots + a_n^2)$$

Phần cuối cùng không phụ thuộc vào x , nên ta có thể bỏ qua nó. Các phần còn lại tạo thành một hàm $nx^2 - 2xs$ trong đó $s = a_1 + a_2 + \dots + a_n$. Đây là một parabol mở lên trên với các nghiệm $x = 0$ và $x = 2s/n$, và giá trị nhỏ nhất là trung bình của các nghiệm $x = s/n$, tức là trung bình của các số $a_1, a_2, \dots, a_n$.

6.5 Nén dữ liệu

Một **mã nhị phân (binary code)** gán cho mỗi ký tự của một chuỗi một **từ mã (codeword)** gồm các bit. Ta có thể *nén* chuỗi bằng mã nhị phân bằng cách thay mỗi ký tự bằng từ mã tương ứng. Ví dụ, mã nhị phân sau gán các từ mã cho các ký tự A–D:

ký tự	từ mã
A	00
B	01
C	10
D	11

Đây là một mã **độ dài cố định (constant-length)** nghĩa là độ dài của mỗi từ mã là như nhau. Ví dụ, ta có thể nén chuỗi AABACDACA như sau:

00 00 01 00 10 11 00 10 00

Dùng mã này, độ dài của chuỗi đã nén là 18 bit. Tuy nhiên, ta có thể nén chuỗi tốt hơn nếu dùng một mã **độ dài biến đổi (variable-length)** trong đó các từ mã có thể có độ dài khác nhau. Khi đó ta có thể gán từ mã ngắn cho các ký tự xuất hiện thường xuyên và từ mã dài cho các ký tự xuất hiện hiếm. Hóa ra một mã **tối ưu (optimal)** cho chuỗi trên như sau:

ký tự	từ mã
A	0
B	110
C	10
D	111

Một mã tối ưu tạo ra chuỗi đã nén ngắn nhất có thể. Trong trường hợp này, chuỗi được nén bằng mã tối ưu là

0 0 110 0 10 111 0 10 0,

nên chỉ cần 15 bit thay vì 18 bit. Do đó, nhờ một mã tốt hơn, ta đã có thể tiết kiệm 3 bit trong chuỗi đã nén.

Ta yêu cầu không có từ mã nào là tiền tố của một từ mã khác. Ví dụ, một mã không được phép chứa cả hai từ mã 10 và 1011. Lý do là ta muốn có thể khôi phục chuỗi ban đầu từ chuỗi đã nén. Nếu một từ mã có thể là tiền tố của một từ mã khác, điều này không phải lúc nào cũng khả thi. Ví dụ, mã sau là *không* hợp lệ:

ký tự	từ mã
A	10
B	11
C	1011
D	111

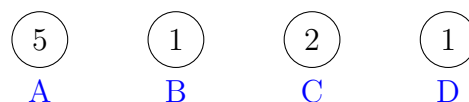
Dùng mã này, sẽ không thể biết chuỗi đã nén 1011 tương ứng với chuỗi AB hay chuỗi C.

Mã hóa Huffman

mã hóa Huffman (Huffman coding)² là một thuật toán tham lam xây dựng một mã tối ưu để nén một chuỗi cho trước. Thuật toán xây dựng một cây nhị phân dựa trên tần suất của các ký tự trong chuỗi, và từ mã của mỗi ký tự có thể được đọc bằng cách đi theo một đường đi từ gốc tới đỉnh tương ứng. Đi sang trái tương ứng với bit 0, và đi sang phải tương ứng với bit 1.

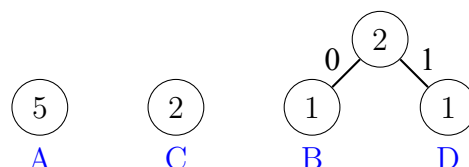
Ban đầu, mỗi ký tự của chuỗi được biểu diễn bởi một đỉnh có trọng số là số lần ký tự đó xuất hiện trong chuỗi. Sau đó, ở mỗi bước, hai đỉnh có trọng số nhỏ nhất được kết hợp bằng cách tạo một đỉnh mới có trọng số là tổng trọng số của các đỉnh ban đầu. Quá trình tiếp tục cho đến khi tất cả các đỉnh đã được kết hợp.

Tiếp theo ta sẽ xem mã hóa Huffman tạo mã tối ưu cho chuỗi AABACDACA như thế nào. Ban đầu, có bốn đỉnh tương ứng với các ký tự của chuỗi:



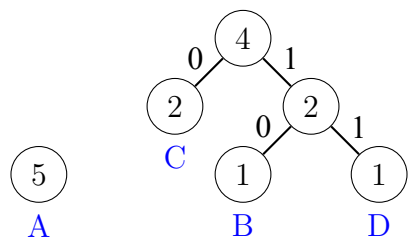
Đỉnh biểu diễn ký tự A có trọng số 5 vì ký tự A xuất hiện 5 lần trong chuỗi. Các trọng số khác cũng được tính tương tự.

Bước đầu tiên là kết hợp các đỉnh tương ứng với các ký tự B và D, cả hai có trọng số 1. Kết quả là:

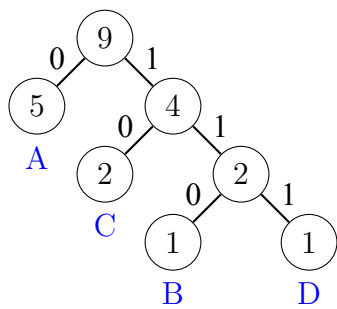


Sau đó, các đỉnh có trọng số 2 được kết hợp:

²D. A. Huffman discovered this method when solving a university course assignment and published the algorithm in 1952 [40].



Cuối cùng, hai đỉnh còn lại được kết hợp:



Bây giờ tất cả các đỉnh đều nằm trong cây, nên mã đã sẵn sàng. Các từ mã sau có thể được đọc từ cây:

ký tự	từ mã
A	0
B	110
C	10
D	111

Chương 7

Quy hoạch động

quy hoạch động (Dynamic programming) là một kỹ thuật kết hợp tính đúng đắn của tìm kiếm đầy đủ và tính hiệu quả của các thuật toán tham lam. Quy hoạch động có thể được áp dụng nếu bài toán có thể được chia thành các bài toán con chồng lấp có thể được giải độc lập.

Quy hoạch động có hai cách dùng:

- **Tìm nghiệm tối ưu (Finding an optimal solution):** Ta muốn tìm một nghiệm lớn nhất có thể hoặc nhỏ nhất có thể.
- **Đếm số nghiệm (Counting the number of solutions):** Ta muốn tính tổng số nghiệm có thể.

Trước hết ta sẽ xem quy hoạch động có thể được dùng để tìm nghiệm tối ưu như thế nào, rồi dùng cùng ý tưởng đó để đếm số nghiệm.

Hiểu quy hoạch động là một cột mốc trong sự nghiệp của mọi lập trình viên thi đấu. Mặc dù ý tưởng cơ bản rất đơn giản, thách thức nằm ở cách áp dụng quy hoạch động cho các bài toán khác nhau. Chương này giới thiệu một tập các bài toán kinh điển là điểm khởi đầu tốt.

7.1 Bài toán đồng xu

Trước hết ta tập trung vào một bài toán mà ta đã thấy trong Chương 6: Cho một tập giá trị đồng xu $\text{coins} = \{c_1, c_2, \dots, c_k\}$ và một tổng tiền mục tiêu n , nhiệm vụ của ta là tạo tổng n bằng ít đồng xu nhất có thể.

Trong Chương 6, ta đã giải bài toán bằng một thuật toán tham lam luôn chọn đồng xu lớn nhất có thể. Thuật toán tham lam đúng, ví dụ, khi các đồng xu là đồng euro, nhưng trong trường hợp tổng quát, thuật toán tham lam không nhất thiết tạo ra nghiệm tối ưu.

Bây giờ là lúc giải bài toán một cách hiệu quả bằng quy hoạch động, để thuật toán hoạt động với mọi tập đồng xu. Thuật toán quy hoạch động dựa trên một hàm đệ quy duyệt qua mọi khả năng để tạo tổng, giống như một thuật toán duyệt vét cạn. Tuy nhiên, thuật toán quy hoạch động hiệu quả vì nó dùng *ghi nhớ (memoization)* và chỉ tính đáp án cho mỗi bài toán con một lần.

Công thức đệ quy

Ý tưởng trong quy hoạch động là phát biểu bài toán theo dạng đệ quy sao cho nghiệm của bài toán có thể được tính từ nghiệm của các bài toán con nhỏ hơn. Trong bài toán đồng xu, một bài toán đệ quy tự nhiên như sau: số đồng xu nhỏ nhất cần để tạo tổng x là bao nhiêu?

Gọi $\text{solve}(x)$ là số đồng xu nhỏ nhất cần cho tổng x . Các giá trị của hàm phụ thuộc vào giá trị của các đồng xu. Ví dụ, nếu $\text{coins} = \{1, 3, 4\}$, các giá trị đầu tiên của hàm như sau:

$\text{solve}(0)$	$=$	0
$\text{solve}(1)$	$=$	1
$\text{solve}(2)$	$=$	2
$\text{solve}(3)$	$=$	1
$\text{solve}(4)$	$=$	1
$\text{solve}(5)$	$=$	2
$\text{solve}(6)$	$=$	2
$\text{solve}(7)$	$=$	2
$\text{solve}(8)$	$=$	2
$\text{solve}(9)$	$=$	3
$\text{solve}(10)$	$=$	3

Ví dụ, $\text{solve}(10) = 3$, vì cần ít nhất 3 đồng xu để tạo tổng 10. Nghiệm tối ưu là $3 + 3 + 4 = 10$.

Tính chất thiết yếu của solve là các giá trị của nó có thể được tính đệ quy từ các giá trị nhỏ hơn. Ý tưởng là tập trung vào đồng xu *đầu tiên* mà ta chọn cho tổng. Ví dụ, trong tình huống trên, đồng xu đầu tiên có thể là 1, 3 hoặc 4. Nếu trước hết ta chọn đồng xu 1, nhiệm vụ còn lại là tạo tổng 9 bằng số đồng xu ít nhất, đây là một bài toán con của bài toán ban đầu. Dĩ nhiên, điều tương tự cũng đúng với các đồng xu 3 và 4. Do đó, ta có thể dùng công thức đệ quy sau để tính số đồng xu nhỏ nhất:

$$\begin{aligned}\text{solve}(x) = \min(&\text{solve}(x - 1) + 1, \\ &\text{solve}(x - 3) + 1, \\ &\text{solve}(x - 4) + 1).\end{aligned}$$

Trường hợp cơ sở của đệ quy là $\text{solve}(0) = 0$, vì không cần đồng xu nào để tạo tổng rỗng. Ví dụ,

$$\text{solve}(10) = \text{solve}(7) + 1 = \text{solve}(4) + 2 = \text{solve}(0) + 3 = 3.$$

Bây giờ ta đã sẵn sàng đưa ra một hàm đệ quy tổng quát tính số đồng xu nhỏ nhất cần để tạo tổng x :

$$\text{solve}(x) = \begin{cases} \infty & x < 0 \\ 0 & x = 0 \\ \min_{c \in \text{coins}} \text{solve}(x - c) + 1 & x > 0 \end{cases}$$

Trước hết, nếu $x < 0$, giá trị là ∞ , vì không thể tạo một tổng tiền âm. Tiếp theo, nếu $x = 0$, giá trị là 0, vì không cần đồng xu nào để tạo tổng rỗng. Cuối cùng, nếu $x > 0$, biến c duyệt qua tất cả khả năng chọn đồng xu đầu tiên của tổng.

Khi đã tìm được một hàm đệ quy giải bài toán, ta có thể trực tiếp cài đặt một lời giải bằng C++ (hằng INF biểu diễn vô cùng):

```
int solve(int x) {
    if (x < 0) return INF;
    if (x == 0) return 0;
    int best = INF;
    for (auto c : coins) {
        best = min(best, solve(x-c)+1);
    }
    return best;
}
```

Tuy nhiên, hàm này vẫn chưa hiệu quả, vì có thể có số cách theo hàm mũ để tạo tổng. Tiếp theo ta sẽ xem cách làm cho hàm hiệu quả bằng một kỹ thuật gọi là ghi nhớ.

Dùng ghi nhớ

Ý tưởng của quy hoạch động là dùng **ghi nhớ (memoization)** để tính hiệu quả các giá trị của một hàm đệ quy. Điều này có nghĩa là các giá trị của hàm được lưu trong một mảng sau khi tính. Với mỗi tham số, giá trị của hàm chỉ được tính đệ quy một lần, và sau đó giá trị có thể được lấy trực tiếp từ mảng.

Trong bài toán này, ta dùng các mảng

```
bool ready[N];
int value[N];
```

trong đó `ready[x]` cho biết giá trị của `solve(x)` đã được tính hay chưa, và nếu đã tính, `value[x]` chứa giá trị này. Hằng N đã được chọn sao cho tất cả giá trị cần thiết đều nằm trong các mảng.

Bây giờ hàm có thể được cài đặt hiệu quả như sau:

```
int solve(int x) {
    if (x < 0) return INF;
    if (x == 0) return 0;
    if (ready[x]) return value[x];
    int best = INF;
    for (auto c : coins) {
        best = min(best, solve(x-c)+1);
    }
    value[x] = best;
    ready[x] = true;
    return best;
}
```

Hàm xử lý các trường hợp cơ sở $x < 0$ và $x = 0$ như trước. Sau đó hàm kiểm tra từ `ready[x]` xem `solve(x)` đã được lưu trong `value[x]` hay chưa, và nếu có, hàm trả về trực tiếp giá trị đó. Ngược lại, hàm tính giá trị của `solve(x)` một cách đệ quy và lưu nó trong `value[x]`.

Hàm này hoạt động hiệu quả, vì đáp án cho mỗi tham số x chỉ được tính đệ quy một lần. Sau khi một giá trị của `solve(x)` đã được lưu trong `value[x]`, nó có thể được lấy hiệu quả mỗi khi hàm được gọi lại với tham số x . Độ phức tạp thời gian của thuật toán là $O(nk)$, trong đó n là tổng mục tiêu và k là số đồng xu.

Lưu ý rằng ta cũng có thể xây dựng mảng `value` một cách *lặp* bằng một vòng lặp đơn giản tính tất cả các giá trị của `solve` cho các tham số $0 \dots n$:

```
value[0] = 0;
for (int x = 1; x <= n; x++) {
    value[x] = INF;
    for (auto c : coins) {
        if (x-c >= 0) {
            value[x] = min(value[x], value[x-c]+1);
        }
    }
}
```

Trên thực tế, hầu hết lập trình viên thi đấu thích cách cài đặt này hơn, vì nó ngắn gọn và có hệ số hằng nhỏ hơn. Từ giờ trở đi, ta cũng dùng các cài đặt dạng lặp trong các ví dụ. Dù vậy, thường dễ suy nghĩ về các lời giải quy hoạch động hơn theo các hàm đệ quy.

Xây dựng một nghiệm

Đôi khi ta được yêu cầu vừa tìm giá trị của nghiệm tối ưu vừa đưa ra một ví dụ về cách xây dựng nghiệm đó. Ví dụ, trong bài toán đồng xu, ta có thể khai báo một mảng khác cho biết với mỗi tổng tiền, đồng xu đầu tiên trong một nghiệm tối ưu:

```
int first[N];
```

Sau đó, ta có thể sửa thuật toán như sau:

```
value[0] = 0;
for (int x = 1; x <= n; x++) {
    value[x] = INF;
    for (auto c : coins) {
        if (x-c >= 0 && value[x-c]+1 < value[x]) {
            value[x] = value[x-c]+1;
            first[x] = c;
        }
    }
}
```

Sau đó, đoạn mã sau có thể được dùng để in các đồng xu xuất hiện trong một nghiệm

tối ưu cho tổng n :

```
while (n > 0) {  
    cout << first[n] << "\n";  
    n -= first[n];  
}
```

Đếm số nghiệm

Bây giờ ta xét một phiên bản khác của bài toán đồng xu, trong đó nhiệm vụ là tính tổng số cách tạo tổng x bằng các đồng xu. Ví dụ, nếu $\text{coins} = \{1, 3, 4\}$ và $x = 5$, có tổng cộng 6 cách:

- $1 + 1 + 1 + 1 + 1$
- $1 + 1 + 3$
- $1 + 3 + 1$
- $3 + 1 + 1$
- $1 + 4$
- $4 + 1$

Một lần nữa, ta có thể giải bài toán bằng đệ quy. Gọi $\text{solve}(x)$ là số cách ta có thể tạo tổng x . Ví dụ, nếu $\text{coins} = \{1, 3, 4\}$, thì $\text{solve}(5) = 6$ và công thức đệ quy là

$$\begin{aligned}\text{solve}(x) = & \text{solve}(x - 1) + \\ & \text{solve}(x - 3) + \\ & \text{solve}(x - 4).\end{aligned}$$

Khi đó, hàm đệ quy tổng quát như sau:

$$\text{solve}(x) = \begin{cases} 0 & x < 0 \\ 1 & x = 0 \\ \sum_{c \in \text{coins}} \text{solve}(x - c) & x > 0 \end{cases}$$

Nếu $x < 0$, giá trị là 0, vì không có nghiệm nào. Nếu $x = 0$, giá trị là 1, vì chỉ có một cách để tạo tổng rỗng. Ngược lại, ta tính tổng của tất cả giá trị dạng $\text{solve}(x - c)$ trong đó c thuộc coins .

Đoạn mã sau xây dựng một mảng count sao cho $\text{count}[x]$ bằng giá trị của $\text{solve}(x)$ với $0 \leq x \leq n$:

```
count[0] = 1;  
for (int x = 1; x <= n; x++) {  
    for (auto c : coins) {  
        if (x - c >= 0) {  
            count[x] += count[x - c];  
        }  
    }  
}
```

Thông thường số nghiệm quá lớn đến mức không cần tính số chính xác, mà chỉ cần đưa ra đáp án modulo m , trong đó, ví dụ, $m = 10^9 + 7$. Điều này có thể được thực hiện bằng cách sửa mã sao cho mọi phép tính đều được thực hiện modulo m . Trong đoạn mã trên, chỉ cần thêm dòng

```
count[x] %= m;
```

sau dòng

```
count[x] += count[x-c];
```

Bây giờ ta đã thảo luận tất cả các ý tưởng cơ bản của quy hoạch động. Vì quy hoạch động có thể được dùng trong nhiều tình huống khác nhau, ta sẽ tiếp tục đi qua một tập các bài toán minh họa thêm về khả năng của quy hoạch động.

7.2 Dãy con tăng dài nhất

Bài toán đầu tiên của ta là tìm **dãy con tăng dài nhất (longest increasing subsequence)** trong một mảng gồm n phần tử. Đây là một dãy có độ dài lớn nhất gồm các phần tử của mảng đi từ trái sang phải, và mỗi phần tử trong dãy lớn hơn phần tử trước đó. Ví dụ, trong mảng

0	1	2	3	4	5	6	7
6	2	5	1	7	4	8	3

dãy con tăng dài nhất chứa 4 phần tử:

0	1	2	3	4	5	6	7
6	2	5	1	7	4	8	3



Gọi $\text{length}(k)$ là độ dài của dãy con tăng dài nhất kết thúc tại vị trí k . Do đó, nếu ta tính tất cả giá trị $\text{length}(k)$ với $0 \leq k \leq n - 1$, ta sẽ biết độ dài của dãy con tăng dài nhất. Ví dụ, các giá trị của hàm cho mảng trên như sau:

```
length(0) = 1
length(1) = 1
length(2) = 2
length(3) = 1
length(4) = 3
length(5) = 2
length(6) = 4
length(7) = 2
```

Ví dụ, $\text{length}(6) = 4$, vì dãy con tăng dài nhất kết thúc tại vị trí 6 gồm 4 phần tử.

Để tính giá trị của $\text{length}(k)$, ta cần tìm một vị trí $i < k$ sao cho $\text{array}[i] < \text{array}[k]$ và $\text{length}(i)$ lớn nhất có thể. Khi đó ta biết rằng $\text{length}(k) = \text{length}(i) + 1$, vì đây là cách tối ưu để thêm $\text{array}[k]$ vào một dãy con. Tuy nhiên, nếu không có vị trí i như vậy, thì $\text{length}(k) = 1$, nghĩa là dãy con chỉ chứa $\text{array}[k]$.

Vì mọi giá trị của hàm có thể được tính từ các giá trị nhỏ hơn, ta có thể dùng quy hoạch động. Trong đoạn mã sau, các giá trị của hàm sẽ được lưu trong một mảng length .

```
for (int k = 0; k < n; k++) {
    length[k] = 1;
    for (int i = 0; i < k; i++) {
        if (array[i] < array[k]) {
            length[k] = max(length[k], length[i] + 1);
        }
    }
}
```

Đoạn mã này chạy trong thời gian $O(n^2)$, vì nó gồm hai vòng lặp lồng nhau. Tuy nhiên, cũng có thể cài đặt phép tính quy hoạch động hiệu quả hơn trong thời gian $O(n \log n)$. Bạn có tìm được cách làm điều này không?

7.3 Đường đi trong lưới

Bài toán tiếp theo của ta là tìm một đường đi từ góc trên bên trái tới góc dưới bên phải của một lưới $n \times n$, sao cho ta chỉ di chuyển xuống và sang phải. Mỗi ô chứa một số nguyên dương, và đường đi cần được xây dựng sao cho tổng các giá trị trên đường đi là lớn nhất có thể.

Hình sau cho thấy một đường đi tối ưu trong một lưới:

3	7	9	2	7
9	8	3	5	5
1	7	9	8	5
3	8	6	4	10
6	3	9	7	8

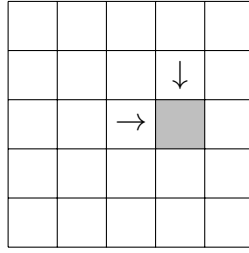
Tổng các giá trị trên đường đi là 67, và đây là tổng lớn nhất có thể trên một đường đi từ góc trên bên trái tới góc dưới bên phải.

Giả sử rằng các hàng và cột của lưới được đánh số từ 1 đến n , và $\text{value}[y][x]$ bằng giá trị của ô (y, x) . Gọi $\text{sum}(y, x)$ là tổng lớn nhất trên một đường đi từ góc trên bên trái tới ô (y, x) . Bây giờ $\text{sum}(n, n)$ cho ta tổng lớn nhất từ góc trên bên trái tới góc dưới bên phải. Ví dụ, trong lưới trên, $\text{sum}(5, 5) = 67$.

Ta có thể tính các tổng một cách đệ quy như sau:

$$\text{sum}(y, x) = \max(\text{sum}(y, x - 1), \text{sum}(y - 1, x)) + \text{value}[y][x]$$

Công thức đệ quy dựa trên nhận xét rằng một đường đi kết thúc tại ô (y, x) có thể đến từ ô $(y, x - 1)$ hoặc ô $(y - 1, x)$:



Do đó, ta chọn hướng tối đa hóa tổng. Ta giả sử rằng $\text{sum}(y, x) = 0$ nếu $y = 0$ hoặc $x = 0$ (vì không tồn tại các đường đi như vậy), nên công thức đệ quy cũng hoạt động khi $y = 1$ hoặc $x = 1$.

Vì hàm sum có hai tham số, mảng quy hoạch động cũng có hai chiều. Ví dụ, ta có thể dùng một mảng

```
int sum[N][N];
```

và tính các tổng như sau:

```
for (int y = 1; y <= n; y++) {
    for (int x = 1; x <= n; x++) {
        sum[y][x] = max(sum[y][x-1], sum[y-1][x]) + value[y][x];
    }
}
```

Độ phức tạp thời gian của thuật toán là $O(n^2)$.

7.4 Bài toán cái túi

Thuật ngữ **cái túi (knapsack)** chỉ các bài toán trong đó một tập các đối tượng được cho, và cần tìm các tập con có một số tính chất nào đó. Các bài toán cái túi thường có thể được giải bằng quy hoạch động.

Trong mục này, ta tập trung vào bài toán sau: Cho một danh sách trọng số $[w_1, w_2, \dots, w_n]$, xác định tất cả các tổng có thể được tạo bằng các trọng số. Ví dụ, nếu các trọng số là $[1, 3, 3, 5]$, các tổng sau là có thể:

0	1	2	3	4	5	6	7	8	9	10	11	12
X	X		X	X	X	X	X	X	X		X	X

Trong trường hợp này, mọi tổng từ $0 \dots 12$ đều có thể, trừ 2 và 10. Ví dụ, tổng 7 là có thể vì ta có thể chọn các trọng số $[1, 3, 3]$.

Để giải bài toán, ta tập trung vào các bài toán con trong đó chỉ dùng k trọng số đầu tiên để tạo các tổng. Gọi $\text{possible}(x, k) = \text{true}$ nếu ta có thể tạo tổng x bằng k trọng số đầu tiên, và ngược lại $\text{possible}(x, k) = \text{false}$. Các giá trị của hàm có thể được tính đệ quy như sau:

$$\text{possible}(x, k) = \text{possible}(x - w_k, k - 1) \vee \text{possible}(x, k - 1)$$

Công thức dựa trên thực tế rằng ta có thể dùng hoặc không dùng trọng số w_k trong tổng. Nếu ta dùng w_k , nhiệm vụ còn lại là tạo tổng $x - w_k$ bằng $k - 1$ trọng số đầu

tiên, và nếu ta không dùng w_k , nhiệm vụ còn lại là tạo tổng x bằng $k - 1$ trọng số đầu tiên. Các trường hợp cơ sở là

$$\text{possible}(x, 0) = \begin{cases} \text{true} & x = 0 \\ \text{false} & x \neq 0 \end{cases}$$

vì nếu không dùng trọng số nào, ta chỉ có thể tạo tổng 0.

Bảng sau cho thấy mọi giá trị của hàm cho các trọng số $[1, 3, 3, 5]$ (ký hiệu "X" chỉ các giá trị true):

$k \backslash x$	0	1	2	3	4	5	6	7	8	9	10	11	12
0	X												
1	X	X											
2	X	X		X	X								
3	X	X		X	X		X	X					
4	X	X		X	X	X	X	X	X	X		X	X

Sau khi tính các giá trị đó, $\text{possible}(x, n)$ cho ta biết liệu ta có thể tạo tổng x bằng *tất cả* trọng số hay không.

Gọi W là tổng của các trọng số. Lời giải quy hoạch động thời gian $O(nW)$ sau đây tương ứng với hàm đệ quy:

```
possible[0][0] = true;
for (int k = 1; k <= n; k++) {
    for (int x = 0; x <= W; x++) {
        if (x - w[k] >= 0) possible[x][k] |= possible[x - w[k]][k - 1];
        possible[x][k] |= possible[x][k - 1];
    }
}
```

Tuy nhiên, đây là một cách cài đặt tốt hơn chỉ dùng một mảng một chiều $\text{possible}[x]$ cho biết liệu ta có thể tạo một tập con có tổng x hay không. Mẹo là cập nhật mảng từ phải sang trái cho mỗi trọng số mới:

```
possible[0] = true;
for (int k = 1; k <= n; k++) {
    for (int x = W; x >= 0; x--) {
        if (possible[x]) possible[x + w[k]] = true;
    }
}
```

Lưu ý rằng ý tưởng tổng quát được trình bày ở đây có thể được dùng trong nhiều bài toán cái túi. Ví dụ, nếu ta được cho các đối tượng có trọng số và giá trị, ta có thể xác định với mỗi tổng trọng số, tổng giá trị lớn nhất của một tập con.

7.5 Khoảng cách chỉnh sửa

khoảng cách chỉnh sửa (edit distance) hay **khoảng cách Levenshtein (Levenshtein distance)**¹ là số thao tác chỉnh sửa nhỏ nhất cần để biến một chuỗi thành một chuỗi khác. Các thao tác chỉnh sửa được phép như sau:

- chèn một ký tự (ví dụ $ABC \rightarrow ABCA$)
- xóa một ký tự (ví dụ $ABC \rightarrow AC$)
- sửa một ký tự (ví dụ $ABC \rightarrow ADC$)

Ví dụ, khoảng cách chỉnh sửa giữa LOVE và MOVIE là 2, vì trước hết ta có thể thực hiện thao tác $LOVE \rightarrow MOVE$ (sửa) rồi thao tác $MOVE \rightarrow MOVIE$ (chèn). Đây là số thao tác nhỏ nhất có thể, vì rõ ràng chỉ một thao tác là không đủ.

Giả sử ta được cho một chuỗi x có độ dài n và một chuỗi y có độ dài m , và ta muốn tính khoảng cách chỉnh sửa giữa x và y . Để giải bài toán, ta định nghĩa một hàm $distance(a, b)$ cho biết khoảng cách chỉnh sửa giữa các tiền tố $x[0 \dots a]$ và $y[0 \dots b]$. Do đó, dùng hàm này, khoảng cách chỉnh sửa giữa x và y bằng $distance(n-1, m-1)$.

Ta có thể tính các giá trị của $distance$ như sau:

$$distance(a, b) = \min(\begin{aligned} &distance(a, b-1) + 1, \\ &distance(a-1, b) + 1, \\ &distance(a-1, b-1) + cost(a, b). \end{aligned})$$

Ở đây $cost(a, b) = 0$ nếu $x[a] = y[b]$, và ngược lại $cost(a, b) = 1$. Công thức xét các cách sau để chỉnh sửa chuỗi x :

- $distance(a, b-1)$: chèn một ký tự vào cuối x
- $distance(a-1, b)$: xóa ký tự cuối khỏi x
- $distance(a-1, b-1)$: khớp hoặc sửa ký tự cuối của x

Trong hai trường hợp đầu, cần một thao tác chỉnh sửa (chèn hoặc xóa). Trong trường hợp cuối, nếu $x[a] = y[b]$, ta có thể khớp các ký tự cuối mà không cần chỉnh sửa, và ngược lại cần một thao tác chỉnh sửa (sửa).

Bảng sau cho thấy các giá trị của $distance$ trong ví dụ:

	M O V I E					
	0	1	2	3	4	5
L	1	1	2	3	4	5
O	2	2	1	2	3	4
V	3	3	2	1	2	3
E	4	4	3	2	2	2

¹The distance is named after V. I. Levenshtein who studied it in connection with binary codes [49].

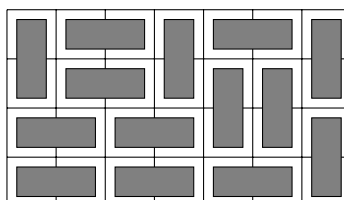
Góc dưới bên phải của bảng cho ta biết rằng khoảng cách chỉnh sửa giữa LOVE và MOVIE là 2. Bảng cũng cho thấy cách xây dựng dãy thao tác chỉnh sửa ngắn nhất. Trong trường hợp này, đường đi như sau:

		M	O	V	I	E	
		0	1	2	3	4	5
L		1	1	2	3	4	5
O		2	2	1	2	3	4
V		3	3	2	1	2	3
E		4	4	3	2	2	2

Các ký tự cuối của LOVE và MOVIE bằng nhau, nên khoảng cách chỉnh sửa giữa chúng bằng khoảng cách chỉnh sửa giữa LOV và MOVI. Ta có thể dùng một thao tác chỉnh sửa để xóa ký tự I khỏi MOVI. Do đó, khoảng cách chỉnh sửa lớn hơn một đơn vị so với khoảng cách chỉnh sửa giữa LOV và MOV, v.v.

7.6 Đếm lát phủ

Đôi khi các trạng thái của một lời giải quy hoạch động phức tạp hơn các tổ hợp cố định của các số. Như một ví dụ, xét bài toán tính số cách khác nhau để lát một lưới $n \times m$ bằng các viên gạch kích thước 1×2 và 2×1 . Ví dụ, một nghiệm hợp lệ cho lưới 4×7 là



và tổng số nghiệm là 781.

Bài toán có thể được giải bằng quy hoạch động bằng cách duyệt lưới từng hàng. Mỗi hàng trong một nghiệm có thể được biểu diễn như một chuỗi chứa m ký tự từ tập $\{\sqcup, \sqcup, \sqcup, \sqcup\}$. Ví dụ, nghiệm trên gồm bốn hàng tương ứng với các chuỗi sau:

- $\sqcup \sqcup \sqcup \sqcup \sqcup \sqcup$
- $\sqcup \sqcup \sqcup \sqcup \sqcup \sqcup$
- $\sqcup \sqcup \sqcup \sqcup \sqcup \sqcup$
- $\sqcup \sqcup \sqcup \sqcup \sqcup \sqcup$

Gọi $\text{count}(k, x)$ là số cách xây dựng một nghiệm cho các hàng $1 \dots k$ của lưới sao cho chuỗi x tương ứng với hàng k . Có thể dùng quy hoạch động ở đây, vì trạng thái của một hàng chỉ bị ràng buộc bởi trạng thái của hàng trước đó.

Một nghiệm hợp lệ nếu hàng 1 không chứa ký tự $\sqcup$, hàng n không chứa ký tự $\sqcup$, và mọi cặp hàng liên tiếp đều *tương thích*. Ví dụ, các hàng $\sqcup \sqcup \sqcup \sqcup \sqcup \sqcup$ và

$\square\square\square \sqcup \sqcup \sqcup$ là tương thích, trong khi các hàng $\square \square \square \square \square \square$ và $\square\square\square\square\square \sqcup$ không tương thích.

Vì một hàng gồm m ký tự và có bốn lựa chọn cho mỗi ký tự, số hàng khác nhau nhiều nhất là 4^m . Do đó, độ phức tạp thời gian của lời giải là $O(n4^{2m})$ vì ta có thể duyệt qua $O(4^m)$ trạng thái có thể cho mỗi hàng, và với mỗi trạng thái, có $O(4^m)$ trạng thái có thể cho hàng trước. Trong thực tế, nên xoay lưới sao cho cạnh ngắn hơn có độ dài m , vì thừa số 4^{2m} chi phối độ phức tạp thời gian.

Có thể làm lời giải hiệu quả hơn bằng cách dùng biểu diễn gọn hơn cho các hàng. Hóa ra chỉ cần biết những cột nào của hàng trước chứa ô trên của một viên gạch dọc. Do đó, ta có thể biểu diễn một hàng chỉ bằng các ký tự $\square$ và $\square$, trong đó $\square$ là một tổ hợp của các ký tự $\sqcup$, $\square$ và $\square$. Dùng biểu diễn này, chỉ có 2^m hàng khác nhau và độ phức tạp thời gian là $O(n2^{2m})$.

Như một ghi chú cuối, cũng có một công thức trực tiếp đáng ngạc nhiên để tính số lát phủ²:

$$\prod_{a=1}^{\lceil n/2 \rceil} \prod_{b=1}^{\lceil m/2 \rceil} 4 \cdot \left(\cos^2 \frac{\pi a}{n+1} + \cos^2 \frac{\pi b}{m+1} \right)$$

Công thức này rất hiệu quả, vì nó tính số lát phủ trong thời gian $O(nm)$, nhưng vì đáp án là tích của các số thực, một vấn đề khi dùng công thức là cách lưu các kết quả trung gian một cách chính xác.

²Surprisingly, this formula was discovered in 1961 by two research teams [43, 67] that worked independently.

Chương 8

Phân tích khấu hao

Độ phức tạp thời gian của một thuật toán thường dễ phân tích chỉ bằng cách xem xét cấu trúc của thuật toán: thuật toán chứa những vòng lặp nào và các vòng lặp được thực hiện bao nhiêu lần. Tuy nhiên, đôi khi một phân tích trực tiếp không cho bức tranh đúng về hiệu quả của thuật toán.

phân tích khấu hao (Amortized analysis) có thể được dùng để phân tích các thuật toán chứa những thao tác có độ phức tạp thời gian thay đổi. Ý tưởng là ước lượng tổng thời gian dùng cho tất cả các thao tác như vậy trong quá trình thực thi thuật toán, thay vì tập trung vào từng thao tác riêng lẻ.

8.1 Phương pháp hai con trỏ

Trong **phương pháp hai con trỏ (two pointers method)**, hai con trỏ được dùng để duyệt qua các giá trị của mảng. Cả hai con trỏ chỉ có thể di chuyển theo một hướng, điều này đảm bảo thuật toán hoạt động hiệu quả. Tiếp theo ta thảo luận hai bài toán có thể được giải bằng phương pháp hai con trỏ.

Tổng mảng con

Như ví dụ đầu tiên, xét một bài toán trong đó ta được cho một mảng gồm n số nguyên dương và một tổng mục tiêu x , và ta muốn tìm một mảng con có tổng là x hoặc báo rằng không tồn tại mảng con như vậy.

Ví dụ, mảng

1	3	2	5	1	1	2	3
---	---	---	---	---	---	---	---

chứa một mảng con có tổng là 8:

1	3	2	5	1	1	2	3
---	---	---	---	---	---	---	---

Bài toán này có thể được giải trong thời gian $O(n)$ bằng phương pháp hai con trỏ. Ý tưởng là duy trì các con trỏ tới giá trị đầu tiên và cuối cùng của một mảng con. Ở mỗi lượt, con trỏ trái di chuyển một bước sang phải, và con trỏ phải di chuyển sang

phải chừng nào tổng mảng con thu được không vượt quá x . Nếu tổng trở thành đúng bằng x , ta đã tìm được một nghiệm.

Như một ví dụ, xét mảng sau và tổng mục tiêu $x = 8$:

1	3	2	5	1	1	2	3
---	---	---	---	---	---	---	---

Mảng con ban đầu chứa các giá trị 1, 3 và 2 có tổng là 6:

1	3	2	5	1	1	2	3
---	---	---	---	---	---	---	---

Sau đó, con trỏ trái di chuyển một bước sang phải. Con trỏ phải không di chuyển, vì nếu không tổng mảng con sẽ vượt quá x .

1	3	2	5	1	1	2	3
---	---	---	---	---	---	---	---

Một lần nữa, con trỏ trái di chuyển một bước sang phải, và lần này con trỏ phải di chuyển ba bước sang phải. Tổng mảng con là $2 + 5 + 1 = 8$, nên đã tìm được một mảng con có tổng là x .

1	3	2	5	1	1	2	3
---	---	---	---	---	---	---	---

Thời gian chạy của thuật toán phụ thuộc vào số bước mà con trỏ phải di chuyển. Mặc dù không có cận trên hữu ích cho số bước con trỏ có thể di chuyển trong một *lượt đơn lẻ*, ta biết rằng con trỏ di chuyển *tổng cộng* $O(n)$ bước trong thuật toán, vì nó chỉ di chuyển sang phải.

Vì cả con trỏ trái và con trỏ phải đều di chuyển $O(n)$ bước trong thuật toán, thuật toán chạy trong thời gian $O(n)$.

Bài toán 2SUM

Một bài toán khác có thể được giải bằng phương pháp hai con trỏ là bài toán sau, còn được gọi là **bài toán 2SUM (2SUM problem)**: cho một mảng gồm n số và một tổng mục tiêu x , hãy tìm hai giá trị trong mảng sao cho tổng của chúng là x , hoặc báo rằng không tồn tại các giá trị như vậy.

Để giải bài toán, trước hết ta sắp xếp các giá trị của mảng theo thứ tự tăng dần. Sau đó, ta duyệt qua mảng bằng hai con trỏ. Con trỏ trái bắt đầu ở giá trị đầu tiên và di chuyển một bước sang phải ở mỗi lượt. Con trỏ phải bắt đầu ở giá trị cuối cùng và luôn di chuyển sang trái cho đến khi tổng của giá trị trái và phải không vượt quá x . Nếu tổng đúng bằng x , ta đã tìm được một nghiệm.

Ví dụ, xét mảng sau và tổng mục tiêu $x = 12$:

1	4	5	6	7	9	9	10
---	---	---	---	---	---	---	----

Các vị trí ban đầu của con trỏ như sau. Tổng các giá trị là $1 + 10 = 11$, nhỏ hơn x .

1	4	5	6	7	9	9	10
↑							↑

Sau đó con trỏ trái di chuyển một bước sang phải. Con trỏ phải di chuyển ba bước sang trái, và tổng trở thành $4 + 7 = 11$.

1	4	5	6	7	9	9	10
	↑			↑			

Sau đó, con trỏ trái lại di chuyển một bước sang phải. Con trỏ phải không di chuyển, và một nghiệm $5 + 7 = 12$ đã được tìm thấy.

1	4	5	6	7	9	9	10
		↑		↑			

Thời gian chạy của thuật toán là $O(n \log n)$, vì trước hết nó sắp xếp mảng trong thời gian $O(n \log n)$, rồi cả hai con trỏ di chuyển $O(n)$ bước.

Lưu ý rằng có thể giải bài toán theo một cách khác trong thời gian $O(n \log n)$ bằng tìm kiếm nhị phân. Trong lời giải như vậy, ta duyệt qua mảng và với mỗi giá trị mảng, ta cố tìm một giá trị khác tạo ra tổng x . Điều này có thể được thực hiện bằng cách làm n lần tìm kiếm nhị phân, mỗi lần mất thời gian $O(\log n)$.

Một bài toán khó hơn là **bài toán 3SUM (3SUM problem)**, yêu cầu tìm *ba* giá trị trong mảng có tổng là x . Dùng ý tưởng của thuật toán trên, bài toán này có thể được giải trong thời gian $O(n^2)$ ¹. Bạn có thấy cách làm không?

8.2 Phần tử nhỏ hơn gần nhất

Phân tích khâu hao thường được dùng để ước lượng số thao tác thực hiện trên một cấu trúc dữ liệu. Các thao tác có thể phân bố không đều, sao cho hầu hết thao tác xảy ra trong một pha nhất định của thuật toán, nhưng tổng số thao tác vẫn bị giới hạn.

Như một ví dụ, xét bài toán tìm cho mỗi phần tử mảng **phần tử nhỏ hơn gần nhất (nearest smaller element)**, tức là phần tử nhỏ hơn đầu tiên đứng trước phần tử đó trong mảng. Có thể không tồn tại phần tử như vậy, trong trường hợp đó thuật toán cần báo điều này. Tiếp theo ta sẽ xem cách giải bài toán hiệu quả bằng cấu trúc ngăn xếp.

Ta đi qua mảng từ trái sang phải và duy trì một ngăn xếp các phần tử mảng. Tại mỗi vị trí mảng, ta xóa các phần tử khỏi ngăn xếp cho đến khi phần tử trên đỉnh nhỏ hơn phần tử hiện tại, hoặc ngăn xếp rỗng. Sau đó, ta báo rằng phần tử trên đỉnh là phần tử nhỏ hơn gần nhất của phần tử hiện tại, hoặc nếu ngăn xếp rỗng, không có phần tử như vậy. Cuối cùng, ta thêm phần tử hiện tại vào ngăn xếp.

Như một ví dụ, xét mảng sau:

¹For a long time, it was thought that solving the 3SUM problem more efficiently than in $O(n^2)$ time would not be possible. However, in 2014, it turned out [30] that this is not the case.

1	3	4	2	5	3	4	2
---	---	---	---	---	---	---	---

Đầu tiên, các phần tử 1, 3 và 4 được thêm vào ngăn xếp, vì mỗi phần tử lớn hơn phần tử trước đó. Do đó, phần tử nhỏ hơn gần nhất của 4 là 3, và phần tử nhỏ hơn gần nhất của 3 là 1.

1	3	4	2	5	3	4	2
---	---	---	---	---	---	---	---

1 → 3 → 4

Phần tử tiếp theo 2 nhỏ hơn hai phần tử trên đỉnh ngăn xếp. Do đó, các phần tử 3 và 4 bị xóa khỏi ngăn xếp, rồi phần tử 2 được thêm vào ngăn xếp. Phần tử nhỏ hơn gần nhất của nó là 1:

1	3	4	2	5	3	4	2
---	---	---	---	---	---	---	---

1 → 2

Sau đó, phần tử 5 lớn hơn phần tử 2, nên nó sẽ được thêm vào ngăn xếp, và phần tử nhỏ hơn gần nhất của nó là 2:

1	3	4	2	5	3	4	2
---	---	---	---	---	---	---	---

1 → 2 → 5

Sau đó, phần tử 3 bị xóa khỏi ngăn xếp và các phần tử 3 và 4 được thêm vào ngăn xếp:

1	3	4	2	5	3	4	2
---	---	---	---	---	---	---	---

1 → 2 → 3 → 4

Cuối cùng, tất cả phần tử trừ 1 bị xóa khỏi ngăn xếp và phần tử cuối cùng 2 được thêm vào ngăn xếp:

1	3	4	2	5	3	4	2
---	---	---	---	---	---	---	---

1 → 2

Hiệu quả của thuật toán phụ thuộc vào tổng số thao tác trên ngăn xếp. Nếu phần tử hiện tại lớn hơn phần tử trên đỉnh ngăn xếp, nó được thêm trực tiếp vào ngăn xếp, điều này hiệu quả. Tuy nhiên, đôi khi ngăn xếp có thể chứa nhiều phần tử lớn hơn và cần thời gian để xóa chúng. Dù vậy, mỗi phần tử được thêm *chính xác một lần* vào ngăn xếp và bị xóa *nhiều nhất một lần* khỏi ngăn xếp. Do đó, mỗi phần tử gây ra $O(1)$ thao tác ngăn xếp, và thuật toán chạy trong thời gian $O(n)$.

8.3 Cực tiểu cửa sổ trượt

Một **cửa sổ trượt (sliding window)** là một mảng con có kích thước cố định di chuyển từ trái sang phải qua mảng. Tại mỗi vị trí cửa sổ, ta muốn tính một số thông tin về các phần tử bên trong cửa sổ. Trong mục này, ta tập trung vào bài toán duy trì **cực tiểu cửa sổ trượt (sliding window minimum)**, nghĩa là ta cần báo giá trị nhỏ nhất bên trong mỗi cửa sổ.

Cực tiểu cửa sổ trượt có thể được tính bằng một ý tưởng tương tự như ta đã dùng để tính các phần tử nhỏ hơn gần nhất. Ta duy trì một hàng đợi trong đó mỗi phần tử lớn hơn phần tử trước đó, và phần tử đầu tiên luôn tương ứng với phần tử nhỏ nhất bên trong cửa sổ. Sau mỗi lần cửa sổ di chuyển, ta xóa các phần tử khỏi cuối hàng đợi cho đến khi phần tử cuối hàng đợi nhỏ hơn phần tử mới của cửa sổ, hoặc hàng đợi trở nên rỗng. Ta cũng xóa phần tử đầu tiên của hàng đợi nếu nó không còn nằm trong cửa sổ. Cuối cùng, ta thêm phần tử mới của cửa sổ vào cuối hàng đợi.

Như một ví dụ, xét mảng sau:

2	1	4	5	3	4	1	2
---	---	---	---	---	---	---	---

Giả sử kích thước của cửa sổ trượt là 4. Ở vị trí cửa sổ đầu tiên, giá trị nhỏ nhất là 1:

2	1	4	5	3	4	1	2
---	---	---	---	---	---	---	---

1 → 4 → 5

Sau đó cửa sổ di chuyển một bước sang phải. Phần tử mới 3 nhỏ hơn các phần tử 4 và 5 trong hàng đợi, nên các phần tử 4 và 5 bị xóa khỏi hàng đợi và phần tử 3 được thêm vào hàng đợi. Giá trị nhỏ nhất vẫn là 1.

2	1	4	5	3	4	1	2
---	---	---	---	---	---	---	---

1 → 3

Sau đó, cửa sổ lại di chuyển, và phần tử nhỏ nhất 1 không còn thuộc cửa sổ nữa. Do đó, nó bị xóa khỏi hàng đợi và giá trị nhỏ nhất bây giờ là 3. Phần tử mới 4 cũng được thêm vào hàng đợi.

2	1	4	5	3	4	1	2
---	---	---	---	---	---	---	---

3 → 4

Phần tử mới tiếp theo 1 nhỏ hơn tất cả phần tử trong hàng đợi. Do đó, tất cả phần tử bị xóa khỏi hàng đợi và hàng đợi sẽ chỉ chứa phần tử 1:

2	1	4	5	3	4	1	2
---	---	---	---	---	---	---	---

1

Cuối cùng, cửa sổ đạt tới vị trí cuối cùng. Phần tử 2 được thêm vào hàng đợi, nhưng giá trị nhỏ nhất bên trong cửa sổ vẫn là 1.

2	1	4	5	3	4	1	2
---	---	---	---	---	---	---	---

1 → 2

Vì mỗi phần tử mảng được thêm vào hàng đợi đúng một lần và bị xóa khỏi hàng đợi nhiều nhất một lần, thuật toán chạy trong thời gian $O(n)$.

Chương 9

Truy vấn đoạn

Trong chương này, ta thảo luận các cấu trúc dữ liệu cho phép xử lý hiệu quả các truy vấn đoạn. Trong một **truy vấn đoạn (range query)**, nhiệm vụ của ta là tính một giá trị dựa trên một mảng con của một mảng. Các truy vấn đoạn điển hình là:

- $\text{sum}_q(a, b)$: tính tổng các giá trị trong đoạn $[a, b]$
- $\text{min}_q(a, b)$: tìm giá trị nhỏ nhất trong đoạn $[a, b]$
- $\text{max}_q(a, b)$: tìm giá trị lớn nhất trong đoạn $[a, b]$

Ví dụ, xét đoạn $[3, 6]$ trong mảng sau:

0	1	2	3	4	5	6	7
1	3	8	4	6	1	3	4

Trong trường hợp này, $\text{sum}_q(3, 6) = 14$, $\text{min}_q(3, 6) = 1$ và $\text{max}_q(3, 6) = 6$.

Một cách đơn giản để xử lý truy vấn đoạn là dùng một vòng lặp đi qua tất cả giá trị mảng trong đoạn. Ví dụ, hàm sau có thể được dùng để xử lý truy vấn tổng trên một mảng:

```
int sum(int a, int b) {  
    int s = 0;  
    for (int i = a; i <= b; i++) {  
        s += array[i];  
    }  
    return s;  
}
```

Hàm này chạy trong thời gian $O(n)$, trong đó n là kích thước của mảng. Do đó, ta có thể xử lý q truy vấn trong thời gian $O(nq)$ bằng hàm này. Tuy nhiên, nếu cả n và q đều lớn, cách tiếp cận này sẽ chậm. May mắn là có những cách xử lý truy vấn đoạn hiệu quả hơn nhiều.

9.1 Truy vấn trên mảng tĩnh

Trước hết ta tập trung vào một tình huống trong đó mảng là *tĩnh*, tức là các giá trị mảng không bao giờ được cập nhật giữa các truy vấn. Trong trường hợp này, chỉ cần xây dựng một cấu trúc dữ liệu tĩnh cho ta đáp án của mọi truy vấn có thể.

Truy vấn tổng

Ta có thể dễ dàng xử lý truy vấn tổng trên một mảng tĩnh bằng cách xây dựng một **mảng tổng tiền tố (prefix sum array)**. Mỗi giá trị trong mảng tổng tiền tố bằng tổng các giá trị trong mảng ban đầu đến vị trí đó, tức là giá trị tại vị trí k là $\text{sum}_q(0, k)$. Mảng tổng tiền tố có thể được xây dựng trong thời gian $O(n)$.

Ví dụ, xét mảng sau:

0	1	2	3	4	5	6	7
1	3	4	8	6	1	4	2

Mảng tổng tiền tố tương ứng như sau:

0	1	2	3	4	5	6	7
1	4	8	16	22	23	27	29

Vì mảng tổng tiền tố chứa tất cả giá trị của $\text{sum}_q(0, k)$, ta có thể tính bất kỳ giá trị nào của $\text{sum}_q(a, b)$ trong thời gian $O(1)$ như sau:

$$\text{sum}_q(a, b) = \text{sum}_q(0, b) - \text{sum}_q(0, a - 1)$$

Bằng cách định nghĩa $\text{sum}_q(0, -1) = 0$, công thức trên cũng đúng khi $a = 0$.

Ví dụ, xét đoạn $[3, 6]$:

0	1	2	3	4	5	6	7
1	3	4	8	6	1	4	2

Trong trường hợp này $\text{sum}_q(3, 6) = 8 + 6 + 1 + 4 = 19$. Tổng này có thể được tính từ hai giá trị của mảng tổng tiền tố:

0	1	2	3	4	5	6	7
1	4	8	16	22	23	27	29

Do đó, $\text{sum}_q(3, 6) = \text{sum}_q(0, 6) - \text{sum}_q(0, 2) = 27 - 8 = 19$.

Cũng có thể tổng quát hóa ý tưởng này lên nhiều chiều hơn. Ví dụ, ta có thể xây dựng một mảng tổng tiền tố hai chiều có thể được dùng để tính tổng của bất kỳ mảng con hình chữ nhật nào trong thời gian $O(1)$. Mỗi tổng trong một mảng như vậy tương ứng với một mảng con bắt đầu tại góc trên bên trái của mảng.

Hình sau minh họa ý tưởng:

		<i>D</i>				<i>C</i>			
		<i>B</i>				<i>A</i>			

Tổng của mảng con màu xám có thể được tính bằng công thức

$$S(A) - S(B) - S(C) + S(D),$$

trong đó $S(X)$ biểu thị tổng các giá trị trong một mảng con hình chữ nhật từ góc trên bên trái tới vị trí của X .

Truy vấn cực tiểu

Truy vấn cực tiểu khó xử lý hơn truy vấn tổng. Dù vậy, có một phương pháp tiên xử lý khá đơn giản trong thời gian $O(n \log n)$, sau đó ta có thể trả lời bất kỳ truy vấn cực tiểu nào trong thời gian $O(1)$ ¹. Lưu ý rằng vì truy vấn cực tiểu và cực đại có thể được xử lý tương tự, ta có thể tập trung vào truy vấn cực tiểu.

Ý tưởng là tính trước tất cả giá trị của $\min_q(a, b)$ where $b - a + 1$ (độ dài của đoạn) là một lũy thừa của hai. Ví dụ, với mảng

0	1	2	3	4	5	6	7
1	3	4	8	6	1	4	2

các giá trị sau được tính:

a	b	$\min_q(a, b)$	a	b	$\min_q(a, b)$	a	b	$\min_q(a, b)$
0	0	1	0	1	1	0	3	1
1	1	3	1	2	3	1	4	3
2	2	4	2	3	4	2	5	1
3	3	8	3	4	6	3	6	1
4	4	6	4	5	1	4	7	1
5	5	1	5	6	1	0	7	1
6	6	4	6	7	2			
7	7	2						

Số giá trị được tính trước là $O(n \log n)$, vì có $O(\log n)$ độ dài đoạn là lũy thừa của hai. Các giá trị có thể được tính hiệu quả bằng công thức đệ quy

$$\min_q(a, b) = \min(\min_q(a, a + w - 1), \min_q(a + w, b)),$$

¹This technique was introduced in [7] and sometimes called the **sparse table** method. There are also more sophisticated techniques [22] where the preprocessing time is only $O(n)$, but such algorithms are not needed in competitive programming.

trong đó $b - a + 1$ là một lũy thừa của hai và $w = (b - a + 1)/2$. Việc tính tất cả các giá trị đó mất thời gian $O(n \log n)$.

Sau đó, bất kỳ giá trị nào của $\min_q(a, b)$ cũng có thể được tính trong thời gian $O(1)$ như cực tiểu của hai giá trị đã tính trước. Gọi k là lũy thừa lớn nhất của hai không vượt quá $b - a + 1$. Ta có thể tính giá trị của $\min_q(a, b)$ bằng công thức

$$\min_q(a, b) = \min(\min_q(a, a + k - 1), \min_q(b - k + 1, b)).$$

Trong công thức trên, đoạn $[a, b]$ được biểu diễn như hợp của các đoạn $[a, a + k - 1]$ và $[b - k + 1, b]$, cả hai đều có độ dài k .

Như một ví dụ, xét đoạn $[1, 6]$:

0	1	2	3	4	5	6	7
1	3	4	8	6	1	4	2

Độ dài của đoạn là 6, và lũy thừa lớn nhất của hai không vượt quá 6 là 4. Do đó, đoạn $[1, 6]$ là hợp của các đoạn $[1, 4]$ và $[3, 6]$:

0	1	2	3	4	5	6	7
1	3	4	8	6	1	4	2

0	1	2	3	4	5	6	7
1	3	4	8	6	1	4	2

Vì $\min_q(1, 4) = 3$ và $\min_q(3, 6) = 1$, ta kết luận rằng $\min_q(1, 6) = 1$.

9.2 Cây chỉ số nhị phân

Một **cây chỉ số nhị phân (binary indexed tree)** hay **cây Fenwick (Fenwick tree)**² có thể được xem là một biến thể động của mảng tổng tiền tố. Nó hỗ trợ hai thao tác thời gian $O(\log n)$ trên một mảng: xử lý truy vấn tổng trên đoạn và cập nhật một giá trị.

Ưu điểm của cây chỉ số nhị phân là nó cho phép ta cập nhật hiệu quả các giá trị mảng giữa các truy vấn tổng. Điều này không thể làm bằng mảng tổng tiền tố, vì sau mỗi lần cập nhật, cần xây dựng lại toàn bộ mảng tổng tiền tố trong thời gian $O(n)$.

Cấu trúc

Mặc dù tên của cấu trúc là *cây chỉ số nhị phân*, nó thường được biểu diễn như một mảng. Trong mục này, ta giả sử tất cả các mảng được đánh chỉ số từ một, vì điều đó làm cài đặt dễ hơn.

Gọi $p(k)$ là lũy thừa lớn nhất của hai chia hết k . Ta lưu một cây chỉ số nhị phân dưới dạng một mảng *tree* sao cho

$$\text{tree}[k] = \text{sum}_q(k - p(k) + 1, k),$$

²The binary indexed tree structure was presented by P. M. Fenwick in 1994 [21].

tức là mỗi vị trí k chứa tổng các giá trị trong một đoạn của mảng ban đầu có độ dài $p(k)$ và kết thúc tại vị trí k . Ví dụ, vì $p(6) = 2$, $\text{tree}[6]$ chứa giá trị của $\text{sum}_q(5, 6)$.

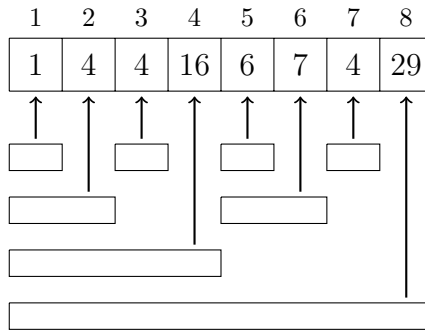
Ví dụ, xét mảng sau:

1	2	3	4	5	6	7	8
1	3	4	8	6	1	4	2

Cây chỉ số nhị phân tương ứng như sau:

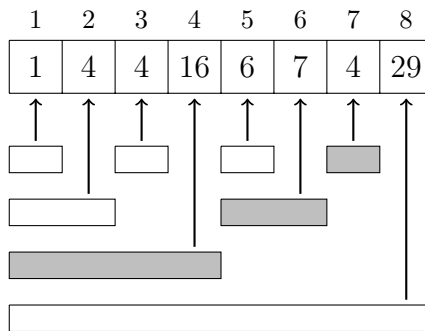
1	2	3	4	5	6	7	8
1	4	4	16	6	7	4	29

Hình sau cho thấy rõ hơn cách mỗi giá trị trong cây chỉ số nhị phân tương ứng với một đoạn trong mảng ban đầu:



Dùng cây chỉ số nhị phân, bất kỳ giá trị nào của $\text{sum}_q(1, k)$ cũng có thể được tính trong thời gian $O(\log n)$, vì một đoạn $[1, k]$ luôn có thể được chia thành $O(\log n)$ đoạn có tổng được lưu trong cây.

Ví dụ, đoạn $[1, 7]$ gồm các đoạn sau:



Do đó, ta có thể tính tổng tương ứng như sau:

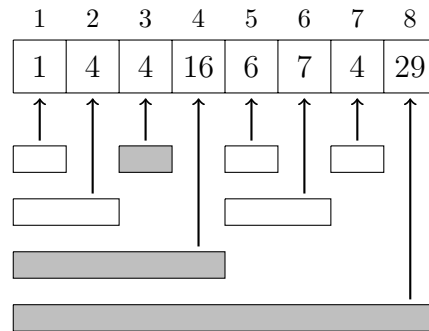
$$\text{sum}_q(1, 7) = \text{sum}_q(1, 4) + \text{sum}_q(5, 6) + \text{sum}_q(7, 7) = 16 + 7 + 4 = 27$$

Để tính giá trị của $\text{sum}_q(a, b)$ trong đó $a > 1$, ta có thể dùng cùng mẹo đã dùng với các mảng tổng tiền tố:

$$\text{sum}_q(a, b) = \text{sum}_q(1, b) - \text{sum}_q(1, a - 1).$$

Vì ta có thể tính cả $\text{sum}_q(1, b)$ và $\text{sum}_q(1, a - 1)$ trong thời gian $O(\log n)$, tổng độ phức tạp thời gian là $O(\log n)$.

Sau đó, sau khi cập nhật một giá trị trong mảng ban đầu, một số giá trị trong cây chỉ số nhị phân cần được cập nhật. Ví dụ, nếu giá trị tại vị trí 3 thay đổi, tổng của các đoạn sau thay đổi:



Vì mỗi phần tử mảng thuộc $O(\log n)$ đoạn trong cây chỉ số nhị phân, chỉ cần cập nhật $O(\log n)$ giá trị trong cây.

Cài đặt

Các thao tác của cây chỉ số nhị phân có thể được cài đặt hiệu quả bằng các thao tác bit. Điều then chốt cần dùng là ta có thể tính bất kỳ giá trị nào của $p(k)$ bằng công thức

$$p(k) = k \& -k.$$

Hàm sau tính giá trị của $\text{sum}_q(1, k)$:

```
int sum(int k) {
    int s = 0;
    while (k >= 1) {
        s += tree[k];
        k -= k&-k;
    }
    return s;
}
```

Hàm sau tăng giá trị mảng tại vị trí k thêm x (x có thể dương hoặc âm):

```
void add(int k, int x) {
    while (k <= n) {
        tree[k] += x;
        k += k&-k;
    }
}
```

Độ phức tạp thời gian của cả hai hàm là $O(\log n)$, vì các hàm truy cập $O(\log n)$ giá trị trong cây chỉ số nhị phân, và mỗi lần di chuyển tới vị trí tiếp theo mất thời gian $O(1)$.

9.3 Cây phân đoạn

Một **cây phân đoạn (segment tree)**³ là một cấu trúc dữ liệu hỗ trợ hai thao tác: xử lý một truy vấn đoạn và cập nhật một giá trị mảng. Cây phân đoạn có thể hỗ trợ truy vấn tổng, truy vấn cực tiểu và cực đại, cùng nhiều truy vấn khác sao cho cả hai thao tác đều chạy trong thời gian $O(\log n)$.

So với cây chỉ số nhị phân, ưu điểm của cây phân đoạn là nó là một cấu trúc dữ liệu tổng quát hơn. Trong khi cây chỉ số nhị phân chỉ hỗ trợ truy vấn tổng⁴, cây phân đoạn cũng hỗ trợ các truy vấn khác. Mặt khác, cây phân đoạn cần nhiều bộ nhớ hơn và khó cài đặt hơn một chút.

Cấu trúc

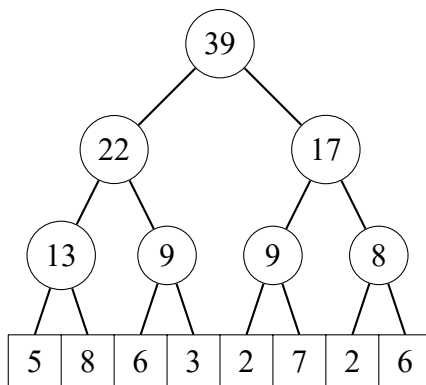
Cây phân đoạn là một cây nhị phân sao cho các đỉnh ở tầng dưới cùng của cây tương ứng với các phần tử mảng, và các đỉnh khác chứa thông tin cần thiết để xử lý truy vấn đoạn.

Trong mục này, ta giả sử kích thước của mảng là một lũy thừa của hai và dùng đánh chỉ số từ không, vì điều này thuận tiện để xây dựng cây phân đoạn cho một mảng như vậy. Nếu kích thước của mảng không phải lũy thừa của hai, ta luôn có thể thêm các phần tử phụ vào nó.

Trước hết ta sẽ thảo luận cây phân đoạn hỗ trợ truy vấn tổng. Như một ví dụ, xét mảng sau:

0	1	2	3	4	5	6	7
5	8	6	3	2	7	2	6

Cây phân đoạn tương ứng như sau:



Mỗi đỉnh trong của cây tương ứng với một đoạn mảng có kích thước là một lũy thừa của hai. Trong cây trên, giá trị của mỗi đỉnh trong là tổng các giá trị mảng tương ứng, và có thể được tính như tổng giá trị của hai đỉnh con trái và phải.

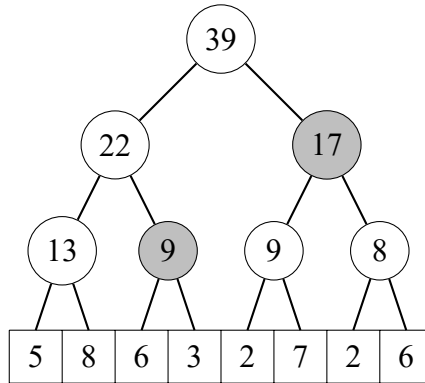
Hóa ra bất kỳ đoạn $[a, b]$ nào cũng có thể được chia thành $O(\log n)$ đoạn có giá trị được lưu trong các đỉnh của cây. Ví dụ, xét đoạn $[2, 7]$:

³The bottom-up-implementation in this chapter corresponds to that in [62]. Similar structures were used in late 1970's to solve geometric problems [9].

⁴In fact, using *two* binary indexed trees it is possible to support minimum queries [16], but this is more complicated than to use a segment tree.

0	1	2	3	4	5	6	7
5	8	6	3	2	7	2	6

Ở đây $\text{sum}_q(2, 7) = 6 + 3 + 2 + 7 + 2 + 6 = 26$. Trong trường hợp này, hai đỉnh cây sau tương ứng với đoạn:

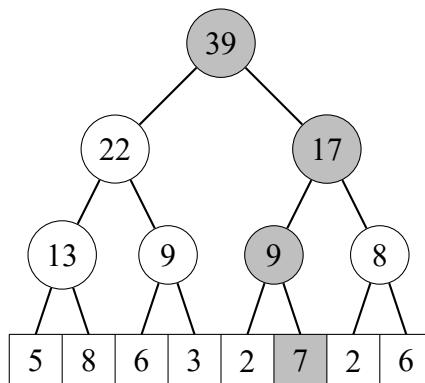


Do đó, một cách khác để tính tổng là $9 + 17 = 26$.

Khi tổng được tính bằng các đỉnh nằm càng cao càng tốt trong cây, cần nhiều nhất hai đỉnh ở mỗi tầng của cây. Vì vậy, tổng số đỉnh là $O(\log n)$.

Sau một lần cập nhật mảng, ta cần cập nhật tất cả các đỉnh có giá trị phụ thuộc vào giá trị đã cập nhật. Điều này có thể được thực hiện bằng cách duyệt đường đi từ phần tử mảng được cập nhật tới đỉnh gốc và cập nhật các đỉnh dọc theo đường đi.

Hình sau cho thấy những đỉnh cây nào thay đổi nếu giá trị mảng 7 thay đổi:

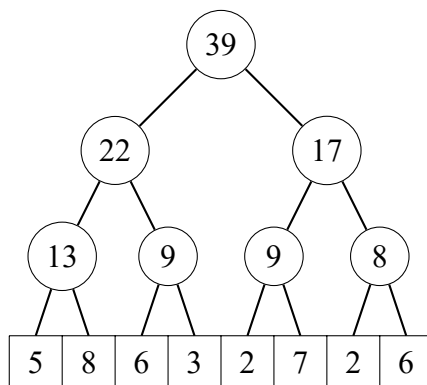


Đường đi từ dưới lên trên luôn gồm $O(\log n)$ đỉnh, nên mỗi lần cập nhật thay đổi $O(\log n)$ đỉnh trong cây.

Cài đặt

Ta lưu cây phân đoạn dưới dạng một mảng gồm $2n$ phần tử, trong đó n là kích thước của mảng ban đầu và là một lũy thừa của hai. Các đỉnh cây được lưu từ trên xuống dưới: $\text{tree}[1]$ là đỉnh gốc, $\text{tree}[2]$ và $\text{tree}[3]$ là các con của nó, v.v. Cuối cùng, các giá trị từ $\text{tree}[n]$ đến $\text{tree}[2n - 1]$ tương ứng với các giá trị của mảng ban đầu ở tầng dưới cùng của cây.

Ví dụ, cây phân đoạn



được lưu như sau:

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
39	22	17	13	9	9	8	5	8	6	3	2	7	2	6

Dùng biểu diễn này, cha của $tree[k]$ là $tree[\lfloor k/2 \rfloor]$, và các con của nó là $tree[2k]$ và $tree[2k + 1]$. Lưu ý rằng điều này suy ra vị trí của một đỉnh là chẵn nếu nó là con trái và lẻ nếu nó là con phải.

Hàm sau tính giá trị của $sum_q(a, b)$:

```

int sum(int a, int b) {
    a += n; b += n;
    int s = 0;
    while (a <= b) {
        if (a%2 == 1) s += tree[a++];
        if (b%2 == 0) s += tree[b--];
        a /= 2; b /= 2;
    }
    return s;
}

```

Hàm duy trì một đoạn ban đầu là $[a + n, b + n]$. Sau đó, ở mỗi bước, đoạn được chuyển lên một tầng cao hơn trong cây, và trước đó, giá trị của các đỉnh không thuộc đoạn cao hơn được cộng vào tổng.

Hàm sau tăng giá trị mảng tại vị trí k thêm x :

```

void add(int k, int x) {
    k += n;
    tree[k] += x;
    for (k /= 2; k >= 1; k /= 2) {
        tree[k] = tree[2*k] + tree[2*k+1];
    }
}

```

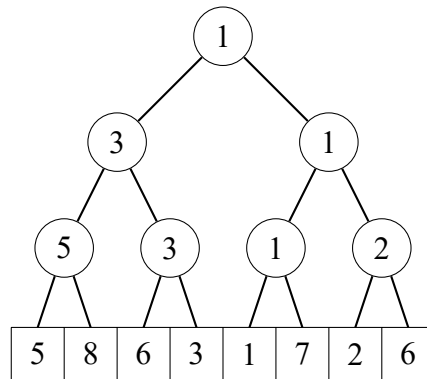
Trước hết hàm cập nhật giá trị ở tầng dưới cùng của cây. Sau đó, hàm cập nhật giá trị của tất cả các đỉnh trong của cây, cho đến khi đạt tới đỉnh gốc của cây.

Cả hai hàm trên đều chạy trong thời gian $O(\log n)$, vì một cây phân đoạn của n phần tử gồm $O(\log n)$ tầng, và các hàm di chuyển lên một tầng trong cây ở mỗi bước.

Các truy vấn khác

Cây phân đoạn có thể hỗ trợ mọi truy vấn đoạn trong đó có thể chia một đoạn thành hai phần, tính đáp án riêng cho từng phần rồi kết hợp các đáp án một cách hiệu quả. Ví dụ về các truy vấn như vậy là cực tiểu và cực đại, ước chung lớn nhất, và các thao tác bit and, or và xor.

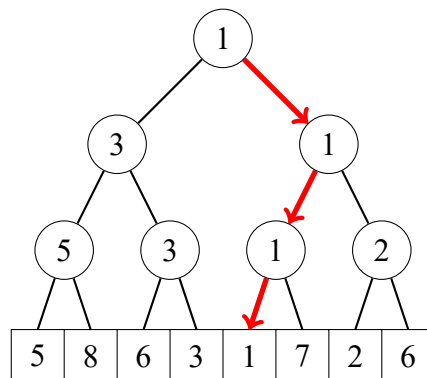
Ví dụ, cây phân đoạn sau hỗ trợ truy vấn cực tiểu:



Trong trường hợp này, mỗi đỉnh cây chứa giá trị nhỏ nhất trong đoạn mảng tương ứng. Đỉnh gốc của cây chứa giá trị nhỏ nhất trong toàn bộ mảng. Các thao tác có thể được cài đặt như trước, nhưng thay vì tổng, ta tính các giá trị cực tiểu.

Cấu trúc của cây phân đoạn cũng cho phép ta dùng tìm kiếm nhị phân để định vị các phần tử mảng. Ví dụ, nếu cây hỗ trợ truy vấn cực tiểu, ta có thể tìm vị trí của một phần tử có giá trị nhỏ nhất trong thời gian $O(\log n)$.

Ví dụ, trong cây trên, một phần tử có giá trị nhỏ nhất 1 có thể được tìm bằng cách duyệt một đường đi xuống từ đỉnh gốc:



9.4 Các kỹ thuật bổ sung

Nén chỉ số

Một hạn chế trong các cấu trúc dữ liệu được xây dựng trên mảng là các phần tử được đánh chỉ số bằng các số nguyên liên tiếp. Khó khăn xuất hiện khi cần các chỉ số lớn. Ví dụ, nếu ta muốn dùng chỉ số 10^9 , mảng phải chứa 10^9 phần tử, điều này sẽ cần quá nhiều bộ nhớ.

Tuy nhiên, ta thường có thể vượt qua hạn chế này bằng cách dùng **nén chỉ số (index compression)**, trong đó các chỉ số ban đầu được thay bằng các chỉ số 1, 2, 3, v.v. Điều này có thể làm được nếu ta biết trước tất cả chỉ số cần dùng trong thuật toán.

Ý tưởng là thay mỗi chỉ số ban đầu x bằng $c(x)$, trong đó c là một hàm nén các chỉ số. Ta yêu cầu thứ tự của các chỉ số không thay đổi, nên nếu $a < b$, thì $c(a) < c(b)$. Điều này cho phép ta thực hiện truy vấn thuận tiện ngay cả khi các chỉ số đã được nén.

Ví dụ, nếu các chỉ số ban đầu là 555, 10^9 và 8, các chỉ số mới là:

$$\begin{aligned}c(8) &= 1 \\c(555) &= 2 \\c(10^9) &= 3\end{aligned}$$

Cập nhật đoạn

Cho đến nay, ta đã cài đặt các cấu trúc dữ liệu hỗ trợ truy vấn đoạn và cập nhật các giá trị đơn lẻ. Bây giờ ta xét tình huống ngược lại, trong đó ta cần cập nhật các đoạn và lấy các giá trị đơn lẻ. Ta tập trung vào một thao tác tăng tất cả phần tử trong đoạn $[a, b]$ thêm x .

Điều đáng ngạc nhiên là ta cũng có thể dùng các cấu trúc dữ liệu được trình bày trong chương này trong tình huống này. Để làm điều đó, ta xây dựng một **mảng hiệu (difference array)** có các giá trị biểu thị hiệu giữa các giá trị liên tiếp trong mảng ban đầu. Do đó, mảng ban đầu là mảng tổng tiền tố của mảng hiệu. Ví dụ, xét mảng sau:

0	1	2	3	4	5	6	7
3	3	1	1	1	5	2	2

Mảng hiệu của mảng trên như sau:

0	1	2	3	4	5	6	7
3	0	-2	0	0	4	-3	0

Ví dụ, giá trị 2 tại vị trí 6 trong mảng ban đầu tương ứng với tổng $3 - 2 + 4 - 3 = 2$ trong mảng hiệu.

Ưu điểm của mảng hiệu là ta có thể cập nhật một đoạn trong mảng ban đầu bằng cách chỉ thay đổi hai phần tử trong mảng hiệu. Ví dụ, nếu ta muốn tăng các giá trị của mảng ban đầu từ vị trí 1 đến 4 thêm 5, chỉ cần tăng giá trị mảng hiệu tại vị trí 1 thêm 5 và giảm giá trị tại vị trí 5 đi 5. Kết quả như sau:

0	1	2	3	4	5	6	7
3	5	-2	0	0	-1	-3	0

Tổng quát hơn, để tăng các giá trị trong đoạn $[a, b]$ thêm x , ta tăng giá trị tại vị trí a thêm x và giảm giá trị tại vị trí $b + 1$ đi x . Do đó, chỉ cần cập nhật các giá trị đơn lẻ và xử lý truy vấn tổng, nên ta có thể dùng cây chỉ số nhị phân hoặc cây phân đoạn.

Một bài toán khó hơn là hỗ trợ cả truy vấn đoạn và cập nhật đoạn. Trong Chương 28, ta sẽ thấy rằng ngay cả điều này cũng khả thi.

Chương 10

Thao tác bit

Mọi dữ liệu trong chương trình máy tính đều được lưu bên trong dưới dạng bit, tức là các số 0 và 1. Chương này thảo luận biểu diễn bit của số nguyên, và trình bày các ví dụ về cách dùng các thao tác bit. Hóa ra thao tác bit có rất nhiều ứng dụng trong lập trình thuật toán.

10.1 Biểu diễn bit

Trong lập trình, một số nguyên n bit được lưu bên trong như một số nhị phân gồm n bit. Ví dụ, kiểu C++ `int` là một kiểu 32 bit, nghĩa là mỗi số `int` gồm 32 bit.

Đây là biểu diễn bit của số `int` 43:

0000000000000000000000000101011

Các bit trong biểu diễn được đánh chỉ số từ phải sang trái. Để chuyển biểu diễn bit $b_k \cdots b_2 b_1 b_0$ thành một số, ta có thể dùng công thức

$$b_k 2^k + \dots + b_2 2^2 + b_1 2^1 + b_0 2^0.$$

Ví dụ,

$$1 \cdot 2^5 + 1 \cdot 2^3 + 1 \cdot 2^1 + 1 \cdot 2^0 = 43.$$

Biểu diễn bit của một số có thể là **có dấu (signed)** hoặc **không dấu (unsigned)**. Thông thường biểu diễn có dấu được dùng, nghĩa là có thể biểu diễn cả số âm và số dương. Một biến có dấu gồm n bit có thể chứa bất kỳ số nguyên nào từ -2^{n-1} đến $2^{n-1} - 1$. Ví dụ, kiểu `int` trong C++ là một kiểu có dấu, nên một biến `int` có thể chứa bất kỳ số nguyên nào từ -2^{31} đến $2^{31} - 1$.

Bit đầu tiên trong biểu diễn có dấu là dấu của số (0 cho số không âm và 1 cho số âm), và $n - 1$ bit còn lại chứa độ lớn của số. **bù hai (Two's complement)** được dùng, nghĩa là số đối của một số được tính bằng cách trước hết đảo tất cả bit trong số đó, rồi tăng số thêm một.

Ví dụ, biểu diễn bit của số `int` -43 là

1111111111111111111111111010101.

Trong biểu diễn không dấu, chỉ có thể dùng các số không âm, nhưng cận trên của giá trị lớn hơn. Một biến không dấu gồm n bit có thể chứa bất kỳ số nguyên nào từ 0

đến $2^n - 1$. Ví dụ, trong C++, một biến unsigned int có thể chứa bất kỳ số nguyên nào từ 0 đến $2^{32} - 1$.

Có một mối liên hệ giữa các biểu diễn: một số có dấu $-x$ bằng một số không dấu $2^n - x$. Ví dụ, đoạn mã sau cho thấy số có dấu $x = -43$ bằng số không dấu $y = 2^{32} - 43$:

```
int x = -43;
unsigned int y = x;
cout << x << "\n"; // -43
cout << y << "\n"; // 4294967253
```

Nếu một số lớn hơn cận trên của biểu diễn bit, số đó sẽ bị tràn. Trong biểu diễn có dấu, số tiếp theo sau $2^{n-1} - 1$ là -2^{n-1} , và trong biểu diễn không dấu, số tiếp theo sau $2^n - 1$ là 0. Ví dụ, xét đoạn mã sau:

```
int x = 2147483647
cout << x << "\n"; // 2147483647
x++;
cout << x << "\n"; // -2147483648
```

Ban đầu, giá trị của x là $2^{31} - 1$. Đây là giá trị lớn nhất có thể được lưu trong một biến int, nên số tiếp theo sau $2^{31} - 1$ là -2^{31} .

10.2 Thao tác bit

Phép and

Phép **and** $x \& y$ tạo ra một số có bit một tại các vị trí mà cả x và y đều có bit một. Ví dụ, $22 \& 26 = 18$, vì

$$\begin{array}{r} 10110 \quad (22) \\ \& \quad 11010 \quad (26) \\ \hline = \quad 10010 \quad (18) \end{array}$$

Dùng phép and, ta có thể kiểm tra một số x có chẵn hay không vì $x \& 1 = 0$ nếu x chẵn, và $x \& 1 = 1$ nếu x lẻ. Tổng quát hơn, x chia hết cho 2^k khi và chỉ khi $x \& (2^k - 1) = 0$.

Phép or

Phép **or** $x \mid y$ tạo ra một số có bit một tại các vị trí mà ít nhất một trong x và y có bit một. Ví dụ, $22 \mid 26 = 30$, vì

$$\begin{array}{r} 10110 \quad (22) \\ \mid \quad 11010 \quad (26) \\ \hline = \quad 11110 \quad (30) \end{array}$$

Phép **xor** $x \wedge y$ tạo ra một số có bit một tại các vị trí mà đúng một trong x và y có bit một. Ví dụ, $22 \wedge 26 = 12$, vì

$$\begin{array}{rcl} & 10110 & (22) \\ \wedge & 11010 & (26) \\ \hline = & 01100 & (12) \end{array}$$

Phép **not** $\sim x$ tạo ra một số trong đó tất cả bit của x đã được đảo. Công thức $\sim x = -x - 1$ đúng, ví dụ, $\sim 29 = -30$.

$$\begin{array}{rcl} x & = & 29 \quad 000000000000000000000000000011101 \\ \sim x & = & -30 \quad 111111111111111111111111111100010 \end{array}$$

Phép dịch bit trái $x \ll k$ thêm k bit không vào số, và phép dịch bit phải $x \gg k$ xóa k bit cuối của số. Ví dụ, $14 \ll 2 = 56$, vì 14 và 56 tương ứng với 1110 và 111000. Tương tự, $49 \gg 3 = 6$, vì 49 và 6 tương ứng với 110001 và 110.

Ứng dụng

Một số có dạng $1 \ll k$ có một bit một ở vị trí k và tất cả bit khác là không, nên ta có thể dùng các số như vậy để truy cập từng bit của số. Cụ thể, bit thứ k của một số là một khi và chỉ khi $x \& (1 \ll k)$ khác không. Đoạn mã sau in biểu diễn bit của một số int x :

```
for (int i = 31; i >= 0; i--) {  
    if (x&(1<<i)) cout << "1";  
    else cout << "0";  
}
```

Ta cũng có thể sửa từng bit của số bằng các ý tưởng tương tự. Ví dụ, công thức $x \mid (1 \ll k)$ đặt bit thứ k của x thành một, công thức $x \& \sim(1 \ll k)$ đặt bit thứ k của x thành không, và công thức $x \wedge (1 \ll k)$ đảo bit thứ k của x .

Công thức $x \& (x - 1)$ đặt bit một cuối cùng của x thành không, và công thức $x \& -x$ đặt tất cả các bit một thành không, trừ bit một cuối cùng. Công thức $x | (x - 1)$ đảo tất cả các bit sau bit một cuối cùng. Cũng lưu ý rằng một số dương x là lũy thừa của hai khi và chỉ khi $x \& (x - 1) = 0$.

Các hàm bổ sung

Trình biên dịch g++ cung cấp các hàm sau để đếm bit:

- `__builtin_clz(x)`: số bit không ở đầu số
- `__builtin_ctz(x)`: số bit không ở cuối số
- `__builtin_popcount(x)`: số bit một trong số
- `__builtin_parity(x)`: tính chẵn lẻ của số bit một

Các hàm có thể được dùng như sau:

```
int x = 5328; // 000000000000000000001010011010000
cout << __builtin_clz(x) << "\n"; // 19
cout << __builtin_ctz(x) << "\n"; // 4
cout << __builtin_popcount(x) << "\n"; // 5
cout << __builtin_parity(x) << "\n"; // 1
```

Mặc dù các hàm trên chỉ hỗ trợ số int, cũng có các phiên bản long long của các hàm với hậu tố ll.

10.3 Biểu diễn tập hợp

Mọi tập con của một tập $\{0, 1, 2, \dots, n-1\}$ có thể được biểu diễn như một số nguyên n bit mà các bit một cho biết những phần tử nào thuộc tập con. Đây là một cách hiệu quả để biểu diễn tập hợp, vì mỗi phần tử chỉ cần một bit bộ nhớ, và các thao tác tập hợp có thể được cài đặt như thao tác bit.

Ví dụ, vì int là kiểu 32 bit, một số int có thể biểu diễn bất kỳ tập con nào của tập $\{0, 1, 2, \dots, 31\}$. Biểu diễn bit của tập $\{1, 3, 4, 8\}$ là

0000000000000000000000000000100011010,

tương ứng với số $2^8 + 2^4 + 2^3 + 2^1 = 282$.

Cài đặt tập hợp

Đoạn mã sau khai báo một biến int x có thể chứa một tập con của $\{0, 1, 2, \dots, 31\}$. Sau đó, mã thêm các phần tử 1, 3, 4 và 8 vào tập và in kích thước của tập.

```
int x = 0;
x |= (1<<1);
x |= (1<<3);
x |= (1<<4);
x |= (1<<8);
cout << __builtin_popcount(x) << "\n"; // 4
```

Sau đó, đoạn mã sau in tất cả các phần tử thuộc tập:


```
for (int i = 0; i < 32; i++) {
    if (x & (1 << i)) cout << i << " ";
}
// output: 1 3 4 8
```

Thao tác tập hợp

Các thao tác tập hợp có thể được cài đặt như các thao tác bit như sau:

	cú pháp tập hợp	cú pháp bit
giao	$a \cap b$	$a \& b$
hợp	$a \cup b$	$a b$
bù	$\bar{a}$	$\sim a$
hiệu	$a \setminus b$	$a \& (\sim b)$

Ví dụ, đoạn mã sau trước hết xây dựng các tập $x = \{1, 3, 4, 8\}$ và $y = \{3, 6, 8, 9\}$, rồi xây dựng tập $z = x \cup y = \{1, 3, 4, 6, 8, 9\}$:

```
int x = (1 << 1) | (1 << 3) | (1 << 4) | (1 << 8);
int y = (1 << 3) | (1 << 6) | (1 << 8) | (1 << 9);
int z = x | y;
cout << __builtin_popcount(z) << "\n"; // 6
```

Duyệt qua các tập con

Đoạn mã sau duyệt qua các tập con của $\{0, 1, \dots, n-1\}$:

```
for (int b = 0; b < (1 << n); b++) {
    // process subset b
}
```

Đoạn mã sau duyệt qua các tập con có đúng k phần tử:

```
for (int b = 0; b < (1 << n); b++) {
    if (__builtin_popcount(b) == k) {
        // process subset b
    }
}
```

Đoạn mã sau duyệt qua các tập con của một tập x :

```
int b = 0;
do {
    // process subset b
} while (b = (b-x) & x);
```

10.4 Tối ưu hóa bằng bit

Nhiều thuật toán có thể được tối ưu hóa bằng các thao tác bit. Những tối ưu hóa như vậy không thay đổi độ phức tạp thời gian của thuật toán, nhưng có thể ảnh hưởng lớn đến thời gian chạy thực tế của mã. Trong mục này, ta thảo luận các ví dụ về những tình huống như vậy.

Khoảng cách Hamming

khoảng cách Hamming (Hamming distance) $\text{hamming}(a, b)$ between two chuỗi a và b có cùng độ dài là số vị trí mà hai chuỗi khác nhau. Ví dụ,

$$\text{hamming}(01101, 11001) = 2.$$

Xét bài toán sau: Cho một danh sách gồm n chuỗi bit, mỗi chuỗi có độ dài k , tính khoảng cách Hamming nhỏ nhất giữa hai chuỗi trong danh sách. Ví dụ, đáp án cho $[00111, 01101, 11110]$ là 2, vì

- $\text{hamming}(00111, 01101) = 2$,
- $\text{hamming}(00111, 11110) = 3$, and
- $\text{hamming}(01101, 11110) = 3$.

Một cách trực tiếp để giải bài toán là duyệt qua mọi cặp chuỗi và tính khoảng cách Hamming của chúng, tạo ra thuật toán thời gian $O(n^2k)$. Hàm sau có thể được dùng để tính khoảng cách:

```
int hamming(string a, string b) {  
    int d = 0;  
    for (int i = 0; i < k; i++) {  
        if (a[i] != b[i]) d++;  
    }  
    return d;  
}
```

Tuy nhiên, nếu k nhỏ, ta có thể tối ưu mã bằng cách lưu các chuỗi bit dưới dạng số nguyên và tính khoảng cách Hamming bằng thao tác bit. Cụ thể, nếu $k \leq 32$, ta chỉ cần lưu các chuỗi dưới dạng giá trị int và dùng hàm sau để tính khoảng cách:

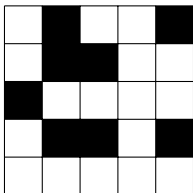
```
int hamming(int a, int b) {  
    return __builtin_popcount(a^b);  
}
```

Trong hàm trên, phép xor xây dựng một chuỗi bit có bit một tại các vị trí mà a và b khác nhau. Sau đó, số bit được tính bằng hàm `__builtin_popcount`.

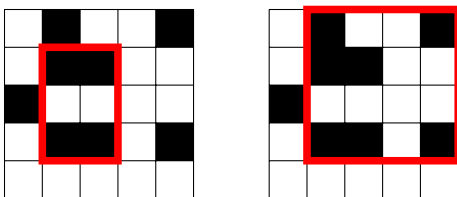
Để so sánh các cách cài đặt, ta sinh một danh sách 10000 chuỗi bit ngẫu nhiên có độ dài 30. Dùng cách tiếp cận đầu tiên, việc tìm kiếm mất 13.5 giây, và sau khi tối ưu bằng bit, nó chỉ mất 0.5 giây. Do đó, mã được tối ưu bằng bit nhanh hơn gần 30 lần so với mã ban đầu.

Đếm lưới con

Như một ví dụ khác, xét bài toán sau: Cho một lưới $n \times n$ trong đó mỗi ô là đen (1) hoặc trắng (0), tính số lưới con mà tất cả các góc đều màu đen. Ví dụ, lưới



chứa hai lưới con như vậy:



Có một thuật toán thời gian $O(n^3)$ để giải bài toán: duyệt qua tất cả $O(n^2)$ cặp hàng và với mỗi cặp (a, b) , tính số cột chứa một ô đen ở cả hai hàng trong thời gian $O(n)$. Đoạn mã sau giả sử `color[y][x]` biểu thị màu ở hàng y và cột x :

```
int count = 0;
for (int i = 0; i < n; i++) {
    if (color[a][i] == 1 && color[b][i] == 1) count++;
}
```

Khi đó, các cột đó tạo ra $\text{count}(\text{count} - 1)/2$ lưới con có góc đen, vì ta có thể chọn bất kỳ hai cột nào trong số chúng để tạo một lưới con.

Để tối ưu thuật toán này, ta chia lưới thành các khối cột sao cho mỗi khối gồm N cột liên tiếp. Sau đó, mỗi hàng được lưu như một danh sách các số N bit mô tả màu của các ô. Bây giờ ta có thể xử lý N cột cùng lúc bằng thao tác bit. Trong đoạn mã sau, `color[y][k]` biểu diễn một khối gồm N màu dưới dạng bit.

```
int count = 0;
for (int i = 0; i <= n/N; i++) {
    count += __builtin_popcount(color[a][i]&color[b][i]);
}
```

Thuật toán thu được chạy trong thời gian $O(n^3/N)$.

Ta sinh một lưới ngẫu nhiên kích thước 2500×2500 và so sánh cài đặt ban đầu với cài đặt tối ưu bằng bit. Trong khi mã ban đầu mất 29.6 giây, phiên bản tối ưu bằng bit chỉ mất 3.1 giây với $N = 32$ (các số int) và 1.7 giây với $N = 64$ (các số long long).

10.5 Quy hoạch động

Thao tác bit cung cấp một cách hiệu quả và thuận tiện để cài đặt các thuật toán quy hoạch động có trạng thái chứa các tập con của phần tử, vì các trạng thái như vậy có

thể được lưu dưới dạng số nguyên. Tiếp theo ta thảo luận các ví dụ kết hợp thao tác bit và quy hoạch động.

Lựa chọn tối ưu

Như ví dụ đầu tiên, xét bài toán sau: Ta được cho giá của k sản phẩm trong n ngày, và muốn mua mỗi sản phẩm chính xác một lần. Tuy nhiên, mỗi ngày ta chỉ được mua nhiều nhất một sản phẩm. Tổng giá nhỏ nhất là bao nhiêu? Ví dụ, xét kịch bản sau ($k = 3$ và $n = 8$):

	0	1	2	3	4	5	6	7
sản phẩm 0	6	9	5	2	8	9	1	6
sản phẩm 1	8	2	6	2	7	5	7	2
sản phẩm 2	5	3	9	7	3	5	1	4

Trong kịch bản này, tổng giá nhỏ nhất là 5:

	0	1	2	3	4	5	6	7
sản phẩm 0	6	9	5	2	8	9	1	6
sản phẩm 1	8	2	6	2	7	5	7	2
sản phẩm 2	5	3	9	7	3	5	1	4

Gọi $\text{price}[x][d]$ là giá của sản phẩm x vào ngày d . Ví dụ, trong kịch bản trên $\text{price}[2][3] = 7$. Sau đó, gọi $\text{total}(S, d)$ là tổng giá nhỏ nhất để mua một tập con S các sản phẩm trước ngày d . Dùng hàm này, nghiệm của bài toán là $\text{total}(\{0 \dots k - 1\}, n - 1)$.

Trước hết, $\text{total}(\emptyset, d) = 0$, vì không tốn gì để mua tập rỗng, và $\text{total}(\{x\}, 0) = \text{price}[x][0]$, vì có một cách để mua một sản phẩm vào ngày đầu tiên. Sau đó, có thể dùng công thức truy hồi sau:

$$\text{total}(S, d) = \min(\text{total}(S, d - 1), \min_{x \in S} (\text{total}(S \setminus x, d - 1) + \text{price}[x][d]))$$

Điều này có nghĩa là ta hoặc không mua sản phẩm nào vào ngày d , hoặc mua một sản phẩm x thuộc S . Trong trường hợp sau, ta xóa x khỏi S và cộng giá của x vào tổng giá.

Bước tiếp theo là tính các giá trị của hàm bằng quy hoạch động. Để lưu các giá trị của hàm, ta khai báo một mảng

```
int total[1<<K][N];
```

trong đó K và N là các hằng đủ lớn. Chiều đầu tiên của mảng tương ứng với biểu diễn bit của một tập con.

Trước hết, các trường hợp $d = 0$ có thể được xử lý như sau:

```
for (int x = 0; x < k; x++) {
    total[1<<x][0] = price[x][0];
}
```

Sau đó, công thức truy hồi chuyển thành đoạn mã sau:

```
for (int d = 1; d < n; d++) {
    for (int s = 0; s < (1<<k); s++) {
        total[s][d] = total[s][d-1];
        for (int x = 0; x < k; x++) {
            if (s & (1<<x)) {
                total[s][d] = min(total[s][d],
                                   total[s^(1<<x)][d-1] + price[x][d]);
            }
        }
    }
}
```

Độ phức tạp thời gian của thuật toán là $O(n2^k k)$.

Từ hoán vị đến tập con

Khi dùng quy hoạch động, ta thường có thể chuyển một phép duyệt trên các hoán vị thành một phép duyệt trên các tập con¹. Lợi ích của việc này là $n!$, số hoán vị, lớn hơn nhiều so với 2^n , số tập con. Ví dụ, nếu $n = 20$, thì $n! \approx 2.4 \cdot 10^{18}$ và $2^n \approx 10^6$. Do đó, với một số giá trị n , ta có thể duyệt qua các tập con hiệu quả nhưng không thể duyệt qua các hoán vị.

Như một ví dụ, xét bài toán sau: Có một thang máy với trọng lượng tối đa x , và n người có trọng lượng đã biết muốn đi từ tầng trệt lên tầng trên cùng. Cần ít nhất bao nhiêu chuyến nếu mọi người vào thang máy theo thứ tự tối ưu?

Ví dụ, giả sử $x = 10$, $n = 5$ và các trọng lượng như sau:

người	trọng lượng
0	2
1	3
2	3
3	5
4	6

Trong trường hợp này, số chuyến ít nhất là 2. Một thứ tự tối ưu là $\{0, 2, 3, 1, 4\}$, chia mọi người thành hai chuyến: đầu tiên $\{0, 2, 3\}$ (tổng trọng lượng 10), rồi $\{1, 4\}$ (tổng trọng lượng 9).

Bài toán có thể được giải dễ dàng trong thời gian $O(n!n)$ bằng cách thử mọi hoán vị có thể của n người. Tuy nhiên, ta có thể dùng quy hoạch động để thu được thuật toán hiệu quả hơn trong thời gian $O(2^n n)$. Ý tưởng là tính với mỗi tập con người hai

¹This technique was introduced in 1962 by M. Held and R. M. Karp [34].

giá trị: số chuyến ít nhất cần dùng và trọng lượng nhỏ nhất của những người đi trong nhóm cuối cùng.

Gọi $\text{weight}[p]$ là trọng lượng của người p . Ta định nghĩa hai hàm: $\text{rides}(S)$ là số chuyến nhỏ nhất cho một tập con S , và $\text{last}(S)$ là trọng lượng nhỏ nhất của chuyến cuối cùng. Ví dụ, trong kịch bản trên

$$\text{rides}(\{1, 3, 4\}) = 2 \quad \text{and} \quad \text{last}(\{1, 3, 4\}) = 5,$$

vì các chuyến tối ưu là $\{1, 4\}$ và $\{3\}$, và chuyến thứ hai có trọng lượng 5. Dĩ nhiên, mục tiêu cuối cùng của ta là tính giá trị của $\text{rides}(\{0 \dots n-1\})$.

Ta có thể tính các giá trị của các hàm một cách đệ quy rồi áp dụng quy hoạch động. Ý tưởng là duyệt qua tất cả người thuộc S và chọn tối ưu người cuối cùng p vào thang máy. Mỗi lựa chọn như vậy tạo ra một bài toán con cho một tập người nhỏ hơn. Nếu $\text{last}(S \setminus p) + \text{weight}[p] \leq x$, ta có thể thêm p vào chuyến cuối. Ngược lại, ta phải dành một chuyến mới ban đầu chỉ chứa p .

Để cài đặt quy hoạch động, ta khai báo một mảng

```
pair<int,int> best[1<<N];
```

chứa với mỗi tập con S một cặp $(\text{rides}(S), \text{last}(S))$. Ta đặt giá trị cho nhóm rỗng như sau:

```
best[0] = {1,0};
```

Sau đó, ta có thể điền mảng như sau:

```
for (int s = 1; s < (1<<n); s++) {
    // initial value: n+1 rides are needed
    best[s] = {n+1,0};
    for (int p = 0; p < n; p++) {
        if (s&(1<<p)) {
            auto option = best[s^(1<<p)];
            if (option.second+weight[p] <= x) {
                // add p to an existing ride
                option.second += weight[p];
            } else {
                // reserve a new ride for p
                option.first++;
                option.second = weight[p];
            }
            best[s] = min(best[s], option);
        }
    }
}
```

Lưu ý rằng vòng lặp trên đảm bảo rằng với bất kỳ hai tập con S_1 và S_2 sao cho $S_1 \subset S_2$, ta xử lý S_1 trước S_2 . Do đó, các giá trị quy hoạch động được tính theo đúng thứ tự.

Đếm tập con

Bài toán cuối cùng của chương này như sau: Cho $X = \{0 \dots n - 1\}$, và mỗi tập con $S \subset X$ được gán một số nguyên $\text{value}[S]$. Nhiệm vụ của ta là tính với mỗi S

$$\text{sum}(S) = \sum_{A \subset S} \text{value}[A],$$

tức là tổng giá trị của các tập con của S .

Ví dụ, giả sử $n = 3$ và các giá trị như sau:

- $\text{value}[\emptyset] = 3$
- $\text{value}[\{0\}] = 1$
- $\text{value}[\{1\}] = 4$
- $\text{value}[\{0, 1\}] = 5$
- $\text{value}[\{2\}] = 5$
- $\text{value}[\{0, 2\}] = 1$
- $\text{value}[\{1, 2\}] = 3$
- $\text{value}[\{0, 1, 2\}] = 3$

Trong trường hợp này, ví dụ,

$$\begin{aligned} \text{sum}(\{0, 2\}) &= \text{value}[\emptyset] + \text{value}[\{0\}] + \text{value}[\{2\}] + \text{value}[\{0, 2\}] \\ &= 3 + 1 + 5 + 1 = 10. \end{aligned}$$

Vì có tổng cộng 2^n tập con, một lời giải khả dĩ là duyệt qua mọi cặp tập con trong thời gian $O(2^{2n})$. Tuy nhiên, bằng quy hoạch động, ta có thể giải bài toán trong thời gian $O(2^n n)$. Ý tưởng là tập trung vào các tổng trong đó các phần tử có thể bị xóa khỏi S bị giới hạn.

Gọi $\text{partial}(S, k)$ là tổng giá trị của các tập con của S với ràng buộc rằng chỉ các phần tử $0 \dots k$ có thể bị xóa khỏi S . Ví dụ,

$$\text{partial}(\{0, 2\}, 1) = \text{value}[\{2\}] + \text{value}[\{0, 2\}],$$

vì ta chỉ có thể xóa các phần tử $0 \dots 1$. Ta có thể tính các giá trị của sum bằng các giá trị của partial , vì

$$\text{sum}(S) = \text{partial}(S, n - 1).$$

Các trường hợp cơ sở của hàm là

$$\text{partial}(S, -1) = \text{value}[S],$$

vì trong trường hợp này không phần tử nào có thể bị xóa khỏi S . Sau đó, trong trường hợp tổng quát, ta có thể dùng công thức truy hồi sau:

$$\text{partial}(S, k) = \begin{cases} \text{partial}(S, k - 1) & k \notin S \\ \text{partial}(S, k - 1) + \text{partial}(S \setminus \{k\}, k - 1) & k \in S \end{cases}$$

Ở đây ta tập trung vào phần tử k . Nếu $k \in S$, ta có hai lựa chọn: hoặc giữ k trong S hoặc xóa nó khỏi S .

Có một cách đặc biệt khéo léo để cài đặt việc tính các tổng. Ta có thể khai báo một mảng

```
int sum[1<<N];
```

sẽ chứa tổng của mỗi tập con. Mảng được khởi tạo như sau:

```
for (int s = 0; s < (1<<n); s++) {  
    sum[s] = value[s];  
}
```

Sau đó, ta có thể điền mảng như sau:

```
for (int k = 0; k < n; k++) {  
    for (int s = 0; s < (1<<n); s++) {  
        if (s & (1<<k)) sum[s] += sum[s^(1<<k)];  
    }  
}
```

Đoạn mã này tính các giá trị của $\text{partial}(S, k)$ với $k = 0 \dots n - 1$ vào mảng `sum`. Vì $\text{partial}(S, k)$ luôn dựa trên $\text{partial}(S, k - 1)$, ta có thể tái sử dụng mảng `sum`, tạo ra một cài đặt rất hiệu quả.

Phần II

Thuật toán đồ thị

Chương 11

Cơ bản về đồ thị

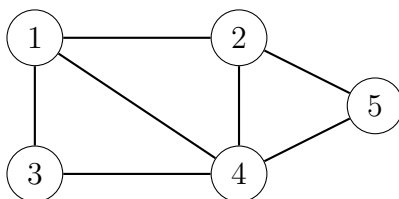
Nhiều bài toán lập trình có thể được giải bằng cách mô hình hóa bài toán thành một bài toán đồ thị và dùng một thuật toán đồ thị phù hợp. Một ví dụ điển hình về đồ thị là mạng lưới đường sá và thành phố trong một quốc gia. Tuy nhiên, đôi khi đồ thị bị ẩn trong bài toán và có thể khó nhận ra nó.

Phần này của sách thảo luận các thuật toán đồ thị, đặc biệt tập trung vào các chủ đề quan trọng trong lập trình thi đấu. Trong chương này, ta đi qua các khái niệm liên quan đến đồ thị, và nghiên cứu những cách khác nhau để biểu diễn đồ thị trong thuật toán.

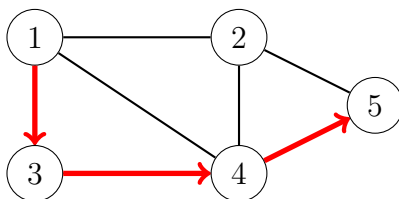
11.1 Thuật ngữ đồ thị

Một **đồ thị (graph)** gồm các **đỉnh (nodes)** và **cạnh (edges)**. Trong sách này, biến n biểu thị số đỉnh trong một đồ thị, và biến m biểu thị số cạnh. Các đỉnh được đánh số bằng các số nguyên $1, 2, \dots, n$.

Ví dụ, đồ thị sau gồm 5 đỉnh và 7 cạnh:



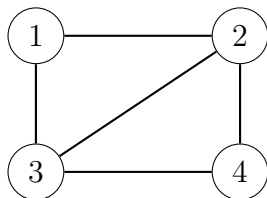
Một **đường đi (path)** đi từ đỉnh a đến đỉnh b qua các cạnh của đồ thị. **Độ dài (length)** của một đường đi là số cạnh trong đường đi đó. Ví dụ, đồ thị trên chứa một đường đi $1 \rightarrow 3 \rightarrow 4 \rightarrow 5$ có độ dài 3 từ đỉnh 1 đến đỉnh 5:



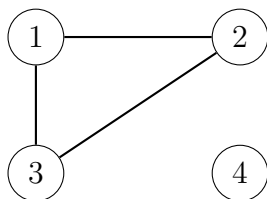
Một đường đi là một **chu trình (cycle)** nếu đỉnh đầu và đỉnh cuối là cùng một đỉnh. Ví dụ, đồ thị trên chứa chu trình $1 \rightarrow 3 \rightarrow 4 \rightarrow 1$. Một đường đi là **đơn (simple)** nếu mỗi đỉnh xuất hiện nhiều nhất một lần trong đường đi.

Tính liên thông

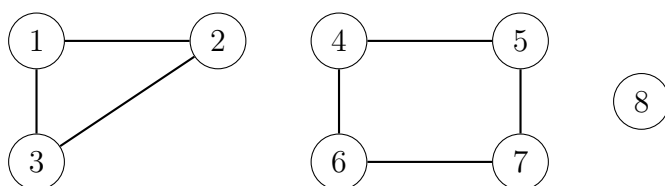
Một đồ thị là **liên thông (connected)** nếu có đường đi giữa bất kỳ hai đỉnh nào. Ví dụ, đồ thị sau là liên thông:



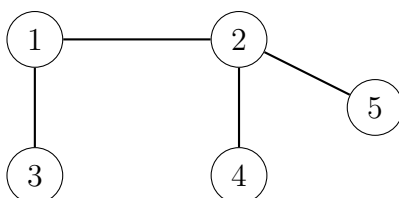
Đồ thị sau không liên thông, vì không thể đi từ đỉnh 4 đến bất kỳ đỉnh nào khác:



Các phần liên thông của một đồ thị được gọi là các **thành phần (components)** của nó. Ví dụ, đồ thị sau chứa ba thành phần: $\{1, 2, 3\}$, $\{4, 5, 6, 7\}$ and $\{8\}$.

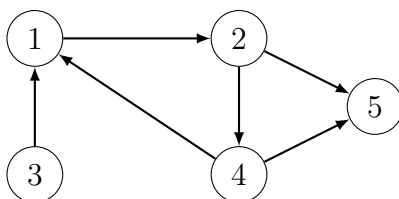


Một **cây (tree)** là một đồ thị liên thông gồm n đỉnh và $n - 1$ cạnh. Có đúng một đường đi giữa bất kỳ hai đỉnh nào của một cây. Ví dụ, đồ thị sau là một cây:



Hướng của cạnh

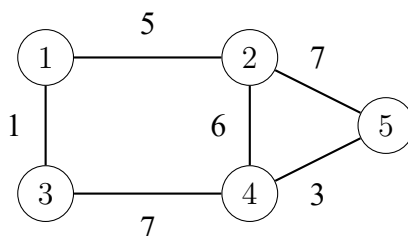
Một đồ thị là **có hướng (directed)** nếu các cạnh chỉ có thể được đi qua theo một hướng. Ví dụ, đồ thị sau là có hướng:



Đồ thị trên chứa một đường đi $3 \rightarrow 1 \rightarrow 2 \rightarrow 5$ từ đỉnh 3 đến đỉnh 5, nhưng không có đường đi từ đỉnh 5 đến đỉnh 3.

Trọng số cạnh

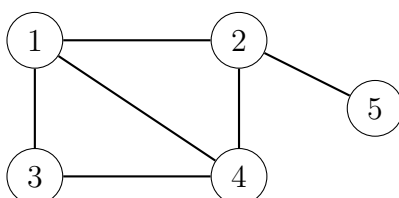
Trong một đồ thị **có trọng số (weighted)**, mỗi cạnh được gán một **trọng số (weight)**. Trọng số thường được diễn giải như độ dài cạnh. Ví dụ, đồ thị sau có trọng số:



Độ dài của một đường đi trong đồ thị có trọng số là tổng trọng số các cạnh trên đường đi. Ví dụ, trong đồ thị trên, độ dài của đường đi $1 \rightarrow 2 \rightarrow 5$ là 12, và độ dài của đường đi $1 \rightarrow 3 \rightarrow 4 \rightarrow 5$ là 11. Đường đi sau là đường đi **ngắn nhất (shortest)** từ đỉnh 1 đến đỉnh 5.

Đỉnh kề và bậc

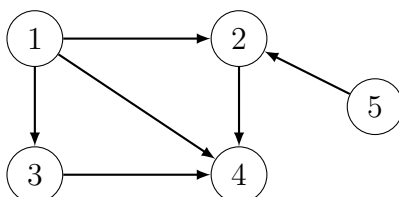
Hai đỉnh là **láng giềng (neighbors)** hoặc **kề nhau (adjacent)** nếu có một cạnh giữa chúng. **Bậc (degree)** của một đỉnh là số đỉnh kề với nó. Ví dụ, trong đồ thị sau, các đỉnh kề của đỉnh 2 là 1, 4 và 5, nên bậc của nó là 3.



Tổng bậc trong một đồ thị luôn là $2m$, trong đó m là số cạnh, vì mỗi cạnh làm tăng bậc của đúng hai đỉnh thêm một. Vì lý do này, tổng bậc luôn là số chẵn.

Một đồ thị là **đều (regular)** nếu bậc của mọi đỉnh là một hằng số d . Một đồ thị là **đầy đủ (complete)** nếu bậc của mọi đỉnh là $n - 1$, tức là đồ thị chứa mọi cạnh có thể giữa các đỉnh.

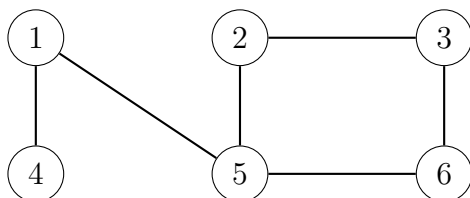
Trong đồ thị có hướng, **bậc vào (indegree)** của một đỉnh là số cạnh kết thúc tại đỉnh đó, và **bậc ra (outdegree)** của một đỉnh là số cạnh bắt đầu từ đỉnh đó. Ví dụ, trong đồ thị sau, bậc vào của đỉnh 2 là 2, và bậc ra của đỉnh 2 là 1.



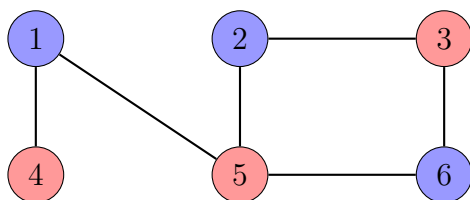
Tô màu

Trong một phép **tô màu (coloring)** đồ thị, mỗi đỉnh được gán một màu sao cho không có hai đỉnh kề nhau có cùng màu.

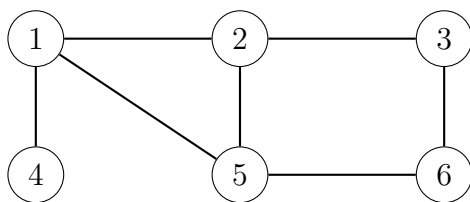
Một đồ thị là **hai phía (bipartite)** nếu có thể tô màu nó bằng hai màu. Hóa ra một đồ thị là hai phía khi và chỉ khi nó không chứa chu trình có số cạnh lẻ. Ví dụ, đồ thị



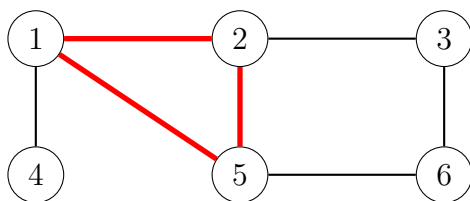
là hai phía, vì nó có thể được tô màu như sau:



Tuy nhiên, đồ thị

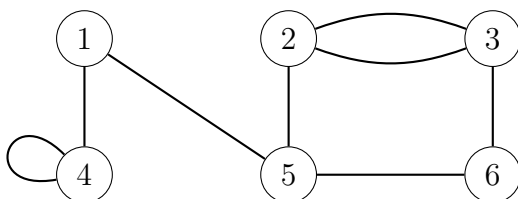


không phải hai phía, vì không thể tô màu chu trình ba đỉnh sau bằng hai màu:



Tính đơn

Một đồ thị là **đơn (simple)** nếu không có cạnh nào bắt đầu và kết thúc tại cùng một đỉnh, và không có nhiều cạnh giữa hai đỉnh. Thông thường ta giả sử các đồ thị là đơn. Ví dụ, đồ thị sau *không* đơn:



11.2 Biểu diễn đồ thị

Có nhiều cách để biểu diễn đồ thị trong thuật toán. Việc chọn cấu trúc dữ liệu phụ thuộc vào kích thước của đồ thị và cách thuật toán xử lý nó. Tiếp theo ta sẽ đi qua ba biểu diễn phổ biến.

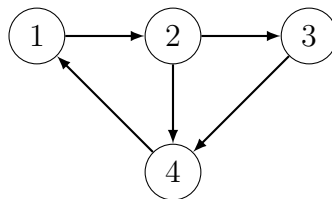
Biểu diễn danh sách kề

Trong biểu diễn danh sách kề, mỗi đỉnh x trong đồ thị được gán một **danh sách kề (adjacency list)** gồm các đỉnh mà có cạnh đi từ x tới đó. Danh sách kề là cách phổ biến nhất để biểu diễn đồ thị, và hầu hết thuật toán có thể được cài đặt hiệu quả bằng chúng.

Một cách thuận tiện để lưu các danh sách kề là khai báo một mảng các vector như sau:

```
vector<int> adj[N];
```

Hằng N được chọn sao cho tất cả danh sách kề có thể được lưu. Ví dụ, đồ thị



có thể được lưu như sau:

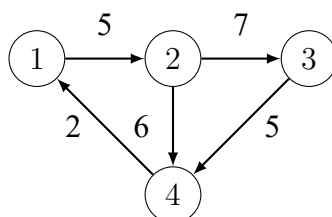
```
adj[1].push_back(2);  
adj[2].push_back(3);  
adj[2].push_back(4);  
adj[3].push_back(4);  
adj[4].push_back(1);
```

Nếu đồ thị vô hướng, nó có thể được lưu theo cách tương tự, nhưng mỗi cạnh được thêm theo cả hai hướng.

Với đồ thị có trọng số, cấu trúc có thể được mở rộng như sau:

```
vector<pair<int,int>> adj[N];
```

Trong trường hợp này, danh sách kề của đỉnh a chứa cặp (b, w) mỗi khi có một cạnh từ đỉnh a đến đỉnh b với trọng số w . Ví dụ, đồ thị



có thể được lưu như sau:

```
adj[1].push_back({2,5});  
adj[2].push_back({3,7});  
adj[2].push_back({4,6});  
adj[3].push_back({4,5});  
adj[4].push_back({1,2});
```

Lợi ích của việc dùng danh sách kề là ta có thể tìm hiệu quả các đỉnh mà ta có thể đi tới từ một đỉnh cho trước qua một cạnh. Ví dụ, vòng lặp sau duyệt qua tất cả đỉnh mà ta có thể đi tới từ đỉnh s :

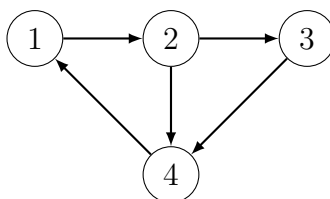
```
for (auto u : adj[s]) {  
    // process node u  
}
```

Biểu diễn ma trận kề

Một **ma trận kề (adjacency matrix)** là một mảng hai chiều cho biết đồ thị chứa những cạnh nào. Ta có thể kiểm tra hiệu quả từ ma trận kề xem có cạnh giữa hai đỉnh hay không. Ma trận có thể được lưu như một mảng

```
int adj[N][N];
```

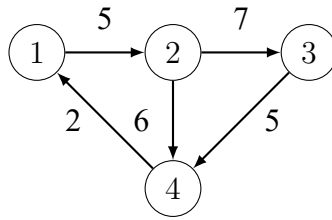
trong đó mỗi giá trị $adj[a][b]$ cho biết đồ thị có chứa cạnh từ đỉnh a đến đỉnh b hay không. Nếu cạnh nằm trong đồ thị, thì $adj[a][b] = 1$, và ngược lại $adj[a][b] = 0$. Ví dụ, đồ thị



có thể được biểu diễn như sau:

	1	2	3	4
1	0	1	0	0
2	0	0	1	1
3	0	0	0	1
4	1	0	0	0

Nếu đồ thị có trọng số, biểu diễn ma trận kề có thể được mở rộng sao cho ma trận chứa trọng số của cạnh nếu cạnh tồn tại. Dùng biểu diễn này, đồ thị



tương ứng với ma trận sau:

	1	2	3	4
1	0	5	0	0
2	0	0	7	6
3	0	0	0	5
4	2	0	0	0

Nhược điểm của biểu diễn ma trận kề là ma trận chứa n^2 phần tử, và thường hầu hết chúng là 0. Vì lý do này, biểu diễn này không thể được dùng nếu đồ thị lớn.

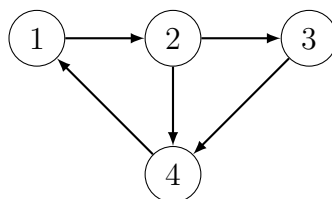
Biểu diễn danh sách cạnh

Một **danh sách cạnh (edge list)** chứa tất cả cạnh của một đồ thị theo một thứ tự nào đó. Đây là một cách thuận tiện để biểu diễn đồ thị nếu thuật toán xử lý tất cả cạnh của đồ thị và không cần tìm các cạnh bắt đầu tại một đỉnh cho trước.

Danh sách cạnh có thể được lưu trong một vector

```
vector<pair<int,int>> edges;
```

trong đó mỗi cặp (a, b) biểu thị rằng có một cạnh từ đỉnh a đến đỉnh b . Do đó, đồ thị



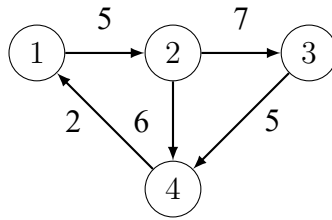
có thể được biểu diễn như sau:

```
edges.push_back({1,2});
edges.push_back({2,3});
edges.push_back({2,4});
edges.push_back({3,4});
edges.push_back({4,1});
```

Nếu đồ thị có trọng số, cấu trúc có thể được mở rộng như sau:

```
vector<tuple<int,int,int>> edges;
```

Mỗi phần tử trong danh sách này có dạng (a, b, w) , nghĩa là có một cạnh từ đỉnh a đến đỉnh b với trọng số w . Ví dụ, đồ thị



có thể được biểu diễn như sau¹:

```
edges.push_back({1,2,5});  
edges.push_back({2,3,7});  
edges.push_back({2,4,6});  
edges.push_back({3,4,5});  
edges.push_back({4,1,2});
```

¹In some older compilers, the function `make_tuple` must be used instead of the braces (for example, `make_tuple(1,2,5)` instead of `{1,2,5}`).

Chương 12

Duyệt đồ thị

Chương này thảo luận hai thuật toán đồ thị nền tảng: tìm kiếm theo chiều sâu và tìm kiếm theo chiều rộng. Cả hai thuật toán đều nhận một đỉnh bắt đầu trong đồ thị, và thăm tất cả các đỉnh có thể đi tới từ đỉnh bắt đầu. Điểm khác biệt giữa hai thuật toán là thứ tự mà chúng thăm các đỉnh.

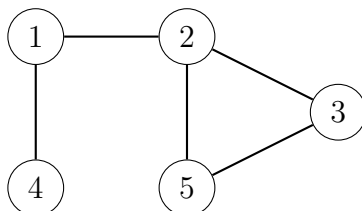
12.1 Tìm kiếm theo chiều sâu

tìm kiếm theo chiều sâu (Depth-first search, DFS) là một kỹ thuật duyệt đồ thị trực tiếp. Thuật toán bắt đầu tại một đỉnh xuất phát, và đi tới tất cả các đỉnh khác có thể đi được từ đỉnh xuất phát bằng các cạnh của đồ thị.

Tìm kiếm theo chiều sâu luôn đi theo một đường đi đơn trong đồ thị chừng nào còn tìm thấy đỉnh mới. Sau đó, nó quay lại các đỉnh trước đó và bắt đầu khám phá các phần khác của đồ thị. Thuật toán theo dõi các đỉnh đã thăm, để mỗi đỉnh chỉ được xử lý một lần.

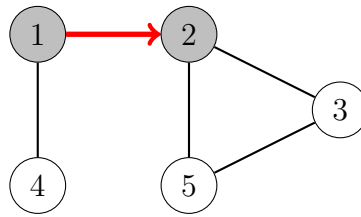
Ví dụ

Hãy xét cách tìm kiếm theo chiều sâu xử lý đồ thị sau:

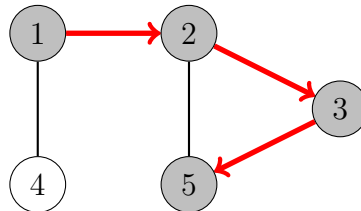


Ta có thể bắt đầu tìm kiếm tại bất kỳ đỉnh nào của đồ thị; bây giờ ta sẽ bắt đầu tìm kiếm tại đỉnh 1.

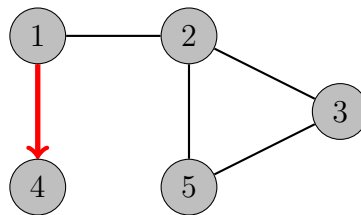
Trước hết tìm kiếm đi tới đỉnh 2:



Sau đó, các đỉnh 3 và 5 sẽ được thăm:



Các đỉnh kề của đỉnh 5 là 2 và 3, nhưng tìm kiếm đã thăm cả hai, nên đã đến lúc quay lại các đỉnh trước đó. Các đỉnh kề của các đỉnh 3 và 2 cũng đã được thăm, nên tiếp theo ta di chuyển từ đỉnh 1 đến đỉnh 4:



Sau đó, tìm kiếm kết thúc vì nó đã thăm tất cả các đỉnh.

Độ phức tạp thời gian của tìm kiếm theo chiều sâu là $O(n + m)$, trong đó n là số đỉnh và m là số cạnh, vì thuật toán xử lý mỗi đỉnh và mỗi cạnh một lần.

Cài đặt

Tìm kiếm theo chiều sâu có thể được cài đặt thuận tiện bằng đệ quy. Hàm dfs sau bắt đầu tìm kiếm theo chiều sâu tại một đỉnh cho trước. Hàm giả sử rằng đồ thị được lưu dưới dạng các danh sách kề trong một mảng

```
vector<int> adj[N];
```

và cũng duy trì một mảng

```
bool visited [N];
```

để theo dõi các đỉnh đã thăm. Ban đầu, mỗi giá trị mảng là false, và khi tìm kiếm đến đỉnh s , giá trị của `visited[s]` trở thành true. Hàm có thể được cài đặt như sau:

```
void dfs(int s) {
    if (visited[s]) return;
    visited[s] = true;
    // process node s
```

```

for (auto u: adj[s]) {
    dfs(u);
}

```

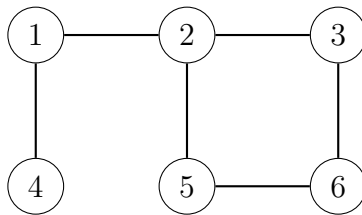
12.2 Tìm kiếm theo chiều rộng

tìm kiếm theo chiều rộng (Breadth-first search, BFS) thăm các đỉnh theo thứ tự tăng dần của khoảng cách từ đỉnh bắt đầu. Do đó, ta có thể tính khoảng cách từ đỉnh bắt đầu đến tất cả các đỉnh khác bằng tìm kiếm theo chiều rộng. Tuy nhiên, tìm kiếm theo chiều rộng khó cài đặt hơn tìm kiếm theo chiều sâu.

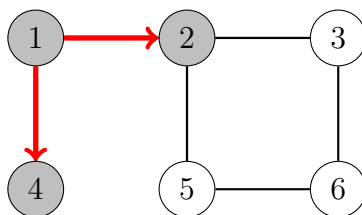
Tìm kiếm theo chiều rộng đi qua các đỉnh từng tầng một. Trước hết tìm kiếm khám phá các đỉnh có khoảng cách từ đỉnh bắt đầu là 1, sau đó các đỉnh có khoảng cách là 2, v.v. Quá trình này tiếp tục cho đến khi tất cả đỉnh đã được thăm.

Ví dụ

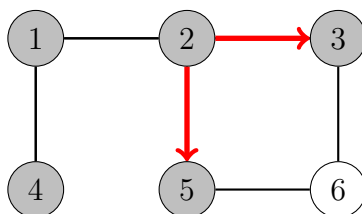
Hãy xét cách tìm kiếm theo chiều rộng xử lý đồ thị sau:



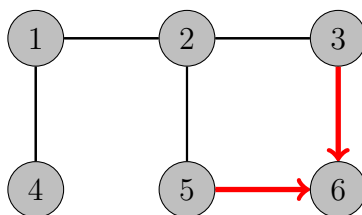
Giả sử tìm kiếm bắt đầu tại đỉnh 1. Trước hết, ta xử lý tất cả đỉnh có thể đi tới từ đỉnh 1 bằng một cạnh duy nhất:



Sau đó, ta đi tiếp tới các đỉnh 3 và 5:



Cuối cùng, ta thăm đỉnh 6:



Bây giờ ta đã tính được khoảng cách từ đỉnh bắt đầu tới tất cả đỉnh của đồ thị. Các khoảng cách như sau:

đỉnh	khoảng cách
1	0
2	1
3	2
4	1
5	2
6	3

Tương tự tìm kiếm theo chiều sâu, độ phức tạp thời gian của tìm kiếm theo chiều rộng là $O(n + m)$, trong đó n là số đỉnh và m là số cạnh.

Cài đặt

Tìm kiếm theo chiều rộng khó cài đặt hơn tìm kiếm theo chiều sâu, vì thuật toán thăm các đỉnh ở những phần khác nhau của đồ thị. Một cách cài đặt điển hình dựa trên một hàng đợi chứa các đỉnh. Ở mỗi bước, đỉnh tiếp theo trong hàng đợi sẽ được xử lý.

Đoạn mã sau giả sử đồ thị được lưu dưới dạng danh sách kề và duy trì các cấu trúc dữ liệu sau:

```
queue<int> q;
bool visited[N];
int distance[N];
```

Hàng đợi q chứa các đỉnh cần xử lý theo thứ tự tăng dần của khoảng cách. Các đỉnh mới luôn được thêm vào cuối hàng đợi, và đỉnh ở đầu hàng đợi là đỉnh tiếp theo cần xử lý. Mảng `visited` cho biết tìm kiếm đã thăm những đỉnh nào, và mảng `distance` sẽ chứa khoảng cách từ đỉnh bắt đầu đến tất cả đỉnh của đồ thị.

Tìm kiếm có thể được cài đặt như sau, bắt đầu tại đỉnh x :

```
visited[x] = true;
distance[x] = 0;
q.push(x);
while (!q.empty()) {
    int s = q.front(); q.pop();
    // process node s
    for (auto u : adj[s]) {
        if (visited[u]) continue;
        visited[u] = true;
```

```

        distance[u] = distance[s]+1;
        q.push(u);
    }
}

```

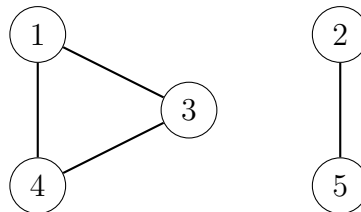
12.3 Ứng dụng

Dùng các thuật toán duyệt đồ thị, ta có thể kiểm tra nhiều tính chất của đồ thị. Thông thường, cả tìm kiếm theo chiều sâu và tìm kiếm theo chiều rộng đều có thể được dùng, nhưng trong thực tế, tìm kiếm theo chiều sâu là lựa chọn tốt hơn, vì nó dễ cài đặt hơn. Trong các ứng dụng sau, ta sẽ giả sử đồ thị là vô hướng.

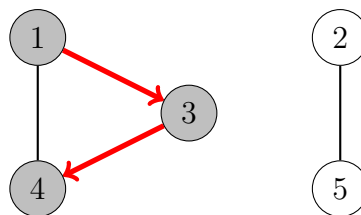
Kiểm tra tính liên thông

Một đồ thị là liên thông nếu có đường đi giữa bất kỳ hai đỉnh nào của đồ thị. Do đó, ta có thể kiểm tra một đồ thị có liên thông hay không bằng cách bắt đầu tại một đỉnh tùy ý và xem liệu ta có thể đi tới tất cả các đỉnh khác hay không.

Ví dụ, trong đồ thị



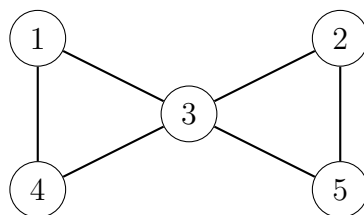
tìm kiếm theo chiều sâu từ đỉnh 1 thăm các đỉnh sau:



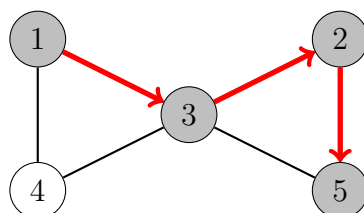
Vì tìm kiếm không thăm tất cả các đỉnh, ta có thể kết luận rằng đồ thị không liên thông. Theo cách tương tự, ta cũng có thể tìm tất cả thành phần liên thông của một đồ thị bằng cách duyệt qua các đỉnh và luôn bắt đầu một tìm kiếm theo chiều sâu mới nếu đỉnh hiện tại chưa thuộc thành phần nào.

Tìm chu trình

Một đồ thị chứa chu trình nếu trong quá trình duyệt đồ thị, ta tìm thấy một đỉnh có đỉnh kề (khác với đỉnh trước đó trên đường đi hiện tại) đã được thăm. Ví dụ, đồ thị



chứa hai chu trình và ta có thể tìm một trong số chúng như sau:



Sau khi di chuyển từ đỉnh 2 đến đỉnh 5, ta nhận thấy rằng đỉnh kề 3 của đỉnh 5 đã được thăm. Do đó, đồ thị chứa một chu trình đi qua đỉnh 3, ví dụ, $3 \rightarrow 2 \rightarrow 5 \rightarrow 3$.

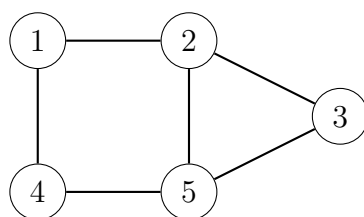
Một cách khác để biết đồ thị có chứa chu trình hay không là chỉ cần tính số đỉnh và số cạnh trong mỗi thành phần. Nếu một thành phần chứa c đỉnh và không có chu trình, nó phải chứa đúng $c - 1$ cạnh (nên nó phải là một cây). Nếu có c cạnh hoặc nhiều hơn, thành phần đó chắc chắn chứa một chu trình.

Kiểm tra tính hai phía

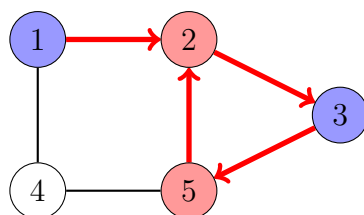
Một đồ thị là hai phía nếu các đỉnh của nó có thể được tô bằng hai màu sao cho không có hai đỉnh kề nhau có cùng màu. Việc kiểm tra một đồ thị có hai phía hay không bằng các thuật toán duyệt đồ thị đơn giản một cách đáng ngạc nhiên.

Ý tưởng là tô đỉnh bắt đầu màu xanh, tất cả đỉnh kề của nó màu đỏ, tất cả đỉnh kề của chúng màu xanh, v.v. Nếu tại một thời điểm nào đó của tìm kiếm, ta nhận thấy rằng hai đỉnh kề nhau có cùng màu, điều này có nghĩa là đồ thị không hai phía. Ngược lại, đồ thị là hai phía và ta đã tìm được một cách tô màu.

Ví dụ, đồ thị



không hai phía, vì một tìm kiếm từ đỉnh 1 diễn ra như sau:



Ta nhận thấy rằng màu của cả hai đỉnh 2 và 5 đều là đỏ, trong khi chúng là các đỉnh kề nhau trong đồ thị. Do đó, đồ thị không hai phía.

Thuật toán này luôn đúng, vì khi chỉ có hai màu khả dụng, màu của đỉnh bắt đầu trong một thành phần xác định màu của tất cả đỉnh khác trong thành phần đó. Việc đỉnh bắt đầu là đỏ hay xanh không tạo ra khác biệt nào.

Lưu ý rằng trong trường hợp tổng quát, việc xác định liệu các đỉnh trong một đồ thị có thể được tô bằng k màu sao cho không có hai đỉnh kề nhau cùng màu là khó. Ngay cả khi $k = 3$, hiện chưa biết thuật toán hiệu quả nào và bài toán là NP-hard.

Chương 13

Đường đi ngắn nhất

Tìm một đường đi ngắn nhất giữa hai đỉnh của một đồ thị là một bài toán quan trọng có nhiều ứng dụng thực tế. Ví dụ, một bài toán tự nhiên liên quan đến mạng lưới đường bộ là tính độ dài ngắn nhất có thể của một lộ trình giữa hai thành phố, khi biết độ dài các con đường.

Trong đồ thị không trọng số, độ dài của một đường đi bằng số cạnh của nó, và ta có thể chỉ cần dùng tìm kiếm theo chiều rộng để tìm một đường đi ngắn nhất. Tuy nhiên, trong chương này, ta tập trung vào đồ thị có trọng số, nơi cần các thuật toán tinh vi hơn để tìm đường đi ngắn nhất.

13.1 Thuật toán Bellman–Ford

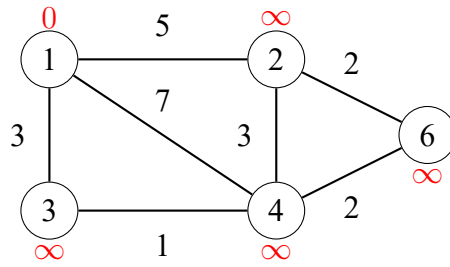
thuật toán Bellman–Ford (Bellman–Ford algorithm)¹ tìm đường đi ngắn nhất từ một đỉnh bắt đầu đến tất cả các đỉnh của đồ thị. Thuật toán có thể xử lý mọi loại đồ thị, miễn là đồ thị không chứa chu trình có độ dài âm. Nếu đồ thị chứa chu trình âm, thuật toán có thể phát hiện điều này.

Thuật toán theo dõi khoảng cách từ đỉnh bắt đầu đến tất cả các đỉnh của đồ thị. Ban đầu, khoảng cách đến đỉnh bắt đầu là 0 và khoảng cách đến tất cả đỉnh khác là vô cùng. Thuật toán giảm các khoảng cách bằng cách tìm các cạnh làm ngắn đường đi cho đến khi không còn có thể giảm bất kỳ khoảng cách nào.

Ví dụ

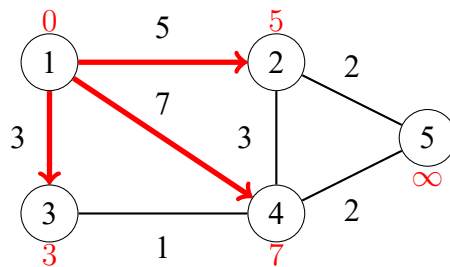
Hãy xét cách thuật toán Bellman–Ford hoạt động trong đồ thị sau:

¹Thuật toán được đặt theo tên R. E. Bellman và L. R. Ford, những người đã công bố nó một cách độc lập lần lượt vào các năm 1958 và 1956 [5, 24].

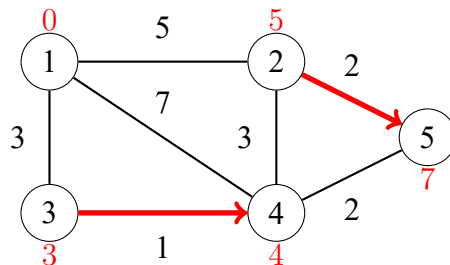


Mỗi đỉnh của đồ thị được gán một khoảng cách. Ban đầu, khoảng cách đến đỉnh bắt đầu là 0, và khoảng cách đến tất cả đỉnh khác là vô cùng.

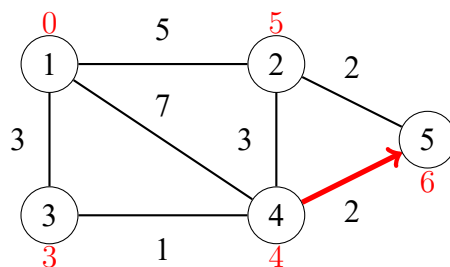
Thuật toán tìm các cạnh làm giảm khoảng cách. Đầu tiên, tất cả cạnh từ đỉnh 1 đều làm giảm khoảng cách:



Sau đó, các cạnh $2 \rightarrow 5$ và $3 \rightarrow 4$ làm giảm khoảng cách:

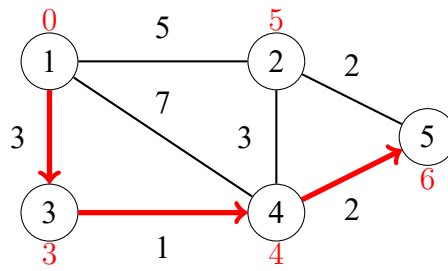


Cuối cùng, còn một thay đổi nữa:



Sau đó, không cạnh nào có thể làm giảm khoảng cách. Điều này có nghĩa là các khoảng cách đã là cuối cùng, và ta đã tính thành công các khoảng cách ngắn nhất từ đỉnh bắt đầu đến tất cả đỉnh của đồ thị.

Ví dụ, khoảng cách ngắn nhất 3 từ đỉnh 1 đến đỉnh 5 tương ứng với đường đi sau:



Cài đặt

Cài đặt sau của thuật toán Bellman–Ford xác định các khoảng cách ngắn nhất từ một đỉnh x đến tất cả đỉnh của đồ thị. Mã giả sử rằng đồ thị được lưu dưới dạng danh sách cạnh edges gồm các bộ dạng (a, b, w) , nghĩa là có một cạnh từ đỉnh a đến đỉnh b với trọng số w .

Thuật toán gồm $n - 1$ vòng, và trong mỗi vòng, thuật toán đi qua tất cả cạnh của đồ thị và cố giảm các khoảng cách. Thuật toán xây dựng một mảng distance sẽ chứa các khoảng cách từ x đến tất cả đỉnh của đồ thị. Hằng INF biểu thị khoảng cách vô cùng.

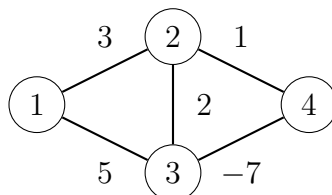
```
for (int i = 1; i <= n; i++) distance[i] = INF;
distance[x] = 0;
for (int i = 1; i <= n-1; i++) {
    for (auto e : edges) {
        int a, b, w;
        tie(a, b, w) = e;
        distance[b] = min(distance[b], distance[a]+w);
    }
}
```

Độ phức tạp thời gian của thuật toán là $O(nm)$, vì thuật toán gồm $n - 1$ vòng và duyệt qua tất cả m cạnh trong mỗi vòng. Nếu không có chu trình âm trong đồ thị, tất cả khoảng cách là cuối cùng sau $n - 1$ vòng, vì mỗi đường đi ngắn nhất có thể chứa nhiều nhất $n - 1$ cạnh.

Trong thực tế, các khoảng cách cuối cùng thường có thể được tìm nhanh hơn $n - 1$ vòng. Do đó, một cách có thể làm thuật toán hiệu quả hơn là dừng thuật toán nếu không có khoảng cách nào được giảm trong một vòng.

Chu trình âm

Thuật toán Bellman–Ford cũng có thể được dùng để kiểm tra đồ thị có chứa chu trình độ dài âm hay không. Ví dụ, đồ thị



chứa một chu trình âm $2 \rightarrow 3 \rightarrow 4 \rightarrow 2$ có độ dài -4 .

Nếu đồ thị chứa một chu trình âm, ta có thể làm ngắn vô hạn lần bất kỳ đường đi nào chứa chu trình đó bằng cách lặp lại chu trình nhiều lần. Do đó, khái niệm đường đi ngắn nhất không còn ý nghĩa trong tình huống này.

Một chu trình âm có thể được phát hiện bằng thuật toán Bellman–Ford bằng cách chạy thuật toán trong n vòng. Nếu vòng cuối cùng làm giảm bất kỳ khoảng cách nào, đồ thị chứa một chu trình âm. Lưu ý rằng thuật toán này có thể được dùng để tìm kiếm một chu trình âm trong toàn bộ đồ thị bất kể đỉnh bắt đầu.

Thuật toán SPFA

thuật toán SPFA (SPFA algorithm) ("Shortest Path Faster Algorithm") [20] là một biến thể của thuật toán Bellman–Ford, thường hiệu quả hơn thuật toán gốc. Thuật toán SPFA không đi qua tất cả cạnh ở mỗi vòng, mà thay vào đó chọn các cạnh cần xét theo cách thông minh hơn.

Thuật toán duy trì một hàng đợi các đỉnh có thể được dùng để giảm khoảng cách. Trước hết, thuật toán thêm đỉnh bắt đầu x vào hàng đợi. Sau đó, thuật toán luôn xử lý đỉnh đầu tiên trong hàng đợi, và khi một cạnh $a \rightarrow b$ làm giảm một khoảng cách, đỉnh b được thêm vào hàng đợi.

Hiệu quả của thuật toán SPFA phụ thuộc vào cấu trúc của đồ thị: thuật toán thường hiệu quả, nhưng độ phức tạp thời gian trường hợp xấu nhất vẫn là $O(nm)$ và có thể tạo các đầu vào làm thuật toán chậm như thuật toán Bellman–Ford gốc.

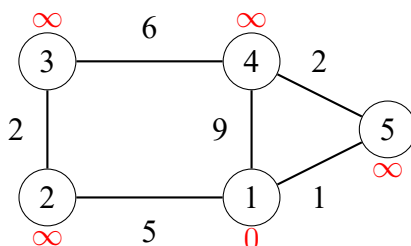
13.2 Thuật toán Dijkstra

thuật toán Dijkstra (Dijkstra's algorithm)² tìm đường đi ngắn nhất từ đỉnh bắt đầu đến tất cả đỉnh của đồ thị, giống thuật toán Bellman–Ford. Lợi ích của thuật toán Dijkstra là nó hiệu quả hơn và có thể được dùng để xử lý đồ thị lớn. Tuy nhiên, thuật toán yêu cầu không có cạnh trọng số âm trong đồ thị.

Giống thuật toán Bellman–Ford, thuật toán Dijkstra duy trì khoảng cách đến các đỉnh và giảm chúng trong quá trình tìm kiếm. Thuật toán Dijkstra hiệu quả vì nó chỉ xử lý mỗi cạnh trong đồ thị một lần, dựa trên thực tế rằng không có cạnh âm.

Ví dụ

Hãy xét cách thuật toán Dijkstra hoạt động trong đồ thị sau khi đỉnh bắt đầu là đỉnh 1:

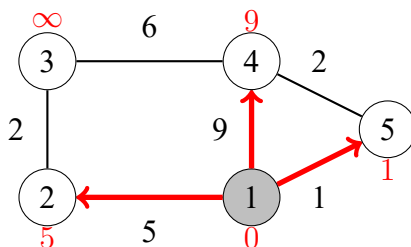


²E. W. Dijkstra công bố thuật toán vào năm 1959 [14]; tuy nhiên, bài báo gốc của ông không đề cập cách cài đặt thuật toán một cách hiệu quả.

Giống như trong thuật toán Bellman–Ford, ban đầu khoảng cách đến đỉnh bắt đầu là 0 và khoảng cách đến tất cả đỉnh khác là vô cùng.

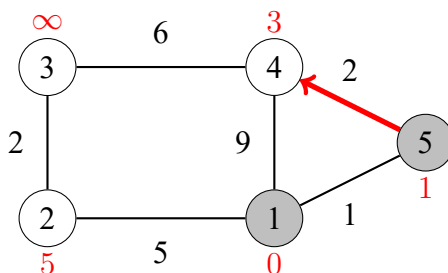
Ở mỗi bước, thuật toán Dijkstra chọn một đỉnh chưa được xử lý và có khoảng cách nhỏ nhất có thể. Đỉnh đầu tiên như vậy là đỉnh 1 với khoảng cách 0.

Khi một đỉnh được chọn, thuật toán đi qua tất cả cạnh bắt đầu tại đỉnh đó và dùng chúng để giảm các khoảng cách:

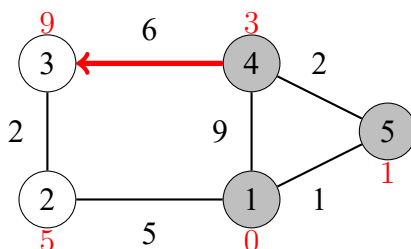


Trong trường hợp này, các cạnh từ đỉnh 1 đã giảm khoảng cách của các đỉnh 2, 4 và 5, có khoảng cách hiện tại là 5, 9 và 1.

Đỉnh tiếp theo được xử lý là đỉnh 5 với khoảng cách 1. Điều này giảm khoảng cách đến đỉnh 4 từ 9 xuống 3:

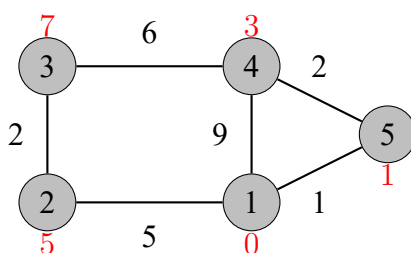


Sau đó, đỉnh tiếp theo là đỉnh 4, làm giảm khoảng cách đến đỉnh 3 xuống 9:



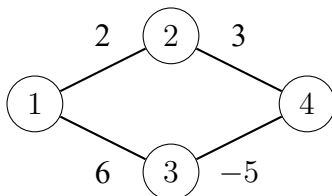
Một tính chất đáng chú ý của thuật toán Dijkstra là mỗi khi một đỉnh được chọn, khoảng cách của nó đã là cuối cùng. Ví dụ, tại thời điểm này của thuật toán, các khoảng cách 0, 1 và 3 là khoảng cách cuối cùng đến các đỉnh 1, 5 và 4.

Sau đó, thuật toán xử lý hai đỉnh còn lại, và các khoảng cách cuối cùng như sau:



Cạnh âm

Hiệu quả của thuật toán Dijkstra dựa trên thực tế rằng đồ thị không chứa cạnh âm. Nếu có một cạnh âm, thuật toán có thể cho kết quả sai. Như một ví dụ, xét đồ thị sau:



Đường đi ngắn nhất từ đỉnh 1 đến đỉnh 4 là $1 \rightarrow 3 \rightarrow 4$ và độ dài của nó là 1. Tuy nhiên, thuật toán Dijkstra tìm đường đi $1 \rightarrow 2 \rightarrow 4$ bằng cách đi theo các cạnh có trọng số nhỏ nhất. Thuật toán không tính đến việc trên đường đi kia, trọng số -5 bù cho trọng số lớn trước đó 6.

Cài đặt

Cài đặt sau của thuật toán Dijkstra tính các khoảng cách nhỏ nhất từ một đỉnh x đến các đỉnh khác của đồ thị. Đồ thị được lưu dưới dạng danh sách kề sao cho $\text{adj}[a]$ chứa một cặp (b, w) mỗi khi có một cạnh từ đỉnh a đến đỉnh b với trọng số w .

Một cài đặt hiệu quả của thuật toán Dijkstra yêu cầu có thể tìm hiệu quả đỉnh chưa xử lý có khoảng cách nhỏ nhất. Một cấu trúc dữ liệu phù hợp cho việc này là hàng đợi ưu tiên chứa các đỉnh được sắp theo khoảng cách của chúng. Dùng hàng đợi ưu tiên, đỉnh tiếp theo cần xử lý có thể được lấy trong thời gian logarit.

Trong đoạn mã sau, hàng đợi ưu tiên q chứa các cặp dạng $(-d, x)$, nghĩa là khoảng cách hiện tại đến đỉnh x là d . Mảng `distance` chứa khoảng cách đến mỗi đỉnh, và mảng `processed` cho biết một đỉnh đã được xử lý hay chưa. Ban đầu, khoảng cách là 0 đến x và ∞ đến tất cả đỉnh khác.

```
for (int i = 1; i <= n; i++) distance[i] = INF;
distance[x] = 0;
q.push({0,x});
while (!q.empty()) {
    int a = q.top().second; q.pop();
    if (processed[a]) continue;
    processed[a] = true;
    for (auto u : adj[a]) {
        int b = u.first, w = u.second;
        if (distance[a]+w < distance[b]) {
            distance[b] = distance[a]+w;
            q.push({-distance[b], b});
        }
    }
}
```

Lưu ý rằng hàng đợi ưu tiên chứa các khoảng cách *âm* đến các đỉnh. Lý do là phiên bản mặc định của hàng đợi ưu tiên C++ tìm các phần tử lớn nhất, trong khi ta muốn

tìm các phần tử nhỏ nhất. Bằng cách dùng khoảng cách âm, ta có thể trực tiếp dùng hàng đợi ưu tiên mặc định³. Cũng lưu ý rằng có thể có nhiều bản sao của cùng một đỉnh trong hàng đợi ưu tiên; tuy nhiên, chỉ bản sao có khoảng cách nhỏ nhất sẽ được xử lý.

Độ phức tạp thời gian của cài đặt trên là $O(n + m \log m)$, vì thuật toán đi qua tất cả đỉnh của đồ thị và với mỗi cạnh thêm nhiều nhất một khoảng cách vào hàng đợi ưu tiên.

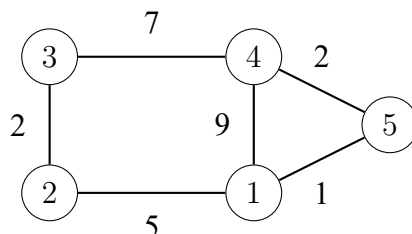
13.3 Thuật toán Floyd–Warshall

thuật toán Floyd–Warshall (Floyd–Warshall algorithm)⁴ cung cấp một cách tiếp cận khác cho bài toán tìm đường đi ngắn nhất. Khác với các thuật toán khác trong chương này, nó tìm tất cả đường đi ngắn nhất giữa các đỉnh trong một lần chạy.

Thuật toán duy trì một mảng hai chiều chứa khoảng cách giữa các đỉnh. Đầu tiên, các khoảng cách chỉ được tính bằng các cạnh trực tiếp giữa các đỉnh, và sau đó thuật toán giảm khoảng cách bằng cách dùng các đỉnh trung gian trong đường đi.

Ví dụ

Hãy xét cách thuật toán Floyd–Warshall hoạt động trong đồ thị sau:



Ban đầu, khoảng cách từ mỗi đỉnh đến chính nó là 0, và khoảng cách giữa các đỉnh a và b là x nếu có một cạnh giữa các đỉnh a và b với trọng số x . Tất cả khoảng cách khác là vô cùng.

Trong đồ thị này, mảng ban đầu như sau:

	1	2	3	4	5
1	0	5	∞	9	1
2	5	0	2	∞	∞
3	∞	2	0	7	∞
4	9	∞	7	0	2
5	1	∞	∞	2	0

³Dĩ nhiên, ta cũng có thể khai báo hàng đợi ưu tiên như trong Chương 4.5 và dùng khoảng cách dương, nhưng cài đặt sẽ dài hơn một chút.

⁴Thuật toán được đặt theo tên R. W. Floyd và S. Warshall, những người đã công bố nó một cách độc lập vào năm 1962 [23, 70].

Thuật toán gồm các vòng liên tiếp. Ở mỗi vòng, thuật toán chọn một đỉnh mới có thể đóng vai trò là đỉnh trung gian trong các đường đi từ thời điểm đó, và các khoảng cách được giảm bằng đỉnh này.

Ở vòng đầu tiên, đỉnh 1 là đỉnh trung gian mới. Có một đường đi mới giữa các đỉnh 2 và 4 có độ dài 14, vì đỉnh 1 nối chúng. Cũng có một đường đi mới giữa các đỉnh 2 và 5 có độ dài 6.

	1	2	3	4	5
1	0	5	∞	9	1
2	5	0	2	14	6
3	∞	2	0	7	∞
4	9	14	7	0	2
5	1	6	∞	2	0

Ở vòng thứ hai, đỉnh 2 là đỉnh trung gian mới. Điều này tạo ra các đường đi mới giữa các đỉnh 1 và 3 và giữa các đỉnh 3 và 5:

	1	2	3	4	5
1	0	5	7	9	1
2	5	0	2	14	6
3	7	2	0	7	8
4	9	14	7	0	2
5	1	6	8	2	0

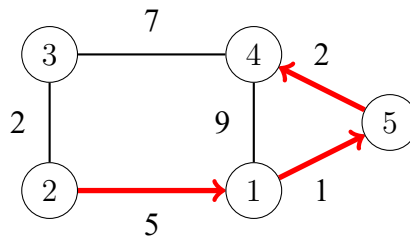
Ở vòng thứ ba, đỉnh 3 là đỉnh trung gian mới. Có một đường đi mới giữa các đỉnh 2 và 4:

	1	2	3	4	5
1	0	5	7	9	1
2	5	0	2	9	6
3	7	2	0	7	8
4	9	9	7	0	2
5	1	6	8	2	0

Thuật toán tiếp tục như vậy, cho đến khi tất cả các đỉnh đã được chọn làm đỉnh trung gian. Sau khi thuật toán kết thúc, mảng chứa khoảng cách nhỏ nhất giữa mọi cặp đỉnh:

	1	2	3	4	5
1	0	5	7	3	1
2	5	0	2	8	6
3	7	2	0	7	8
4	3	8	7	0	2
5	1	6	8	2	0

Ví dụ, mảng cho ta biết rằng khoảng cách ngắn nhất giữa các đỉnh 2 và 4 là 8. Điều này tương ứng với đường đi sau:



Cài đặt

Ưu điểm của thuật toán Floyd–Warshall là nó dễ cài đặt. Đoạn mã sau xây dựng một ma trận khoảng cách trong đó $\text{distance}[a][b]$ là khoảng cách ngắn nhất giữa các đỉnh a và b . Đầu tiên, thuật toán khởi tạo distance bằng ma trận kề adj của đồ thị:

```
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= n; j++) {
        if (i == j) distance[i][j] = 0;
        else if (adj[i][j]) distance[i][j] = adj[i][j];
        else distance[i][j] = INF;
    }
}
```

Sau đó, các khoảng cách ngắn nhất có thể được tìm như sau:

```
for (int k = 1; k <= n; k++) {
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            distance[i][j] = min(distance[i][j],
                                   distance[i][k] + distance[k][j]);
        }
    }
}
```

Độ phức tạp thời gian của thuật toán là $O(n^3)$, vì nó chứa ba vòng lặp lồng nhau đi qua các đỉnh của đồ thị.

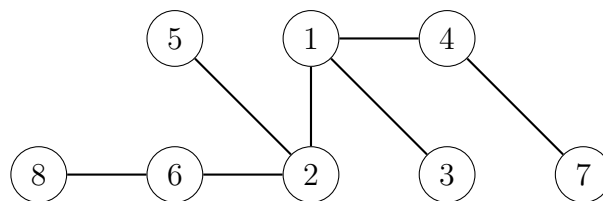
Vì cài đặt của thuật toán Floyd–Warshall đơn giản, thuật toán có thể là một lựa chọn tốt ngay cả khi chỉ cần tìm một đường đi ngắn nhất duy nhất trong đồ thị. Tuy nhiên, thuật toán chỉ có thể được dùng khi đồ thị đủ nhỏ để độ phức tạp bậc ba vẫn đủ nhanh.

Chương 14

Thuật toán trên cây

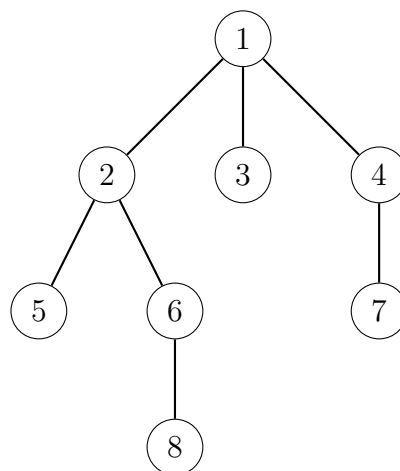
Một **cây (tree)** là một đồ thị liên thông, không có chu trình, gồm n đỉnh và $n - 1$ cạnh. Xóa bất kỳ cạnh nào khỏi một cây sẽ chia nó thành hai thành phần, và thêm bất kỳ cạnh nào vào một cây sẽ tạo ra một chu trình. Hơn nữa, luôn tồn tại đúng một đường đi giữa hai đỉnh bất kỳ của một cây.

Ví dụ, cây sau gồm 8 đỉnh và 7 cạnh:



Các **lá (leaves)** của một cây là các đỉnh có bậc 1, tức là chỉ có một đỉnh kề. Ví dụ, các lá của cây trên là các đỉnh 3, 5, 7 và 8.

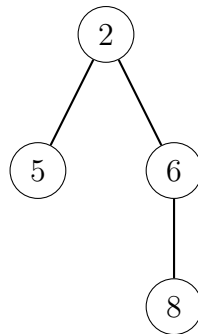
Trong một cây **có gốc (rooted)**, một trong các đỉnh được chọn làm **gốc (root)** của cây, và tất cả các đỉnh còn lại được đặt bên dưới gốc. Ví dụ, trong cây sau, đỉnh 1 là đỉnh gốc.



Trong một cây có gốc, các **con (children)** của một đỉnh là các đỉnh kề nằm bên dưới nó, còn **cha (parent)** của một đỉnh là đỉnh kề nằm phía trên nó. Mỗi đỉnh có đúng

một cha, ngoại trừ gốc không có cha. Ví dụ, trong cây trên, các con của đỉnh 2 là các đỉnh 5 và 6, và cha của nó là đỉnh 1.

Cấu trúc của một cây có gốc có tính *đệ quy (recursive)*: mỗi đỉnh của cây đóng vai trò là gốc của một **cây con (subtree)** chứa chính đỉnh đó và tất cả các đỉnh nằm trong các cây con của các con của nó. Ví dụ, trong cây trên, cây con của đỉnh 2 gồm các đỉnh 2, 5, 6 và 8:



14.1 Duyệt cây

Các thuật toán duyệt đồ thị tổng quát có thể được dùng để duyệt các đỉnh của một cây. Tuy nhiên, việc duyệt cây dễ cài đặt hơn so với duyệt một đồ thị tổng quát, vì trong cây không có chu trình và không thể đi tới một đỉnh từ nhiều hướng.

Cách thông dụng để duyệt một cây là bắt đầu một tìm kiếm theo chiều sâu từ một đỉnh bất kỳ. Có thể dùng hàm đệ quy sau:

```
void dfs(int s, int e) {  
    // process node s  
    for (auto u : adj[s]) {  
        if (u != e) dfs(u, s);  
    }  
}
```

Hàm nhận hai tham số: đỉnh hiện tại s và đỉnh trước đó e . Mục đích của tham số e là bảo đảm rằng quá trình tìm kiếm chỉ di chuyển tới các đỉnh chưa được thăm.

Lời gọi hàm sau bắt đầu tìm kiếm tại đỉnh x :

```
dfs(x, 0);
```

Trong lời gọi đầu tiên, $e = 0$, vì không có đỉnh trước đó, và ta được phép đi theo bất kỳ hướng nào trong cây.

Quy hoạch động

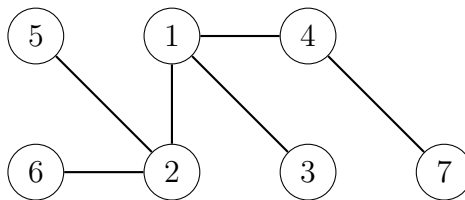
Quy hoạch động có thể được dùng để tính một số thông tin trong khi duyệt cây. Chẳng hạn, dùng quy hoạch động, ta có thể tính trong thời gian $O(n)$ cho mỗi đỉnh của một cây có gốc số đỉnh trong cây con của nó hoặc độ dài đường đi dài nhất từ đỉnh đó tới một lá.

Lấy ví dụ, hãy tính cho mỗi đỉnh s một giá trị $\text{count}[s]$: số đỉnh trong cây con của nó. Cây con chứa chính đỉnh đó và tất cả các đỉnh trong các cây con của các con của nó, vì vậy ta có thể tính số đỉnh một cách đệ quy bằng đoạn mã sau:

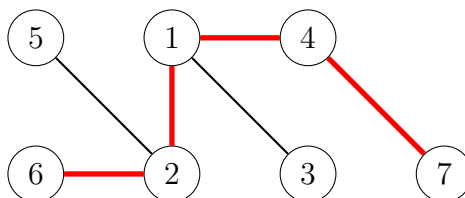
```
void dfs(int s, int e) {
    count[s] = 1;
    for (auto u : adj[s]) {
        if (u == e) continue;
        dfs(u, s);
        count[s] += count[u];
    }
}
```

14.2 Đường kính

Đường kính (diameter) của một cây là độ dài lớn nhất của một đường đi giữa hai đỉnh. Ví dụ, xét cây sau:



Đường kính của cây này là 4, tương ứng với đường đi sau:



Lưu ý rằng có thể có nhiều đường đi có độ dài lớn nhất. Trong đường đi trên, ta có thể thay đỉnh 6 bằng đỉnh 5 để thu được một đường đi khác có độ dài 4.

Tiếp theo, ta sẽ thảo luận hai thuật toán thời gian $O(n)$ để tính đường kính của một cây. Thuật toán thứ nhất dựa trên quy hoạch động, còn thuật toán thứ hai dùng hai lần tìm kiếm theo chiều sâu.

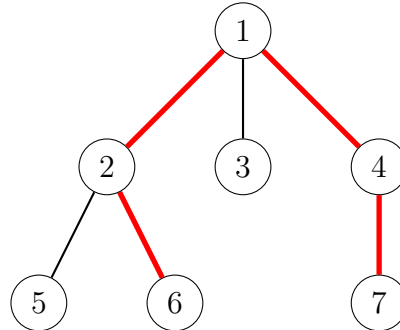
Thuật toán 1

Một cách tiếp cận tổng quát cho nhiều bài toán trên cây là trước hết gán gốc cho cây một cách tùy ý. Sau đó, ta có thể thử giải bài toán riêng rẽ cho từng cây con. Thuật toán đầu tiên của ta để tính đường kính dựa trên ý tưởng này.

Một nhận xét quan trọng là mọi đường đi trong một cây có gốc đều có một *điểm cao nhất*: đỉnh cao nhất thuộc đường đi đó. Do đó, ta có thể tính cho mỗi đỉnh độ dài

của đường đi dài nhất có điểm cao nhất là đỉnh đó. Một trong các đường đi này tương ứng với đường kính của cây.

Ví dụ, trong cây sau, đỉnh 1 là điểm cao nhất trên đường đi tương ứng với đường kính:



Ta tính cho mỗi đỉnh x hai giá trị:

- $\text{toLeaf}(x)$: độ dài lớn nhất của một đường đi từ x tới một lá bất kỳ
- $\text{maxLength}(x)$: độ dài lớn nhất của một đường đi có điểm cao nhất là x

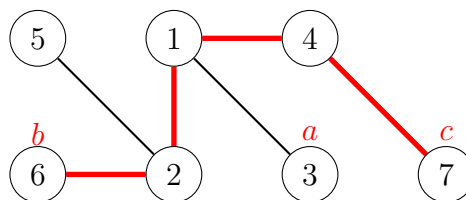
Ví dụ, trong cây trên, $\text{toLeaf}(1) = 2$, vì có đường đi $1 \rightarrow 2 \rightarrow 6$, và $\text{maxLength}(1) = 4$, vì có đường đi $6 \rightarrow 2 \rightarrow 1 \rightarrow 4 \rightarrow 7$. Trong trường hợp này, $\text{maxLength}(1)$ bằng đường kính.

Có thể dùng quy hoạch động để tính các giá trị trên cho mọi đỉnh trong thời gian $O(n)$. Trước hết, để tính $\text{toLeaf}(x)$, ta duyệt qua các con của x , chọn một con c có $\text{toLeaf}(c)$ lớn nhất và cộng thêm một vào giá trị này. Sau đó, để tính $\text{maxLength}(x)$, ta chọn hai con phân biệt a và b sao cho tổng $\text{toLeaf}(a) + \text{toLeaf}(b)$ là lớn nhất rồi cộng thêm hai vào tổng này.

Thuật toán 2

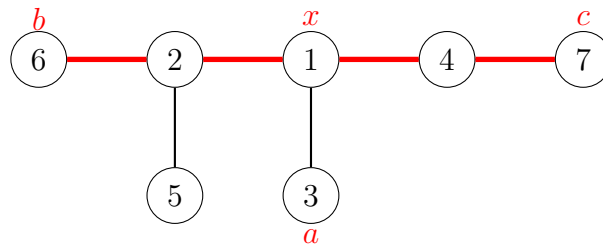
Một cách hiệu quả khác để tính đường kính của một cây dựa trên hai lần tìm kiếm theo chiều sâu. Trước hết, ta chọn một đỉnh tùy ý a trong cây và tìm đỉnh b xa nhất từ a . Sau đó, ta tìm đỉnh c xa nhất từ b . Đường kính của cây là khoảng cách giữa b và c .

Trong đồ thị sau, a , b và c có thể là:



Đây là một phương pháp đẹp, nhưng vì sao nó đúng?

Ta có thể hiểu dễ hơn nếu vẽ cây theo cách khác sao cho đường đi tương ứng với đường kính nằm ngang, còn tất cả các đỉnh khác treo xuống từ đường đi này:

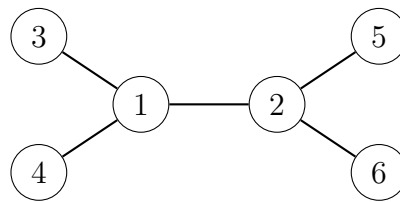


Đỉnh x chỉ vị trí mà đường đi từ đỉnh a nhập vào đường đi tương ứng với đường kính. Đỉnh xa nhất từ a là đỉnh b , đỉnh c hoặc một đỉnh khác có khoảng cách tới đỉnh x ít nhất cũng lớn như vậy. Do đó, đỉnh này luôn là một lựa chọn hợp lệ làm đầu mút của một đường đi tương ứng với đường kính.

14.3 Tất cả các đường đi dài nhất

Bài toán tiếp theo là tính cho mọi đỉnh trong cây độ dài lớn nhất của một đường đi bắt đầu tại đỉnh đó. Bài toán này có thể được xem là một tổng quát hóa của bài toán đường kính cây, vì giá trị lớn nhất trong các độ dài đó bằng đường kính của cây. Bài toán này cũng có thể được giải trong thời gian $O(n)$.

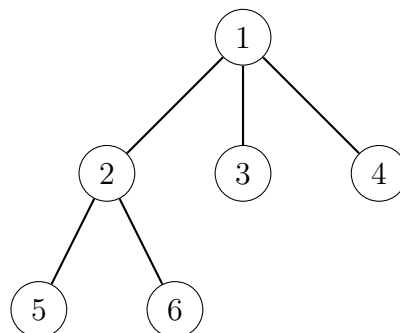
Lấy ví dụ, xét cây sau:



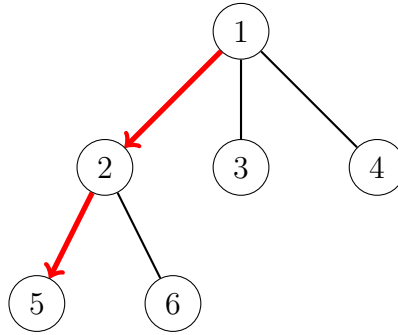
Kí hiệu $\text{maxLength}(x)$ là độ dài lớn nhất của một đường đi bắt đầu tại đỉnh x . Ví dụ, trong cây trên, $\text{maxLength}(4) = 3$, vì có đường đi $4 \rightarrow 1 \rightarrow 2 \rightarrow 6$. Bảng đầy đủ các giá trị như sau:

đỉnh x	1	2	3	4	5	6
$\text{maxLength}(x)$	2	2	3	3	3	3

Trong bài toán này, một điểm khởi đầu tốt để giải bài toán cũng là gán gốc cho cây một cách tùy ý:

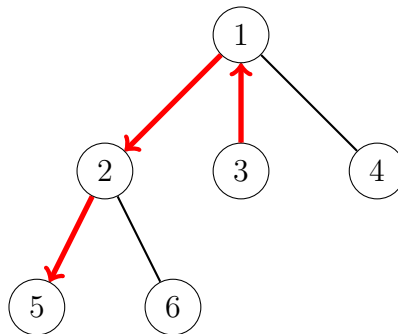


Phân đầu tiên của bài toán là tính cho mỗi đỉnh x độ dài lớn nhất của một đường đi đi qua một con của x . Ví dụ, đường đi dài nhất từ đỉnh 1 đi qua con 2 của nó:

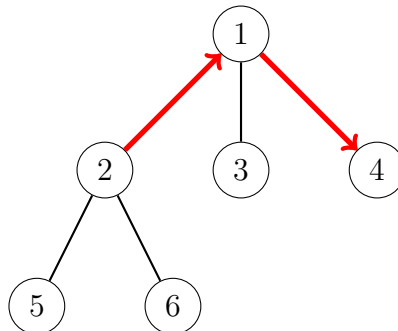


Phần này dễ giải trong thời gian $O(n)$, vì ta có thể dùng quy hoạch động như đã làm trước đó.

Sau đó, phần thứ hai của bài toán là tính cho mỗi đỉnh x độ dài lớn nhất của một đường đi đi qua cha p của nó. Ví dụ, đường đi dài nhất từ đỉnh 3 đi qua cha 1 của nó:



Thoạt nhìn, có vẻ như ta nên chọn đường đi dài nhất từ p . Tuy nhiên, cách này *không* phải lúc nào cũng đúng, vì đường đi dài nhất từ p có thể đi qua x . Sau đây là một ví dụ cho tình huống này:



Tuy vậy, ta vẫn có thể giải phần thứ hai trong thời gian $O(n)$ bằng cách lưu *hai* độ dài lớn nhất cho mỗi đỉnh x :

- $\text{maxLength}_1(x)$: độ dài lớn nhất của một đường đi từ x
- $\text{maxLength}_2(x)$ độ dài lớn nhất của một đường đi từ x theo một hướng khác với đường đi thứ nhất

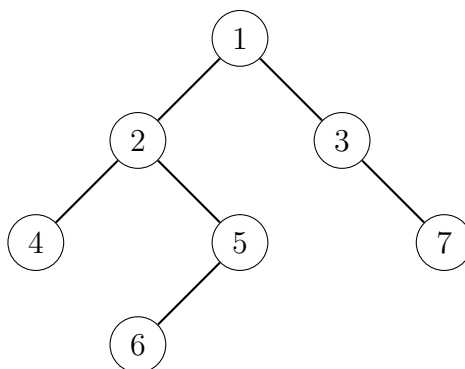
Ví dụ, trong đồ thị trên, $\text{maxLength}_1(1) = 2$ khi dùng đường đi $1 \rightarrow 2 \rightarrow 5$, và $\text{maxLength}_2(1) = 1$ khi dùng đường đi $1 \rightarrow 3$.

Cuối cùng, nếu đường đi tương ứng với $\text{maxLength}_1(p)$ đi qua x , ta kết luận rằng độ dài lớn nhất là $\text{maxLength}_2(p) + 1$, còn nếu không thì độ dài lớn nhất là $\text{maxLength}_1(p) + 1$.

14.4 Cây nhị phân

Một **cây nhị phân (binary tree)** là một cây có gốc trong đó mỗi đỉnh có một cây con trái và một cây con phải. Có thể một cây con của một đỉnh là rỗng. Do đó, mỗi đỉnh trong một cây nhị phân có không, một hoặc hai con.

Ví dụ, cây sau là một cây nhị phân:



Các đỉnh của một cây nhị phân có ba thứ tự tự nhiên tương ứng với các cách khác nhau để duyệt cây một cách đệ quy:

- **thứ tự trước (pre-order):** trước hết xử lý gốc, sau đó duyệt cây con trái, rồi duyệt cây con phải
- **thứ tự giữa (in-order):** trước hết duyệt cây con trái, sau đó xử lý gốc, rồi duyệt cây con phải
- **thứ tự sau (post-order):** trước hết duyệt cây con trái, sau đó duyệt cây con phải, rồi xử lý gốc

Với cây trên, các đỉnh theo thứ tự trước là $[1, 2, 4, 5, 6, 3, 7]$, theo thứ tự giữa là $[4, 2, 6, 5, 1, 3, 7]$ và theo thứ tự sau là $[4, 6, 5, 2, 7, 3, 1]$.

Nếu biết thứ tự trước và thứ tự giữa của một cây, ta có thể khôi phục chính xác cấu trúc của cây. Ví dụ, cây trên là cây duy nhất có thể có thứ tự trước $[1, 2, 4, 5, 6, 3, 7]$ và thứ tự giữa $[4, 2, 6, 5, 1, 3, 7]$. Tương tự, thứ tự sau và thứ tự giữa cũng xác định cấu trúc của một cây.

Tuy nhiên, tình huống sẽ khác nếu ta chỉ biết thứ tự trước và thứ tự sau của một cây. Trong trường hợp này, có thể có nhiều hơn một cây khớp với các thứ tự đó. Ví dụ, trong cả hai cây



thứ tự trước là $[1, 2]$ và thứ tự sau là $[2, 1]$, nhưng cấu trúc của hai cây là khác nhau.

Tài liệu tham khảo

- [1] A. V. Aho, J. E. Hopcroft and J. Ullman. *Data Structures and Algorithms*, Addison-Wesley, 1983.
- [2] R. K. Ahuja and J. B. Orlin. Distance directed augmenting path algorithms for maximum flow and parametric maximum flow problems. *Naval Research Logistics*, 38(3):413–430, 1991.
- [3] A. M. Andrew. Another efficient algorithm for convex hulls in two dimensions. *Information Processing Letters*, 9(5):216–219, 1979.
- [4] B. Aspvall, M. F. Plass and R. E. Tarjan. A linear-time algorithm for testing the truth of certain quantified boolean formulas. *Information Processing Letters*, 8(3):121–123, 1979.
- [5] R. Bellman. On a routing problem. *Quarterly of Applied Mathematics*, 16(1):87–90, 1958.
- [6] M. Beck, E. Pine, W. Tarrat and K. Y. Jensen. New integer representations as the sum of three cubes. *Mathematics of Computation*, 76(259):1683–1690, 2007.
- [7] M. A. Bender and M. Farach-Colton. The LCA problem revisited. In *Latin American Symposium on Theoretical Informatics*, 88–94, 2000.
- [8] J. Bentley. *Programming Pearls*. Addison-Wesley, 1999 (2nd edition).
- [9] J. Bentley and D. Wood. An optimal worst case algorithm for reporting intersections of rectangles. *IEEE Transactions on Computers*, C-29(7):571–577, 1980.
- [10] C. L. Bouton. Nim, a game with a complete mathematical theory. *Annals of Mathematics*, 3(1/4):35–39, 1901.
- [11] Croatian Open Competition in Informatics, <http://hsin.hr/coci/>
- [12] Codeforces: On "Mo's algorithm", <http://codeforces.com/blog/entry/20032>
- [13] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein. *Introduction to Algorithms*, MIT Press, 2009 (3rd edition).
- [14] E. W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1(1):269–271, 1959.

- [15] K. Diks et al. *Looking for a Challenge? The Ultimate Problem Set from the University of Warsaw Programming Competitions*, University of Warsaw, 2012.
- [16] M. Dima and R. Ceterchi. Efficient range minimum queries using binary indexed trees. *Olympiad in Informatics*, 9(1):39–44, 2015.
- [17] J. Edmonds. Paths, trees, and flowers. *Canadian Journal of Mathematics*, 17(3):449–467, 1965.
- [18] J. Edmonds and R. M. Karp. Theoretical improvements in algorithmic efficiency for network flow problems. *Journal of the ACM*, 19(2):248–264, 1972.
- [19] S. Even, A. Itai and A. Shamir. On the complexity of time table and multi-commodity flow problems. *16th Annual Symposium on Foundations of Computer Science*, 184–193, 1975.
- [20] D. Fanding. A faster algorithm for shortest-path – SPFA. *Journal of Southwest Jiaotong University*, 2, 1994.
- [21] P. M. Fenwick. A new data structure for cumulative frequency tables. *Software: Practice and Experience*, 24(3):327–336, 1994.
- [22] J. Fischer and V. Heun. Theoretical and practical improvements on the RMQ-problem, with applications to LCA and LCE. In *Annual Symposium on Combinatorial Pattern Matching*, 36–48, 2006.
- [23] R. W. Floyd Algorithm 97: shortest path. *Communications of the ACM*, 5(6):345, 1962.
- [24] L. R. Ford. Network flow theory. RAND Corporation, Santa Monica, California, 1956.
- [25] L. R. Ford and D. R. Fulkerson. Maximal flow through a network. *Canadian Journal of Mathematics*, 8(3):399–404, 1956.
- [26] R. Freivalds. Probabilistic machines can use less running time. In *IFIP congress*, 839–842, 1977.
- [27] F. Le Gall. Powers of tensors and fast matrix multiplication. In *Proceedings of the 39th International Symposium on Symbolic and Algebraic Computation*, 296–303, 2014.
- [28] M. R. Garey and D. S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*, W. H. Freeman and Company, 1979.
- [29] Google Code Jam Statistics (2017), <https://www.go-hero.net/jam/17>
- [30] A. Grønlund and S. Pettie. Threesomes, degenerates, and love triangles. In *Proceedings of the 55th Annual Symposium on Foundations of Computer Science*, 621–630, 2014.
- [31] P. M. Grundy. Mathematics and games. *Eureka*, 2(5):6–8, 1939.

- [32] D. Gusfield. *Algorithms on Strings, Trees and Sequences: Computer Science and Computational Biology*, Cambridge University Press, 1997.
- [33] S. Halim and F. Halim. *Competitive Programming 3: The New Lower Bound of Programming Contests*, 2013.
- [34] M. Held and R. M. Karp. A dynamic programming approach to sequencing problems. *Journal of the Society for Industrial and Applied Mathematics*, 10(1):196–210, 1962.
- [35] C. Hierholzer and C. Wiener. Über die Möglichkeit, einen Linienzug ohne Wiederholung und ohne Unterbrechung zu umfahren. *Mathematische Annalen*, 6(1), 30–32, 1873.
- [36] C. A. R. Hoare. Algorithm 64: Quicksort. *Communications of the ACM*, 4(7):321, 1961.
- [37] C. A. R. Hoare. Algorithm 65: Find. *Communications of the ACM*, 4(7):321–322, 1961.
- [38] J. E. Hopcroft and J. D. Ullman. A linear list merging algorithm. Technical report, Cornell University, 1971.
- [39] E. Horowitz and S. Sahni. Computing partitions with applications to the knapsack problem. *Journal of the ACM*, 21(2):277–292, 1974.
- [40] D. A. Huffman. A method for the construction of minimum-redundancy codes. *Proceedings of the IRE*, 40(9):1098–1101, 1952.
- [41] The International Olympiad in Informatics Syllabus, <https://people.ksp.sk/~misof/ioi-syllabus/>
- [42] R. M. Karp and M. O. Rabin. Efficient randomized pattern-matching algorithms. *IBM Journal of Research and Development*, 31(2):249–260, 1987.
- [43] P. W. Kasteleyn. The statistics of dimers on a lattice: I. The number of dimer arrangements on a quadratic lattice. *Physica*, 27(12):1209–1225, 1961.
- [44] C. Kent, G. M. Landau and M. Ziv-Ukelson. On the complexity of sparse exon assembly. *Journal of Computational Biology*, 13(5):1013–1027, 2006.
- [45] J. Kleinberg and É. Tardos. *Algorithm Design*, Pearson, 2005.
- [46] D. E. Knuth. *The Art of Computer Programming. Volume 2: Seminumerical Algorithms*, Addison–Wesley, 1998 (3rd edition).
- [47] D. E. Knuth. *The Art of Computer Programming. Volume 3: Sorting and Searching*, Addison–Wesley, 1998 (2nd edition).
- [48] J. B. Kruskal. On the shortest spanning subtree of a graph and the traveling salesman problem. *Proceedings of the American Mathematical Society*, 7(1):48–50, 1956.

- [49] V. I. Levenshtein. Binary codes capable of correcting deletions, insertions, and reversals. *Soviet physics doklady*, 10(8):707–710, 1966.
- [50] M. G. Main and R. J. Lorentz. An $O(n \log n)$ algorithm for finding all repetitions in a string. *Journal of Algorithms*, 5(3):422–432, 1984.
- [51] J. Pachocki and J. Radoszewski. Where to use and how not to use polynomial string hashing. *Olympiads in Informatics*, 7(1):90–100, 2013.
- [52] I. Parberry. An efficient algorithm for the Knight’s tour problem. *Discrete Applied Mathematics*, 73(3):251–260, 1997.
- [53] D. Pearson. A polynomial-time algorithm for the change-making problem. *Operations Research Letters*, 33(3):231–234, 2005.
- [54] R. C. Prim. Shortest connection networks and some generalizations. *Bell System Technical Journal*, 36(6):1389–1401, 1957.
- [55] 27-Queens Puzzle: Massively Parallel Enumeration and Solution Counting. <https://github.com/preusser/q27>
- [56] M. I. Shamos and D. Hoey. Closest-point problems. In *Proceedings of the 16th Annual Symposium on Foundations of Computer Science*, 151–162, 1975.
- [57] M. Sharir. A strong-connectivity algorithm and its applications in data flow analysis. *Computers & Mathematics with Applications*, 7(1):67–72, 1981.
- [58] S. S. Skiena. *The Algorithm Design Manual*, Springer, 2008 (2nd edition).
- [59] S. S. Skiena and M. A. Revilla. *Programming Challenges: The Programming Contest Training Manual*, Springer, 2003.
- [60] SZKOpuł, <https://szkopul.edu.pl/>
- [61] R. Sprague. Über mathematische Kampfspiele. *Tohoku Mathematical Journal*, 41:438–444, 1935.
- [62] P. Stańczyk. *Algorytmika praktyczna w konkursach Informatycznych*, MSc thesis, University of Warsaw, 2006.
- [63] V. Strassen. Gaussian elimination is not optimal. *Numerische Mathematik*, 13(4):354–356, 1969.
- [64] R. E. Tarjan. Efficiency of a good but not linear set union algorithm. *Journal of the ACM*, 22(2):215–225, 1975.
- [65] R. E. Tarjan. Applications of path compression on balanced trees. *Journal of the ACM*, 26(4):690–715, 1979.
- [66] R. E. Tarjan and U. Vishkin. Finding biconnected components and computing tree functions in logarithmic parallel time. In *Proceedings of the 25th Annual Symposium on Foundations of Computer Science*, 12–20, 1984.

- [67] H. N. V. Temperley and M. E. Fisher. Dimer problem in statistical mechanics – an exact result. *Philosophical Magazine*, 6(68):1061–1063, 1961.
- [68] USA Computing Olympiad, <http://www.usaco.org/>
- [69] H. C. von Warnsdorf. *Des Rösselsprunges einfachste und allgemeinste Lösung*. Schmalkalden, 1823.
- [70] S. Warshall. A theorem on boolean matrices. *Journal of the ACM*, 9(1):11–12, 1962.